PSOPT User Manual

Victor Becerra
Email: v.m.becerra@ieee.org
www: http://www.psopt.net
August 6, 2026

Disclaimer

This software is provided “as is” and is distributed free of charge. It comes with no warrantees of any kind. See the license terms for more details. The author does hope, however, that users will find this software useful for research and other purposes.

Contents

1. Introduction to $\mathcal{PSOPT}$

1.1 About $\mathcal{PSOPT}$

$\mathcal{PSOPT}$ is an open source optimal control package written in C++ that uses direct collocation methods. These methods solve optimal control problems by approximating the time-dependent variables using global or local polynomials. This allows to discretize the differential equations and continuous constraints over a grid of nodes, and to compute any integrals associated with the problem using well known quadrature formulas. Nonlinear programming then is used to find local optimal solutions. $\mathcal{PSOPT}$ has also been extended to use the integrated residuals transcription, a method that is particularly useful for problems that have singular arcs, stiff dynamics or non-smooth solutions. $\mathcal{PSOPT}$ is able to deal with problems with the following characteristics:

- Single or multiphase problems
- Continuous time nonlinear dynamics
- General endpoint constraints
- Nonlinear path constraints (equalities or inequalities) on states and/or control variables
- Integral constraints
- Interior point constraints
- Bounds on controls and state variables
- General cost function with Lagrange and Mayer terms.
- Free or fixed initial and final conditions
- Linear or nonlinear linkages between phases
- Fixed or free initial time
- Fixed or free final time
- Optimisation of static parameters

- Parameter estimation problems with sampled measurements
- Differential equations with delayed variables.

The implementation has the following features:

- Automatic scaling
- Automatic first and second derivatives using the CppAD library
- Numerical differentiation by using sparse finite differences
- Automatic mesh refinement
- Automatic identification of the Jacobian and Hessian sparsity.
- DAE formulation, so that differential and algebraic constraints can be implemented in the same C++ function.

$\mathcal{PSOPT}$ has interfaces to the following NLP solvers:

- IPOPT: an open source C++ implementation of an interior point method for large scale problems. See https://projects.coin-or.org/Ipopt for further details.
- SNOPT: is a Sequential Quadratic Programming algorithm for the optimisation of constrained large-scale problems. See http://www.sbsi-sol-optimize.com/manuals/SNOPT-Manual.pdf for further details.

1.1.1 Reasons to use $\mathcal{PSOPT}$

These are some reasons why users may wish to use $\mathcal{PSOPT}$:

- Users who for any reason do not have access to commercial optimal control solvers and wish to employ a free open source package for optimal control which does not need a proprietary software environment to run.
- Users who need to link an optimal control solver from stand alone applications written in C++ or other programming languages.
- Users who want to do research with the software, for instance by implementing their own problems, or by customising the code.

$\mathcal{PSOPT}$ does not require a commercial software environment to run on, or to be compiled. $\mathcal{PSOPT}$ is fully compatible with the `gcc` compiler, and has been developed under Linux, a free operating system. Note also that the default NLP solver (IPOPT) requires a sparse linear solver from a range of options, some of which are available at no cost. The author has personally used free linear solver `MUMPS`.

1.2 PSOPT user's group

A user's group has been created with the purpose of enabling users to share their experiences with using PSOPT , and to keep a public record of exchanges with the author. It is also a way of being informed about the latest developments with PSOPT and to ask for help. Membership is free and open. The PSOPT user's group is located at:

http://groups.google.com/group/psopt-users-group

1.2.1 About the author

Victor Becerra obtained his first degree in Electrical Engineering in 1990, and worked for two years in power systems analysis and control for a power generation and transmission company. He obtained his PhD for his work on the development of nonlinear optimal control methods from City University, London, in 1994. Between 1994 and 1999 he was a Research Fellow at the Control Engineering Research Centre at City University, London. Between 2000 and 2015 he was an academic at the School of Systems Engineering, University of Reading, UK, where he became a Professor of Automatic Control in 2012. Between 2011 and 2012, he was seconded at the Ford Motor Company in Dunton, Essex, with funding by the Royal Academy of Engineering, where he developed methods for the calibration of gasoline engine oil temperature dynamic models. In 2015, he took the position of Professor of Power Systems Engineering at the University of Portsmouth, UK. He is a Senior Member of the IEEE, a Senior Member of the AIAA, and a Fellow of the Institute of Engineering and Technology. During his career, he has received research funding from the EPSRC, the Royal Academy of Engineering, the European Union, the Knowledge Transfer Partnership programme, Innovate UK and UK industry. He has published over 150 research papers and two books. His web site is:

https://researchportal.port.ac.uk/en/persons/victor-becerra

1.2.2 Contacting the author

The author is open to discussing with users potential research collaboration leading to publications, academic exchanges, or joint projects. He can be contacted directly at his email address `v.m.becerra@ieee.org`.

1.2.3 How you can help

You may help improve PSOPT in a number of ways.

- Sending bug reports (bug tracking system: https://github.com/PSOPT/psopt/issues/)
- Sending corrections to the documentation, please use the above link.

- Discussing with the author ways to improve the computational aspects or capabilities of the software.
- Sending to the author proposed modifications to the source code, for consideration to be included in a future release of $\mathcal{PSOPT}$, usually through pull requests in GitHub.
- Sending source code with new examples which may be included (with due acknowledgement) in future releases of $\mathcal{PSOPT}$.
- Porting the software to new architectures.
- If you have had a good experience with $\mathcal{PSOPT}$, tell your students or colleagues about it.
- Quoting the use of $\mathcal{PSOPT}$ in your scientific publications. The recommended reference for $\mathcal{PSOPT}$ is:

  – Becerra, V.M. (2010). "Solving complex optimal control problems at no cost with PSOPT". *Proc. IEEE Multi-conference on Systems and Control*, Yokohama, Japan, September 7-10, 2010, pp. 1391-1396

  and the following is the recommended form to cite this document:

  – Becerra, V.M. (2020). *PSOPT Optimal Control Solver User Manual. Release 5*. Available: https://github.com/PSOPT/psopt/blob/master/doc/
- Developing interfaces to other NLP solvers.

1.3 Rolling Release

From March 2025, $\mathcal{PSOPT}$ will feature a rolling release mode. Rolling release is a concept in software development of frequently delivering updates to applications. This is in contrast to a standard or point release development model which uses software versions which replace the previous version. Users can download the latest source code from the GitHub repository. The documentation will also be updated on a rolling release basis.

1.4 Other documents in this set

The $\mathcal{PSOPT}$ documentation is now made of this document, which introduces the software, its methods and its usage. A second document provides details and results from a number of examples. Finally, the GitHub page provides the latest installation instructions.

1.5 GitHub repository and home page

The downloadable $\mathcal{PSOPT}$ distribution is held in its GitHub page:

https://github.com/PSOPT/psopt

A web page is maintained to provide general information about $\mathcal{PSOPT}$:

http://www.psopt.net

1.6 General problem formulation

$\mathcal{PSOPT}$ solves the following general optimal control problem with N_p phases:

Problem $\mathcal{P}_1$

Find the control trajectories, $u^{(i)}(t), t \in [t_0^{(i)}, t_f^{(i)}]$, state trajectories $x^{(i)}(t), t \in [t_0^{(i)}, t_f^{(i)}]$, static parameters $p^{(i)}$, and times $t_0^{(i)}, t_f^{(i)}$, $i = 1, \ldots, N_p$, to minimise the following performance index:

$$J = \sum_{i=1}^{N_p} \left[\varphi^{(i)}[x^{(i)}(t_f^{(i)}), p^{(i)}, t_f^{(i)}] + \int_{t_0^{(i)}}^{t_f^{(i)}} L^{(i)}[x^{(i)}(t), u^{(i)}(t), p^{(i)}, t]dt \right]$$

subject to the differential constraints:

$$\dot{x}^{(i)}(t) = f^{(i)}[x^{(i)}(t), u^{(i)}(t), p^{(i)}, t],\ t \in [t_0^{(i)}, t_f^{(i)}],$$

the path constraints

$$h_L^{(i)} \leq h^{(i)}[x^{(i)}(t), u^{(i)}(t), p^{(i)}, t] \leq h_U^{(i)},\ t \in [t_0^{(i)}, t_f^{(i)}],$$

the event constraints:

$$e_L^{(i)} \leq e^{(i)}[x^{(i)}(t_0^{(i)}), u^{(i)}(t_0^{(i)}), x^{(i)}(t_f^{(i)}), u^{(i)}(t_f^{(i)}), p^{(i)}, t_0^{(i)}, t_f^{(i)}] \leq e_U^{(i)},$$

the phase linkage constraints:

$$\begin{aligned}
\Psi_l \leq \Psi[&x^{(1)}(t_0^{(1)}), u^{(1)}(t_0^{(1)}),\\
&x^{(1)}(t_f^{(1)}), u^{(1)}(t_f^{(1)}), p^{(1)}, t_0^{(1)}, t_f^{(1)},\\
&x^{(2)}(t_0^{(2)}), u^{(2)}(t_0^{(2)})\\
&, x^{(2)}(t_f^{(2)}), u^{(2)}(t_f^{(2)}), p^{(2)}, t_0^{(2)}, t_f^{(2)},\\
&\vdots\\
&x^{(N_p)}(t_0^{(N_p)}), u^{(N_p)}(t_0^{(N_p)}),\\
&x^{(N_p)}(t_f^{(N_p)}), u^{(N_p)}(t_f^{(N_p)})), p^{(N_p)}, t_0^{(N_p)}, t_f^{(N_p)}] \leq \Psi_u
\end{aligned}$$

the bound constraints:

$$u_L^{(i)} \le u^i(t) \le u_U^{(i)},\ t \in [t_0^{(i)}, t_f^{(i)}],$$
$$x_L^{(i)} \le x^i(t) \le x_U^{(i)},\ t \in [t_0^{(i)}, t_f^{(i)}],$$
$$p_L^{(i)} \le p^{(i)} \le p_U^{(i)},$$
$$\underline{t}_0^{(i)} \le t_0^{(i)} \le \bar{t}_0^{(i)},$$
$$\underline{t}_f^{(i)} \le t_f^{(i)} \le \bar{t}_f^{(i)},$$

and the following constraints:

$$t_f^{(i)} - t_0^{(i)} \ge 0,$$

where $i = 1, \ldots, N_p$, and

$$\begin{aligned}
u^{(i)} &: [t_0^{(i)}, t_f^{(i)}] \to \mathcal{R}^{n_u^{(i)}} \\
x^{(i)} &: [t_0^{(i)}, t_f^{(i)}] \to \mathcal{R}^{n_x^{(i)}} \\
p^{(i)} &\in \mathcal{R}^{n_p^{(i)}} \\
\varphi^{(i)} &: \mathcal{R}^{n_x^{(i)}} \times \mathcal{R}^{n_x^{(i)}} \times \mathcal{R}^{n_p^{(i)}} \times \mathcal{R} \times \mathcal{R} \to \mathcal{R} \\
L^{(i)} &: \mathcal{R}^{n_x^{(i)}} \times \mathcal{R}^{n_u^{(i)}} \times \mathcal{R}^{n_p^{(i)}} \times [t_0^{(i)}, t_f^{(i)}] \to \mathcal{R} \\
f^{(i)} &: \mathcal{R}^{n_x^{(i)}} \times \mathcal{R}^{n_u^{(i)}} \times \mathcal{R}^{n_p^{(i)}} \times [t_0^{(i)}, t_f^{(i)}] \to \mathcal{R}^{n_x^{(i)}} \\
h^{(i)} &: \mathcal{R}^{n_x^{(i)}} \times \mathcal{R}^{n_u^{(i)}} \times \mathcal{R}^{n_p^{(i)}} \times [t_0^{(i)}, t_f^{(i)}] \to \mathcal{R}^{n_h^{(i)}} \\
e^{(i)} &: \mathcal{R}^{n_x^{(i)}} \times \mathcal{R}^{n_u^{(i)}} \times \mathcal{R}^{n_x^{(i)}} \times \mathcal{R}^{n_u^{(i)}} \times \Re^{n_p^{(i)}} \times \mathcal{R} \times \mathcal{R} \to \mathcal{R}^{n_e^{(i)}} \\
\Psi &: U_\Psi \to \mathcal{R}^{n_\psi}
\end{aligned} \tag{1.1}$$

where U_ψ is the domain of function Ψ.

A multiphase problem like $\mathcal{P}_1$ is defined and discussed in the book by Betts [4].

1.7 Overview of the discretisation methods employed by $\mathcal{PSOPT}$

$\mathcal{PSOPT}$ uses pseudospectral, local (trapezoidal and Hermite-Simpson) and integrated residual approaches to discretise and solve optimal control problems. Pseudospectral methods were originally developed for the solution of partial differential equations and have become a widely applied computational tool in fluid dynamics [9, 8]. Moreover, over the last 15 years or so, pseudospectral techniques have emerged as important computational methods for solving optimal control problems [13, 14, 16, 33, 21]. While finite difference methods approximate the derivatives of a function using local information, pseudospectral methods are, in contrast, global in the sense that they use information over samples of the whole domain of the function to approximate its derivatives at selected points. Using these methods, the state and control functions are approximated as a weighted sum of smooth basis functions, which are often chosen to be Legendre or Chebyshev polynomials in the interval $[-1, 1]$, and collocation of the differential-algebraic equations is performed at orthogonal collocation points, which are selected to yield interpolation of high accuracy. One of the main appeals of pseudospectral methods is their exponential (or spectral) rate of convergence, which is faster than any polynomial rate. Another advantage is that with relatively coarse grids it is possible to achieve good accuracy [38]. In cases where global collocation is unsuitable (for example, when the solution exhibits discontinuities), multi-domain pseudospectral techniques have been proposed, where the problem is divided into a number of subintervals and global collocation is performed along each subinterval [8].

Pseudospectral methods directly discretize the original optimal control problem to formulate a nonlinear programming problem, which is then solved numerically using a sparse nonlinear programming solver to find approximate local optimal solutions. Approximation theory and practice shows that pseudospectral methods are well suited for approximating smooth functions, integrations, and differentiations [7, 38], all of which are relevant to optimal control problems. For differentiation, the derivatives of the state functions at the discretisation nodes are easily computed by multiplying a constant differentiation matrix by a matrix with the state values at the nodes. Thus, the differential equations of the optimal control problem are approximated by a set of algebraic equations. The integration in the cost functional of an optimal control problem is approximated by well known Gauss quadrature rules, consisting of a weighted sum of the function values at the discretisation nodes. Moreover, as is the case with other direct methods for optimal control, it is easy to represent state and control dependent constraints.

The Legendre pseudospectral method for optimal control problems was originally proposed by Elnagar and co-workers in 1995 [13]. Since then, authors such as Ross, Fahroo and co-workers have analysed, extended and applied the method. For instance, convergence analysis is presented in [22], while an extension of the method to multi-phase problems is given in [33]. An application that has received publicity is the use of the Legendre pseudospectral method for generating real time trajectories for a NASA spacecraft maneouvre [21]. The Chebyshev pseudospectral method for optimal control problems was originally proposed in 1988 [39]. Fahroo and Ross proposed an alternative method for trajectory optimisation using Chebyshev polynomials [16]. The Gauss pseudospectral method, in which the dynamics are collocated at the interior Legendre–Gauss points, was developed by Benson and co-workers [2, 3], who also established a costate estimation procedure relating the multipliers of the discretised problem to the continuous costates. The Radau pseudospectral method collocates at the Legendre–Gauss–Radau points, one of which lies at an endpoint of the interval; convergence rates for Radau collocation were established by Kameswaran and Biegler [20], and the method was developed for both finite- and infinite-horizon problems, together with the associated costate estimation, by Garg, Rao and co-workers [18]. A unified framework relating the Gauss, Radau and

Lobatto (Legendre–Gauss–Lobatto) pseudospectral schemes was subsequently presented in [17].

Some details on approximating continuous functions using Legendre and Chebyshev polynomials are given below. Interested readers are referred to [7] for further details.

$\mathcal{PSOPT}$ implements several global pseudospectral discretisations: the Legendre (LGL) and Chebyshev (CGL) methods described in this chapter, together with the Legendre–Gauss–Radau (LGR) and Legendre–Gauss (LG) methods of Section 1.10. $\mathcal{PSOPT}$ is not restricted to pseudospectral methods: it also provides the local discretisations of Section 1.11, including the trapezoidal and Hermite-Simpson collocation methods. In addition to these collocation methods, $\mathcal{PSOPT}$ offers an integrated-residual transcription, described in Chapter 3, which measures (and optionally bounds) the dynamic error between the nodes and is particularly useful for problems with singular arcs, stiff dynamics or non-smooth solutions.

1.8 Pseudospectral approximations

1.8.1 Interpolation and the Lagrange polynomial

It is a well known fact in numerical analysis [6] that if $\tau_0, \tau_1, \ldots, \tau_N$ are $N+1$ distinct numbers and f is a function whose values are given at those numbers, then a unique polynomial $P(\tau)$ of degree at most N exists with

$$f(\tau_k) = P(\tau_k), \;\; \text{for } k = 0, 1, \ldots, N$$

This polynomial is given by:

$$P(\tau) = \sum_{k=0}^{N} f(\tau_k)\mathcal{L}_k(\tau)$$

where

$$\mathcal{L}_k(\tau) = \prod_{i=0, i\neq k}^{N} \frac{\tau - \tau_i}{\tau_k - \tau_i} \tag{1.2}$$

$P(\tau)$ is known as the Lagrange interpolating polynomial and $\mathcal{L}_k(\tau)$ are known as Lagrange basis polynomials.

1.8.2 Polynomial expansions

Assume that $\{p_k\}_{k=0,1,\ldots}$ is a system of algebraic polynomials, with degree of $p_k = k$, that are mutually orthogonal over the interval $[-1, 1]$ with respect to a weight function w:

$$\int_{-1}^{1} p_k(\tau)p_m(\tau)w(\tau)\mathrm{d}\tau = 0, \;\; \text{for } m \neq k$$

Define $L^2_w[-1,1]$ as the space of functions where the norm:

$$||v||_w = \left(\int_{-1}^{1} |v(\tau)|^2 w(\tau) \mathrm{d}\tau\right)^{1/2}$$

is finite. A function $f \in L^2_w[-1,1]$ in terms of the system $\{p_k\}$ can be represented as a series expansion:

$$f(\tau) = \sum_{k=0}^{\infty} \hat{f}_k p_k(\tau)$$

where the coefficients of the expansion are given by:

$$\hat{f}_k = \frac{1}{||p_k||^2} \int_{-1}^{1} f(\tau) p_k(\tau) w(\tau) \mathrm{d}\tau \tag{1.3}$$

The truncated expansion of f for a given N is:

$$\mathcal{P}_N f(\tau) = \sum_{k=0}^{N} \hat{f}_k p_k(\tau)$$

This type of expansion is at the heart of spectral and pseudospectral methods.

1.8.3 Legendre polynomials and numerical quadrature

A particular class of orthogonal polynomials are the Legendre polynomials, which are the eigenfunctions of a singular Sturm-Liouville problem [7]. Let $L_N(\tau)$ denote the Legendre polynomial of order N, which may be generated from:

$$L_N(\tau) = \frac{1}{2^N N!} \frac{d^N}{d\tau^N} (\tau^2 - 1)^N$$

Legendre polynomials are orthogonal over [-1,1] with the weight function $w = 1$. Examples of Legendre polynomials are:

$$\begin{aligned} L_0(\tau) &= 1 \\ L_1(\tau) &= \tau \\ L_2(\tau) &= \frac{1}{2}(3\tau^2 - 1) \\ L_3(\tau) &= \frac{1}{2}(5\tau^3 - 3\tau) \end{aligned}$$

Figure 1.1 illustrates the Legendre polynomials $L_N(\tau)$ for $N = 0, 1, 2, 4, 5, 10$.

Let τ_k, $k = 0, \ldots, N$ be the Lagrange-Gauss-Lobatto (LGL) nodes, which are defined as $\tau_0 = -1$, $\tau_N = 1$, and τ_k, being the roots of $\dot{L}_N(\tau)$ in the interval $[-1, 1]$ for $k = 1, 2, \ldots, N-1$. There are no explicit formulas to compute the roots of $\dot{L}_N(\tau)$, but they can be computed using known numerical algorithms. For example, for $N = 20$, the LGL nodes $\tau_k, k = 0, \ldots, 20$ are shown in Figure 1.2.

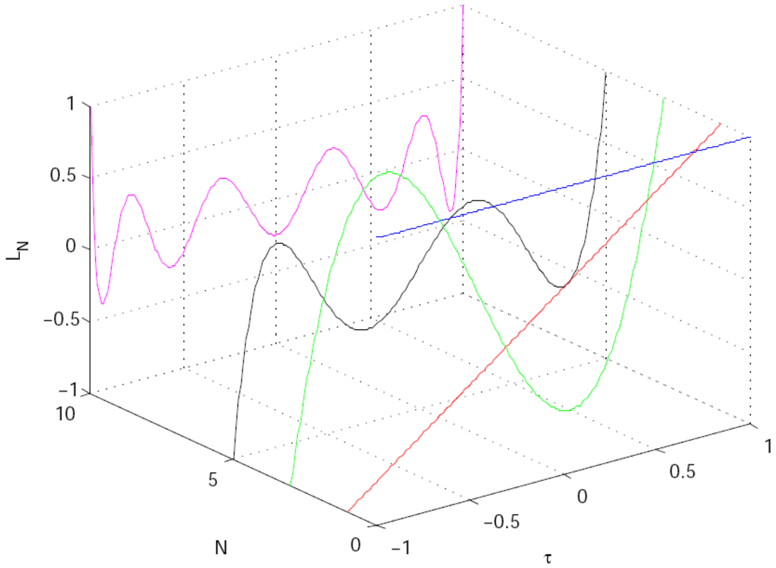

Figure 1.1: Illustration of the Legendre polynomials $L_N(\tau)$ for $N = 0, 1, 2, 4, 5, 10$.

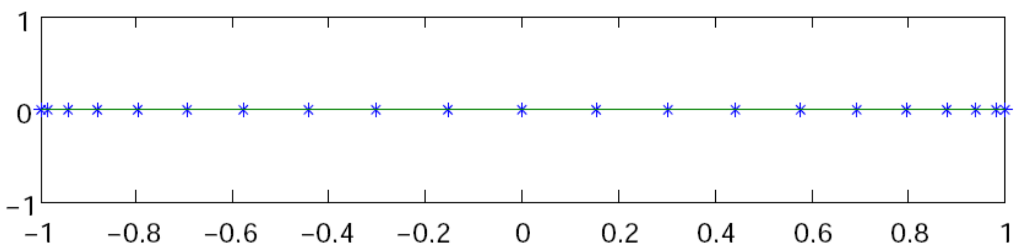

Figure 1.2: Illustration of the Legendre Gauss Lobatto (LGL) nodes for $N = 20$.

Note that if $h(\tau)$ is a polynomial of degree $\leq 2N-1$, its integral over $\tau \in [-1,1]$ can be exactly computed as follows:

$$\int_{-1}^{1} h(\tau)d\tau = \sum_{k=0}^{N} h(\tau_k)w_k \tag{1.4}$$

where τ_k, $k = 0,\ldots,N$ are the LGL nodes and the weights w_k are given by:

$$w_k = \frac{2}{N(N+1)} \frac{1}{[L_N(\tau_k)]^2}, \ \ k = 0,\ldots,N. \tag{1.5}$$

If $L(\tau)$ is a general smooth function, then for a suitable N, its integral over $\tau \in [-1,1]$ can be approximated as follows:

$$\int_{-1}^{1} L(\tau)d\tau \approx \sum_{k=0}^{N} L(\tau_k)w_k \tag{1.6}$$

The LGL nodes are selected to yield highly accurate numerical integrals. For example, consider the definite integral

$$\int_{-1}^{1} e^t \cos(t)dt$$

The exact value of this integral to 7 decimal places is 1.9334214. For $N = 3$ we have $\tau = [-1, -0.4472, 0.4472, 1]$, $w = [0.1667, 0.8333, 0.8333, 0.1667]$, hence

$$\int_{-1}^{1} e^t \cos(t)dt \approx w^T h(\tau) = 1.9335$$

so that the error is $\mathcal{O}(10^{-5})$. On the other hand, if $N = 5$, then the approximate value is 1.9334215, so that the error is $\mathcal{O}(10^{-7})$.

1.8.4 Interpolation and Legendre polynomials

The Legendre-Gauss-Lobatto quadrature motivates the following expression to approximate the weights of the expansion (1.3):

$$\hat{f}_k \approx \tilde{f}_k = \frac{1}{\gamma_k} \sum_{j=0}^{N} f(\tau_j)L_k(\tau_j)w_j$$

where

$$\gamma_k = \sum_{j=0}^{N} L_k^2(\tau_j)w_j$$

It is simple to prove (see [19]) that with these weights, function $f : [-1, 1] \rightarrow \Re$ can be interpolated over the LGL nodes as a discrete expansion using Legendre polynomials:

$$I_N f(\tau) = \sum_{k=0}^{N} \tilde{f}_k L_k(\tau) \tag{1.7}$$

such that

$$I_N f(\tau_j) = f(\tau_j) \tag{1.8}$$

Because $I_N f(\tau)$ is an interpolant of $f(\tau)$ at the LGL nodes, and since the interpolating polynomial is unique, we may express $I_N f(\tau)$ as a Lagrange interpolating polynomial:

$$I_N f(\tau) = \sum_{k=0}^{N} f(\tau_k)\mathcal{L}_k(\tau) \tag{1.9}$$

so that the expressions (1.7) and (1.9) are mathematically equivalent. Expression (1.9) is computationally advantageous since, as discussed below, it allows to express the approximate values of the derivatives of the function f at the nodes as a matrix multiplication. It is possible to write the Lagrange basis polynomials $\mathcal{L}_k(\tau)$ as follows [19]:

$$\mathcal{L}_k(\tau) = \frac{1}{N(N+1)L_N(\tau_k)} \frac{(\tau^2 - 1)\dot{L}_N(\tau)}{\tau - \tau_k}$$

The use of polynomial interpolation to approximate a function using the LGL points is known in the literature as the *Legendre pseudospectral approximation method.* Denote $f^N(\tau) = I_N f(\tau)$. Then, we have:

$$f(\tau) \approx f^N(\tau) = \sum_{k=0}^{N} f(\tau_k)\mathcal{L}_k(\tau) \tag{1.10}$$

It should be noted that $\mathcal{L}_k(\tau_j) = 1$ if $k = j$ and $\mathcal{L}_k(\tau_j)$=0, if $k \neq j$, so that:

$$f^N(\tau_k) = f(\tau_k) \tag{1.11}$$

Regarding the accuracy and error estimates of the Legendre pseudospectral approximation, it is well known that for smooth functions $f(\tau)$, the rate of convergence of $f^N(\tau)$ to $f(\tau)$ at the collocation points is faster than any power of $1/N$. The convergence of the pseudospectral approximations used by $\mathcal{PSOPT}$ has been analysed by Canuto *et al* [7].

Figure 1.3 shows the degree N interpolation of the function $f(\tau) = 1/(1+\tau+15\tau^2)$ in $(N+1)$ equispaced and LGL points for $N = 20$. With increasing N, the errors increase exponentially in the equispaced case (this is known as the Runge phenomenon) whereas in the LGL case they decrease exponentially.

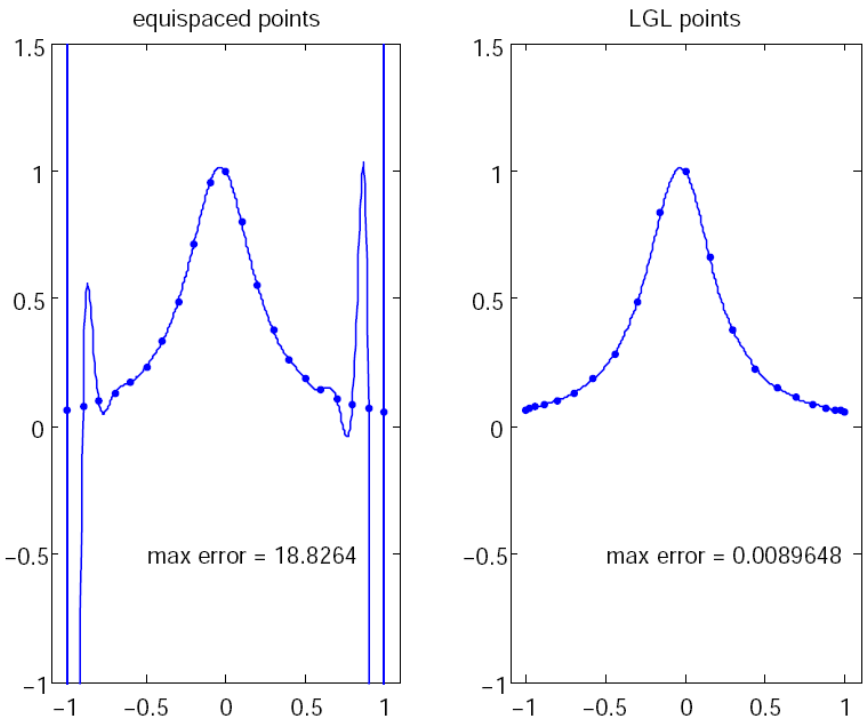

Figure 1.3: Illustration of polynomial interpolation over equispaced and LGL nodes

.

1.8.5 Approximate differentiation

The derivatives of $f^N(\tau)$ in terms of $f(\tau)$ at the LGL points τ_k can be obtained by differentiating Eqn. (1.10). The result can be expressed as a matrix multiplication, such that:

$$\dot{f}(\tau_k) \approx \dot{f}^N(\tau_k) = \sum_{i=0}^{N} D_{ki} f(\tau_i)$$

where

$$D_{ki} = \begin{cases} -\frac{L_N(\tau_k)}{L_N(\tau_i)} \frac{1}{\tau_k - \tau_i} & \text{if } k \neq i \\ N(N+1)/4 & \text{if } k = i = 0 \\ -N(N+1)/4 & \text{if } k = i = N \\ 0 & \text{otherwise} \end{cases} \tag{1.12}$$

which is known as the differentiation matrix.

For example, this is the Legendre differentiation matrix for $N = 5$.

$$D = \begin{bmatrix} 7.5000 & -10.1414 & 4.0362 & -2.2447 & 1.3499 & -0.5000 \\ 1.7864 & 0 & -2.5234 & 1.1528 & -0.6535 & 0.2378 \\ -0.4850 & 1.7213 & 0 & -1.7530 & 0.7864 & -0.2697 \\ 0.2697 & -0.7864 & 1.7530 & 0 & -1.7213 & 0.4850 \\ -0.2378 & 0.6535 & -1.1528 & 2.5234 & 0 & -1.7864 \\ 0.5000 & -1.3499 & 2.2447 & -4.0362 & 10.1414 & -7.5000 \end{bmatrix}$$

Figure 1.4 shows the Legendre differentiation of $f(t) = \sin(5t^2)$ for $N = 20$ and $N = 30$. Note the vertical scales in the error curves. Figure 1.5 shows the maximum error in the Legendre differentiation of $f(t) = \sin(5t^2)$ as a function of N. Notice that the error initially decreases very rapidly until such high precision is achieved (accuracy in the order of 10^{-12}) that round off errors due to the finite precision of the computer prevent any further reductions. This phenomenon is known as *spectral accuracy*.

1.8.6 Approximating a continuous function using Chebyshev polynomials

$\mathcal{PSOPT}$ also has facilities for pseudospectral function approximation using Chebyshev polynomials. Let $T_N(\tau)$ denote the Chebyshev polynomial of order N, which may be generated from:

$$T_N(\tau) = \cos(N \cos^{-1}(\tau)) \tag{1.13}$$

Let τ_k, $k = 0, \ldots, N$ be the Chebyshev-Gauss-Lobatto (CGL) nodes in the interval $[-1, 1]$, which are defined as $\tau_k = -\cos(\pi k/N)$ for $k = 0, 1, \ldots, N$.

Given any real-valued function $f(\tau) : [-1, 1] \to \Re$, it can be approximated by the Chebyshev pseudospectral method:

$$f(\tau) \approx f^N(\tau) = \sum_{k=0}^{N} f(\tau_k) \varphi_k(\tau) \tag{1.14}$$

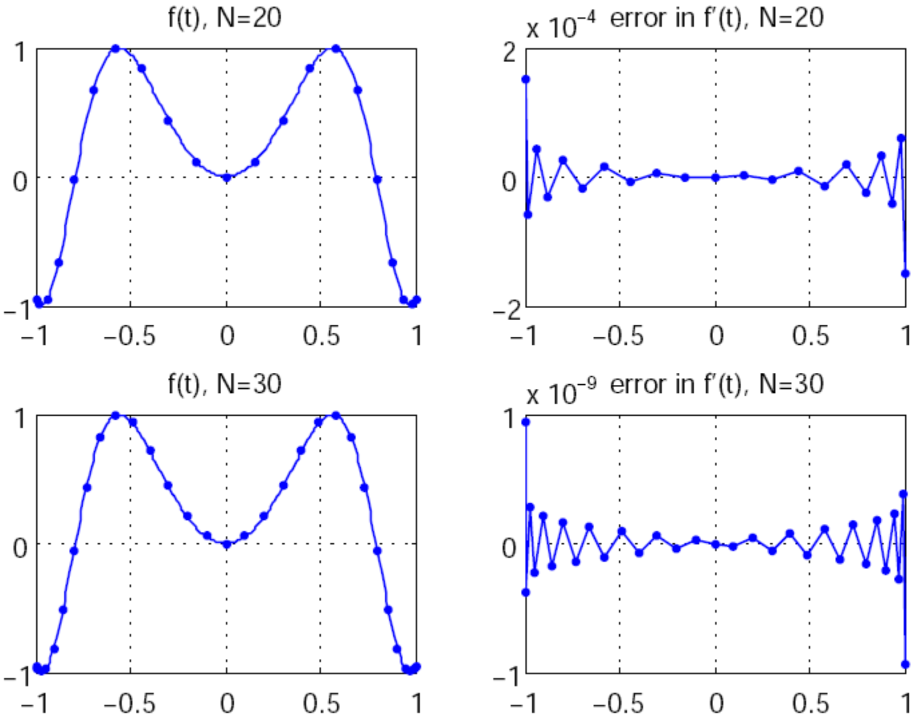

Figure 1.4: Legendre differentiation of $f(t) = \sin(5t^2)$ for $N = 20$ and $N = 30$.

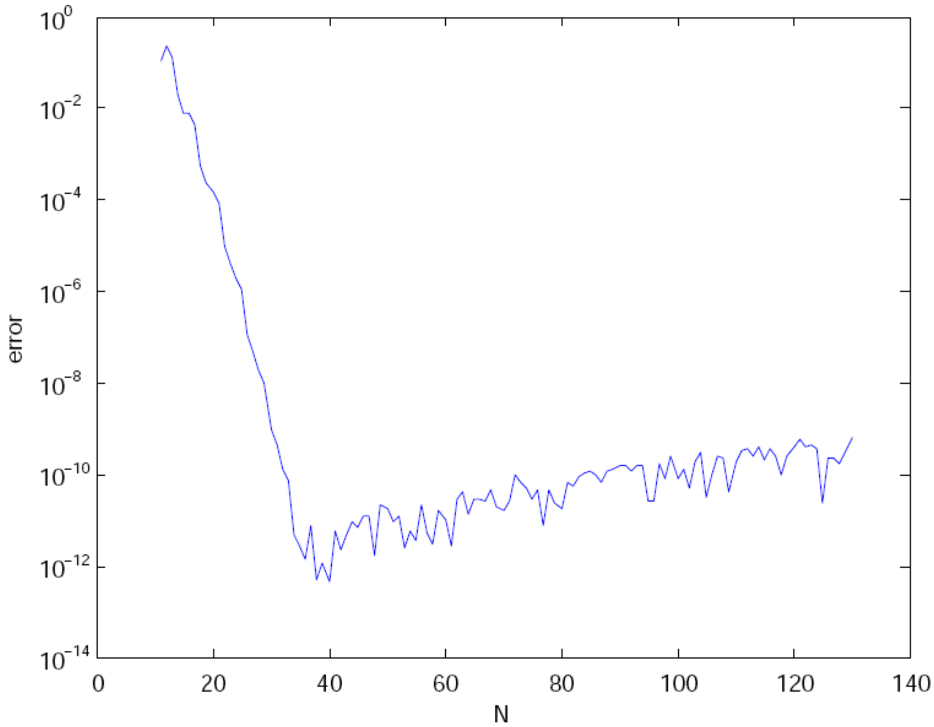

Figure 1.5: Maximum error in the Legendre differentiation of $f(t) = \sin(5t^2)$ as a function of N.

where the Lagrange interpolating polynomial $\varphi_k(\tau)$ is defined by:

$$\varphi_k(\tau) = \frac{(-1)^{k+1}}{N^2\bar{c}_k}\frac{(1-\tau^2)\dot{T}_N(\tau)}{\tau-\tau_k} \tag{1.15}$$

where

$$\bar{c}_k = \begin{cases} 2 & k=0,N \\ 1 & 1\leq k\leq N-1 \end{cases} \tag{1.16}$$

It should be noted that $\varphi_k(\tau_j) = 1$ if $k=j$ and $\varphi_k(\tau_j)$=0, if $k \neq j$, so that:

$$f^N(\tau_k) = f(\tau_k) \tag{1.17}$$

1.8.7 Differentiation with Chebyshev polynomials

The derivatives of $f^N(\tau)$ in terms of $f(\tau)$ at the CGL points τ_k can be obtained by differentiating (1.14). The result can be expressed as a matrix multiplication, such that:

$$\dot{f}(\tau_k) \approx \dot{F}^N(\tau_k) = \sum_{i=0}^{N} D_{ki} f(\tau_i) \tag{1.18}$$

where

$$D_{ki} = \begin{cases} \frac{\bar{c}_k}{2\bar{c}_i}\frac{(-1)^{k+i}}{\sin[(k+i)\pi/2N]\sin[(k-i)\pi/2N]} & \text{if } k\neq i \\ \frac{\tau_k}{2\sin^2[k\pi/N]} & \text{if } i\leq k=i\leq N-1 \\ -\frac{2N^2+1}{6} & \text{if } k=i=0 \\ \frac{2N^2+1}{6} & \text{if } k=i=N \end{cases} \tag{1.19}$$

which is known as the differentiation matrix.

1.8.8 Numerical quadrature at the Chebyshev-Gauss-Lobatto nodes

Integrals of the running cost and of other smooth integrands sampled at the Chebyshev–Gauss–Lobatto (CGL) nodes are evaluated in $\mathcal{PSOPT}$ using *Clenshaw–Curtis* quadrature [38]. For a general function $L(\tau)$ and a suitable N,

$$\int_{-1}^{1} L(\tau)\,d\tau \approx \sum_{k=0}^{N} w_k\, L(\tau_k), \tag{1.20}$$

where τ_k, $k=0,\ldots,N$ are the CGL nodes and the weights w_k are obtained from the discrete cosine transform of the sequence $2/(1-j^2)$ over the even indices j (the explicit form and a short implementation are given in [38]). The weights are symmetric and strictly positive, the rule is exact for polynomials of degree $\leq N$, and it converges spectrally (faster than any fixed algebraic order) for smooth integrands. Because the rule integrates the constant function exactly, the weights sum to the length of the interval,

$$\sum_{k=0}^{N} w_k = 2. \tag{1.21}$$

Unlike the classical Gauss–Lobatto rule associated with the Chebyshev weight $g(\tau) = 1/\sqrt{1-\tau^2}$, equation (1.20) carries unit weight and therefore applies directly to the integrand $L(\tau)$. Earlier versions of $\mathcal{PSOPT}$ used the weighted rule, which requires dividing the integrand by g before summation; since $1/g(\tau) = \sqrt{1-\tau^2}$ vanishes with infinite slope at $\tau = \pm 1$, the resulting product is not smooth at the endpoints and the discretised cost converged only algebraically. Clenshaw–Curtis quadrature removes this weight, restoring spectral accuracy of the cost and making the cost quadrature uniform with the other global pseudospectral methods. The same weights w_k enter the covector (costate) mapping for the Chebyshev method (Section 1.9.1).

1.8.9 Differentiation with reduced round-off errors

The following differentiation matrix, which offers reduced round-off errors [7], is employed optionally by $\mathcal{PSOPT}$. It can be used both with Legrendre and Chebyshed points.

$$D_{jl} = \begin{cases} -\frac{\delta_l}{\delta_j}\frac{(-1)^{j+l}}{\tau_j-\tau_l} & j \neq l \\ \sum\limits_{i=0, i\neq j}^{N} \frac{\delta_i}{\delta_j}\frac{(-1)^{i+j}}{\tau_j-\tau_i} & j = l \end{cases} \tag{1.22}$$

1.9 Legendre and Chebyshev pseudospectral discretisations

To illustrate the pseudospectral discretisations employed in $\mathcal{PSOPT}$, consider the following single phase continuous optimal control problem:

Problem $\mathcal{P}_2$

Find the control trajectories, $u(t), t \in [t_0, t_f]$, state trajectories $x(t), t \in [t_0, t_f]$, static parameters p, and times t_0, t_f, to minimise the following performance index:

$$J = \varphi[x(t_0), x(t_f), p, t_0, t_f] + \int_{t_0}^{t_f} L[x(t), u(t), p, t]dt$$

subject to the differential constraints:

$$\dot{x}(t) = f[x(t), u(t), p, t],\ t \in [t_0, t_f],$$

the path constraints

$$h_L \leq h[x(t), u(t), p, t] \leq h_U,\ t \in [t_0, t_f]$$

the event constraints:

$$e_L \leq e[x(t_0), u(t_0), x(t_f), u(t_f), p, t_0, t_f] \leq e_U,$$

the bound constraints on states and controls:

$$u_L \leq u(t) \leq u_U,\ t \in [t_0, t_f],$$

$$x_L \leq x(t) \leq x_U,\ t \in [t_0, t_f],$$

and the constraints:

$$\begin{aligned} \underline{t}_0 \leq t_0 \leq \bar{t}_0, \\ \underline{t}_f \leq t_f \leq \bar{t}_f, \\ t_f - t_0 \geq 0, \end{aligned}$$

where

$$\begin{aligned}
& u : [t_0, t_f] \to \mathcal{R}^{n_u} \\
& x : [t_0, t_f] \to \mathcal{R}^{n_x} \\
& p \in \mathcal{R}^{n_p} \\
& \varphi : \mathcal{R}^{n_x} \times \mathcal{R}^{n_x} \times \mathcal{R}^{n_p} \times \mathcal{R} \times \mathcal{R} \to \mathcal{R} \\
& L : \mathcal{R}^{n_x} \times \mathcal{R}^{n_u} \times \mathcal{R}^{n_p} \times [t_0, t_f] \to \mathcal{R} \\
& f : \mathcal{R}^{n_x} \times \mathcal{R}^{n_u} \times \mathcal{R}^{n_p} \times [t_0, t_f] \to \mathcal{R}^{n_x} \\
& h : \mathcal{R}^{n_x} \times \mathcal{R}^{n_u} \times \mathcal{R}^{n_p} \times [t_0, t_f] \to \mathcal{R}^{n_h} \\
& e : \mathcal{R}^{n_x} \times \mathcal{R}^{n_u} \times \mathcal{R}^{n_x} \times \mathcal{R}^{n_u} \times \Re^{n_p} \times \mathcal{R} \times \mathcal{R} \to \mathcal{R}^{n_e}
\end{aligned} \tag{1.23}$$

By introducing the transformation:

$$\tau \leftarrow \frac{2}{t_f - t_0} t - \frac{t_f + t_0}{t_f - t_0},$$

it is possible to write problem $\mathcal{P}_3$ using a new independent variable τ in the interval $[-1, 1]$, as follows:

Problem $\mathcal{P}_3$

Find the control trajectories, $u(\tau), \tau \in [-1, 1]$, state trajectories $x(\tau), \tau \in [-1, 1]$, and times t_0, t_f, to minimise the following performance index:

$$J = \varphi[x(-1), x(1), p, t_0, t_f] + \frac{t_f - t_0}{2} \int_{-1}^{1} L[x(\tau), u(\tau), p, \tau] d\tau$$

subject to the differential constraints:

$$\dot{x}(\tau) = \frac{t_f - t_0}{2} f[x(\tau), u(\tau), p, \tau],\ \tau \in [-1, 1],$$

the path constraints

$$h_L \leq h[x(\tau), u(\tau), p, \tau] \leq h_U,\ \tau \in [-1, 1]$$

the event constraints:

$$e_L \le e[x(-1), u(-1), x(1), u(1), p, t_0, t_f] \le e_U,$$

the bound constraints on controls and states:

$$u_L \le u(\tau) \le u_U,\ \tau \in [-1, 1],$$

$$x_L \le x(\tau) \le x_U,\ \tau \in [-1, 1],$$

and the constraints:

$$\underline{t}_0 \le t_0 \le \bar{t}_0,$$

$$\underline{t}_f \le t_f \le \bar{t}_f,$$

$$t_f - t_0 \ge 0,$$

The description below refers to the Legendre pseudospectral approximation method. The procedure employed with the Chebyshev approximation method is very similar. In the Legendre pseudospectral approximation of problem $\mathcal{P}_3$, the state $x(\tau)$, $\tau \in [-1, 1]$ is approximated by the N-order Lagrange polynomial $x^N(\tau)$ based on interpolation at the Legendre-Gauss-Lobatto (LGL) quadrature nodes, so that:

$$x(\tau) \approx x^N(\tau) = \sum_{k=0}^{N} x(\tau_k)\phi_k(\tau) \tag{1.24}$$

Moreover, the control $u(\tau)$, $\tau \in [-1, 1]$ is similarly approximated using an interpolating polynomial:

$$u(\tau) \approx u^N(\tau) = \sum_{k=0}^{N} u(\tau_k)\phi_k(\tau) \tag{1.25}$$

Note that, from (1.11), $x^N(\tau_k) = x(\tau_k)$ and $u^N(\tau_k) = u(\tau_k)$. The derivative of the state vector is approximated as follows:

$$\dot{x}(\tau_k) \approx \dot{x}^N(\tau_k) = \sum_{i=0}^{N} D_{ki} x^N(\tau_i),\ i = 0, \dots N \tag{1.26}$$

where D is the $(N+1) \times (N+1)$ the differentiation matrix given by (1.12).

Define the following $n_u \times (N+1)$ matrix to store the trajectories of the controls at the LGL nodes:

$$U^N = \begin{bmatrix} u_1(\tau_0) & u_1(\tau_1) & \dots & u_1(\tau_N) \\ u_2(\tau_0) & u_2(\tau_1) & \dots & u_2(\tau_N) \\ \vdots & \vdots & \ddots & ,\vdots \\ u_{n_u}(\tau_0) & u_{n_u}(\tau_1) & \dots & u_{n_u}(\tau_N) \end{bmatrix} \tag{1.27}$$

Define the following $n_x \times (N+1)$ matrices to store, respectively, the trajectories of the states and their derivatives at the LGL nodes:

$$X^N = \begin{bmatrix} x_1(\tau_0) & x_1(\tau_1) & \dots & x_1(\tau_N) \\ x_2(\tau_0) & x_2(\tau_1) & \dots & x_2(\tau_N) \\ \vdots & \vdots & \ddots & ,\vdots \\ x_{n_x}(\tau_0) & x_{n_x}(\tau_1) & \dots & x_{n_x}(\tau_N) \end{bmatrix} \tag{1.28}$$

and

$$\dot{X}^N = \begin{bmatrix} \dot{x}_1(\tau_0) & \dot{x}_1(\tau_1) & \dots & \dot{x}_1(\tau_N) \\ \dot{x}_2(\tau_0) & \dot{x}_2(\tau_1) & \dots & \dot{x}_2(\tau_N) \\ \vdots & \vdots & \ddots & ,\vdots \\ \dot{x}_{n_x}(\tau_0) & \dot{x}_{n_x}(\tau_1) & \dots & \dot{x}_{n_x}(\tau_N) \end{bmatrix} \tag{1.29}$$

From (1.26), X^N and $\dot{X}_N$ are related as follows:

$$\dot{X}^N = X^N D^T \tag{1.30}$$

Now, form the following $n_x \times (N+1)$ matrix with the right hand side of the differential constraints evaluated at the LGL nodes:

$$F^N = \frac{t_0 - t_f}{2} \begin{bmatrix} f_1(x^N(\tau_0), u^N(\tau_0), p, \tau_0) & \dots & f_1(x^N(\tau_N), u^N(\tau_N), p, \tau_N) \\ f_2(x^N(\tau_0), u^N(\tau_0), p, \tau_0) & \dots & f_2(x^N(\tau_N), u^N(\tau_N), p, \tau_N) \\ \vdots & \ddots & \vdots \\ f_{n_x}(x^N(\tau_0), u^N(\tau_0), p, \tau_0) & \dots & f_{n_x}(x^N(\tau_N), u^N(\tau_N), p, \tau_N) \end{bmatrix} \tag{1.31}$$

Now, define the differential defects at the collocation points as the $n_x \times (N+1)$ matrix:

$$\zeta^N = \dot{X}^N - F^N = X^N D^T - F^N \tag{1.32}$$

Define the matrix of path constraint function values evaluated at the LGL nodes:

$$H^N = \begin{bmatrix} h_1(x^N(\tau_0), u^N(\tau_0), p, \tau_0) & \dots & h_1(x^N(\tau_N), p, u^N(\tau_N), \tau_N) \\ h_2(x^N(\tau_0), u^N(\tau_0), p, \tau_0) & \dots & h_2(x^N(\tau_N), u^N(\tau_N), p, \tau_N) \\ \vdots & \ddots & \vdots \\ h_{n_h}(x^N(\tau_0), u^N(\tau_0), p, \tau_0) & \dots & h_{n_h}(x^N(\tau_N), u^N(\tau_N), p, \tau_N) \end{bmatrix} \tag{1.33}$$

The objective function of $\mathcal{P}_3$ is approximated as follows:

$$\begin{aligned} J &= \varphi[x(-1), x(1), p, t_0, t_f] + \frac{t_f - t_0}{2} \int_{-1}^{1} L[x(\tau), u(\tau), p, \tau] d\tau \\ &\approx \varphi[x^N(-1), x^N(1), p, t_0, t_f] + \frac{t_f - t_0}{2} \sum_{k=0}^{N} L[x^N(\tau_k), u^N(\tau_k), p, \tau_k] w_k \end{aligned} \tag{1.34}$$

where the weights w_k are defined in (1.5).

We are now ready to express problem $\mathcal{P}_3$ as a nonlinear programming problem, as follows.

Problem $\mathcal{P}_4$

$$\min_y F(y) \tag{1.35}$$

subject to:

$$\begin{aligned} G_l \leq & G(y) \leq G_u \\ y_l \leq & y \leq y_u \end{aligned} \tag{1.36}$$

The decision vector y, which has dimension $n_y = (n_u(N+1) + n_x(N+1) + n_p + 2)$, is constructed as follows:

$$y = \begin{bmatrix} \text{vec}(U^N) \\ \text{vec}(X^N) \\ p \\ t_0 \\ t_f \end{bmatrix} \tag{1.37}$$

The objective function is:

$$F(y) = \varphi[x^N(-1), x^N(1), p, t_0, t_f] + \frac{t_f - t_0}{2} \sum_{k=0}^{N} L[x^N(\tau_k), u^N(\tau_k), p, \tau_k] w_k \tag{1.38}$$

while the constraint function $G(y)$, which is of dimension $n_g = n_x(N+1) + n_h(N+1) + n_e + 1$, is given by:

$$G(y) = \begin{bmatrix} \text{vec}(\zeta^N) \\ \text{vec}(H^N) \\ e[x^N(-1), u^N(-1), x^N(1), u^N(1), p, t_0, t_f] \\ t_f - t_0 \end{bmatrix}, \tag{1.39}$$

The constraint bounds are given by:

$$G_l = \begin{bmatrix} \mathbf{0}_{n_x(N+1)} \\ \text{stack}(h_L, N+1) \\ e_L \\ (\underline{t}_0 - \bar{t}_f) \end{bmatrix}, \; G_u \begin{bmatrix} \mathbf{0}_{n_x(N+1)} \\ \text{stack}(h_U, N+1) \\ e_U \\ 0 \end{bmatrix}, \tag{1.40}$$

and the bounds on the decision vector are given by:

$$y_l = \begin{bmatrix} \text{stack}(u_l, N+1) \\ \text{stack}(x_L, N+1) \\ p_L \\ \underline{t}_0 \\ \underline{t}_f \end{bmatrix}, \; y_u \begin{bmatrix} \text{stack}(u_U, N+1) \\ \text{stack}(x_U, N+1) \\ p_U \\ \bar{t}_0 \\ \bar{t}_f \end{bmatrix}, \tag{1.41}$$

where vec(A) forms a nm-column vector by vertically stacking the columns of the $n \times m$ matrix A, and stack(x, n) creates a mn-column vector by stacking n copies of column m-vector x.

1.9.1 Costate estimates

The costate estimates for the Legendre and Chebyshev methods are described below; the corresponding estimates for the Radau and Gauss methods are given in Section 1.10.

Legendre approximation method

$\mathcal{PSOPT}$ implements the following approximation for the costates $\lambda(\tau) \in \Re^{n_x}, \tau \in [-1, 1]$ associated with $\mathcal{P}_3$ [15]:

$$\lambda(\tau) \approx \lambda^N(\tau) = \sum_{k=0}^{N} \lambda(\tau_k)\phi_k(\tau), \tau \in [-1, 1] \tag{1.42}$$

The costate values at the LGL nodes are given by:

$$\lambda(\tau_k) = \frac{\tilde{\lambda}_k}{w_k}, \ \ k = 0, \ldots N \tag{1.43}$$

where w_k are the weights given by (1.5), and $\tilde{\lambda}_k \in \Re^{n_x}$, $k = 0, \ldots, N$ are the KKTs multiplier associated with the collocation constraints $\text{vec}(\zeta^N) = 0$. The KKT multipliers can normally be obtained from the NLP solver, which allows $\mathcal{PSOPT}$ to return estimates of the costate trajectories at the LGL nodes.

It is known from the literature [15] that the costate estimates in the Legendre discretisation method sometimes oscillate around the true values. To mitigate this, the estimates are smoothed by taking a weighted average for the estimats at k using the costate estimates at $k-1$, k and $k+1$ obtained from (1.42).

Chebyshev approximation method

When the cost quadrature uses the Clenshaw–Curtis weights of Section 1.8.8, the covector (costate) mapping for the Chebyshev method takes the same form as for the Legendre method:

$$\lambda(\tau_k) = \frac{\tilde{\lambda}_k}{w_k}, \ \ k = 0, \ldots, N \tag{1.44}$$

where w_k are the Clenshaw–Curtis weights (equation (1.20)) and $\tilde{\lambda}_k \in \Re^{n_x}$, $k = 0, \ldots, N$ are the KKT multipliers associated with the collocation constraints $\text{vec}(\zeta^N) = 0$. Because these weights are strictly positive at every CGL node, including the endpoints $\tau_0 = -1$ and $\tau_N = 1$, equation (1.44) is well defined for all $k = 0, \ldots, N$ and, unlike the earlier mapping that carried a $1/\sqrt{1-\tau_k^2}$ factor [31], requires no endpoint extrapolation. With this mapping the smoothing applied to the Legendre estimates (Section 1.9.1) is not needed for the Chebyshev method.

1.9.2 Discretizing a multiphase problem

It now becomes straightforward to describe the discretisation used by $\mathcal{PSOPT}$ in the case of $\mathcal{P}_1$, a problem with multiple phases, to form a nonlinear programming problem like $\mathcal{P}_4$. The decision variables of the NLP associated with $\mathcal{P}_1$ are given by:

$$y = \begin{bmatrix} \mathrm{vec}(U^{N,(1)}) \\ \mathrm{vec}(X^{N,(1)}) \\ p^{(1)} \\ t_0^{(1)} \\ t_f^{(1)} \\ \vdots \\ \mathrm{vec}(U^{N,(N_p)}) \\ \mathrm{vec}(X^{N,(N_p)}) \\ p^{(N_p)} \\ t_0^{(N_p)} \\ t_f^{(N_p)} \end{bmatrix} \tag{1.45}$$

where N_p is the number of phases in the problem, and the superindex in parenthesis indicates the phase to which the variables belong. The constraint function $G(y)$ is given by:

$$G(y) = \begin{bmatrix} \mathrm{vec}(\zeta^{N,(1)}) \\ \mathrm{vec}(H^{N,(1)}) \\ e[x^{N,(1)}(-1), u^{N,(1)}(-1), x^{N,(1)}(1), u^{N,(1)}(1), p^{(1)}, t_0^{(1)}, t_f^{(1)}] \\ t_f^{(1)} - t_0^{(1)} \\ \vdots \\ \mathrm{vec}(\zeta^{N,(N_p)}) \\ \mathrm{vec}(H^{N,(N_p)}) \\ e[x^{N,(N_p)}(-1), u^{N,(N_p)}(-1), x^{N,(N_p)}(1), u^{N,(N_p)}(1), p^{(N_p)}, t_0^{(N_p)}, t_f^{(N_p)}] \\ t_f^{(N_p)} - t_0^{(N_p)} \\ \Psi \end{bmatrix}, \tag{1.46}$$

where Ψ corresponds to the linkage constraints associated with the problem, evaluated at y.

Based on the problem information, it is straightforward (but not shown here) to form the bounds on the decision variables y_l, y_u and the bounds on the constraints function G_l, G_u to complete the definition of the NLP problem associated with $\mathcal{P}_1$.

1.10 Radau and Gauss pseudospectral discretisations

In addition to the Legendre (LGL) and Chebyshev (CGL) methods described above, $\mathcal{PSOPT}$ provides two further global pseudospectral discretisations, selected by setting `algorithm.collocation_method` to `"Radau"` or `"Gauss"`. The three Legendre families are all built from the roots of the Legendre polynomial L_N and its derivative, but they differ in which endpoints of $[-1, 1]$ are collocated: the LGL points include both endpoints, the Legendre–Gauss–Radau (LGR) points include only one, and the Legendre–Gauss (LG) points include neither [17]. This apparently small difference has a marked effect on the accuracy of the costate estimates, which is the main motivation for offering these schemes. The precise definitions of the node sets, and a unified treatment of the three methods, are given by Garg et al. [17, 18].

1.10.1 Legendre–Gauss–Radau (LGR) collocation

The LGR points consist of one endpoint of $[-1, 1]$ together with N interior points; the opposite endpoint is not collocated. The state and control are interpolated by Lagrange polynomials, and the dynamics are collocated at the LGR points using a rectangular, full-rank differentiation matrix [17]. The discretisation otherwise follows the same pattern as the Legendre method described above: the differential defects $\zeta = XD^\top - F$ are imposed as equality constraints, and the Lagrange term of the cost is evaluated with the Gauss–Radau quadrature weights. The Radau scheme is asymmetric in time, which makes it natural for initial-value-like problems and for the individual phases of a multi-phase formulation.

1.10.2 Legendre–Gauss (LG) collocation and the terminal-state variable

The LG points are the N interior roots of the Legendre polynomial L_N; neither $\tau = -1$ nor $\tau = +1$ is collocated. Because the final state is then not attached to any collocation node, the Gauss scheme introduces the terminal state $x_f = x(t_f)$ as an explicit decision variable, related to the initial state through a Gauss quadrature of the dynamics,

$$x_f = x(t_0) + \frac{t_f - t_0}{2} \sum_{k=1}^{N} w_k \, f[x(\tau_k), u(\tau_k), p, \tau_k], \tag{1.47}$$

which is appended to the nonlinear program as an additional vector equality constraint, while the dynamics are collocated at the N interior LG points. This construction, due to Benson [2] and placed in a unified framework by Garg et al. [17], is what gives the Gauss method its characteristically accurate costates.

1.10.3 Costate estimates for the Radau and Gauss methods

For both schemes the discrete costates are recovered from the Karush–Kuhn–Tucker multipliers $\tilde{\lambda}_k$ of the differential defect constraints, scaled by the quadrature weights,

$$\lambda(\tau_k) = \frac{\tilde{\lambda}_k}{w_k}, \tag{1.48}$$

in close analogy with the Legendre case of Section 1.9.1. The essential difference is that the LGR and LG differentiation operators are full rank, so the discrete adjoint system is an exact transcription of the continuous one; the resulting costates do not exhibit the boundary oscillations of the LGL method, and the Gauss method in particular yields the most accurate costates of the pseudospectral families [17, 18]. For the Gauss scheme the terminal costate is obtained from the multiplier of the terminal-state constraint (1.47). A unified derivation of these costate mappings is given in [17].

1.11 Local discretisations

Direct collocation methods that use local information to approximate the functions associated with an optimal control problem are well established [4]. These include for instance the trapezoidal and Hermite-Simpson methods. A further, non-collocation alternative — the integrated-residual transcription, which controls the dynamic error between the nodes rather than only at them — is described in Chapter 3. Sometimes, it may be convenient for users to compare the performance and solutions obtained by means of the pseudospectral methods implemented in $\mathcal{PSOPT}$, with local discretisation methods. Also, if a given problem cannot be solved by means of a pseudospectral discretisation, the user has the option to try the local discretisations implemented in $\mathcal{PSOPT}$. The main impact of using a local discretisation method as opposed to a pseudospectral discretisation method, is that the resulting Jacobian and Hessian matrices needed by the NLP solver are more sparse with local methods, which facilitates the NLP solution. This becomes more noticeable as the number of grid points increases. The disadvantage of using a local method is that the spectral accuracy in the discretisation of the differential constraints offered by pseudospectral methods is lost. Moreover, the accuracy of Gauss type integration employed in pseudospectral methods is also lost if pseudospectral grids are not used.

Note also that local mesh refinement methods are well established. These methods concentrate more grid points in areas of greater activity in the function, which helps improve the local accuracy of the solution. The trapezoidal method has an accuracy of $\mathcal{O}(h^2)$, while the Hermite-Simpson method has an accuracy of $\mathcal{O}(h^4)$, where h is the local interval between grid points. Both the trapezoidal and Hermite-Simpson discretisation methods are widely used in computational optimal control, and have solve many challenging problems [4]. When the user selects the trapezoidal or Hermite-Simpson discretisations, and if the initial grid points are not provided, the grid is started with equal spacing between grid points. In these two cases any integrals associated with the problem are computed using the trapezoidal and Simpson quadrature method, respectively.

The local discretisations implemented in $\mathcal{PSOPT}$ are described below. For simplicity, the phase index has been omitted and reference is made to single phase problems. However, the methods can also be used with multi-phase problems.

1.11.1 Trapezoidal method

With the trapezoidal method [4], the defect constraints are computed as follows:

$$\zeta(\tau_k) = x(\tau_{k+1}) - x(\tau_k) - \frac{h_k}{2}(f_k + f_{k+1}), \tag{1.49}$$

where $\zeta(\tau_k) \in \Re^{n_x}$ is the vector of differential defect constraints at node τ_k, $k = 0, \ldots, N-1$, $h_k = \tau_{k+1} - \tau_k$, $f_k = f[(\tau_k), u(\tau_k), p, \tau_k]$, $f_{k+1} = f[x(\tau_{k+1}), u(\tau_{k+1}), p, \tau_{k+1}]$. This gives rise to $n_x N$ differential defect constraints. In this case, the decision vector for single phase problems is given by equation (1.37), so that it is the same as the one used in the Legendre and Chebyshev methods.

1.11.2 Hermite-Simpson method

With the Hermite-Simpson method [4], the defect constraints are computed as follows:

$$\zeta(\tau_k) = x(\tau_{k+1}) - x(\tau_k) - \frac{h_k}{6}(f_k + 4\bar{f}_{k+1} + f_{k+1}), \tag{1.50}$$

where

$$\bar{f}_{k+1} = f[\bar{x}_{k+1}, \bar{u}_{k+1}, p, \tau_k + \frac{h_k}{2}]$$

$$\bar{x}_{k+1} = \frac{1}{2}(x(\tau_k) + x(\tau_{k+1})) + \frac{h_k}{8}(f_k - f_{k+1})$$

where $\zeta(\tau_k) \in \Re^{n_x}$ is the vector of differential defect constraints at node τ_k, $k = 0, \ldots, N-1$, $h_k = \tau_{k+1} - \tau_k$, $f_k = f[(\tau_k), u(\tau_k), p, \tau_k]$, $f_{k+1} = f[x(\tau_{k+1}), u(\tau_{k+1}), p, \tau_{k+1}]$, and $\bar{u}_{k+1} = \bar{u}(\tau_{k+1})$ is a vector of midpoint controls (which are also decision variables). This gives rise to $n_x N$ differential defect constraints. In this case, the decision vector for single phase problems is given by

$$y = \begin{bmatrix} \mathrm{vec}(U^N) \\ \mathrm{vec}(X^N) \\ p \\ \mathrm{vec}(\bar{U}^N) \\ t_0 \\ t_f \end{bmatrix} \tag{1.51}$$

with

$$\bar{U}^N = \begin{bmatrix} \bar{u}_1(\tau_1) & \bar{u}_1(\tau_2) & \ldots & \bar{u}_1(\tau_N) \\ \bar{u}_2(\tau_1) & \bar{u}_2(\tau_2) & \ldots & \bar{u}_2(\tau_N) \\ \vdots & \vdots & \ddots & ,\vdots \\ \bar{u}_{n_u}(\tau_1) & \bar{u}_{n_u}(\tau_2) & \ldots & \bar{u}_{n_u}(\tau_N) \end{bmatrix} \tag{1.52}$$

so that this decision vector is different from the one used in the Legendre and Chebyshev methods as it includes the midpoint controls.

1.11.3 Costate estimates with local discretisations

In the case of the trapezoidal and Hermite-Simpson discretisations, the costates at the discretisation nodes are approximated according to the following equation:

$$\lambda(t_{k+\frac{1}{2}}) \approx \frac{\tilde{\lambda}_k}{2h_k},\ k = 0, \ldots N-1$$

where $\lambda(t_{k+\frac{1}{2}})$ is the costate estimate at the midpoint in the interval between t_k and t_k+, $\tilde{\lambda}_k \in \Re^{n_x}$ is the vector of Lagrange multiplers obtained from the nonlinear programming solver corresponding to the differential defect constraints, and $h_k = t_{k+1} - t_k$.

1.12 Parameter estimation problems

A parameter estimation problem arises when it is required to find values for parameters associated with a model of a system based on observations from the actual system. These are also called *inverse problems*. The approach used in the $\mathcal{PSOPT}$ implementation uses the same techniques used for solving optimal control problems, with a special objective function used to measure the accuracy of the model for given parameter values.

1.12.1 Single-phase case

For the sake of simplicity consider first a single phase problem defined over $t_0 \le t \le t_f$ with the dynamics given by a set of ODEs:

$$\dot{x} = f[x(t), u(t), p, t]$$

the path constraints

$$h_L \le h[x(t), u(t), p, t] \le h_U$$

the event constraints

$$e_L \le e[x(t_0), u(t_0), x(t_f), u(t_f), p, t_0, t_f] \le e_U$$

Consider the following model of the observations (or measurements) taken from the system:

$$y(\theta) = g[x(\theta), u(\theta), p, \theta]$$

where $g : \mathcal{R}^{n_x} \times \mathcal{R}^{n_u} \times \mathcal{R}^{n_p} \times \mathcal{R} \to \mathcal{R}^{n_o}$ is the observations function, and $y(\theta) \in \mathcal{R}^{n_o}$ is the estimated observation at sampling instant θ. Assume that $\{\tilde{y}\}_{k=1}^{n_s}$ is a sequence of n_s observations corresponding to the sampling instants $\{\theta_k\}_{k=1}^{n_s}$.

The objective is to choose the parameter vector $p \in \mathcal{R}^{n_p}$ to minimise the cost function:

$$J = \frac{1}{2} \sum_{k=1}^{n_s} \sum_{j=1}^{n_o} r_{j,k}^2$$

where the residual $r_{j,k} \in \mathcal{R}$ is given by:

$$r_{j,k} = w_{j,k}[g_j[x(\theta_k), u(\theta_k), p, \theta_k] - \tilde{y}_{j,k}]$$

where $w_{j,k} \in \mathcal{R}, j = 1, \ldots, n_o, k = 1, \ldots, n_s$ is a positive residual weight, g_j is the j-th element of the vector observations function g, and $\tilde{y}_{j,k}$ is the j-th element of the actual observation vector at time instant θ_k.

Note that in the parameter estimation case the times t_0 and t_f are assumed to be fixed. The sampling instants need not coincide with the collocation points, but they must obey the relationship:

$$t_0 \le \theta_k \le t_f, \;\; k = 1, \ldots, n_s$$

1.12.2 Multi-phase case

In the case of a problem with N_p phases, let $t_0^{(i)} \le t \le t_f^{(i)}$ be the intervals for each phase, with the dynamics given by a set of ODEs:

$$\dot{x}^{(i)} = f^{(i)}[x(t)^{(i)}, u^{(i)}(t), p^{(i)}, t]$$

the path constraints

$$h_L^{(i)} \le h^{(i)}[x^{(i)}(t), u^{(i)}(t), p^{(i)}, t] \le h_U^{(i)}$$

the event constraints

$$e_L^{(i)} \le e^{(i)}[x^{(i)}(t_0), u^{(i)}(t_0), x^{(i)}(t_f), u^{(i)}(t_f), p^{(i)}, t_0^{(i)}, t_f^{(i)}] \le e_U^{(i)}$$

Consider the following model of the observations (or measurements) taken from the system for each phase:

$$y^{(i)}(\theta_k^{(i)}) = g^{(i)}[x^{(i)}(\theta_k^{(i)}), u^{(i)}(\theta_k^{(i)}), p^{(i)}, \theta_k^{(i)}]$$

where $g^{(i)} : \mathcal{R}^{n_x^{(i)}} \times \mathcal{R}^{n_u^{(i)}} \times \mathcal{R}^{n_p^{(i)}} \times \mathcal{R} \to \mathcal{R}^{n_o^{(i)}}$ is the observations function for each phase, and $y^{(i)}(\theta_k^{(i)}) \in \mathcal{R}^{n_o^{(i)}}$ is the estimated observation at sampling instant $\theta_k^{(i)}$. Assume that $\{\tilde{y}^{(i)}\}_{k=1}^{n_s^{(i)}}$ is a sequence of $n_s^{(i)}$ observations corresponding to the sampling instants $\left\{\theta_k^{(i)}\right\}_{k=1}^{n_s^{(i)}}$.

The objective is to choose the set of parameter vectors $p^{(i)} \in \mathcal{R}^{n_p^{(i)}}, i \in [1, N_p]$ to minimise the cost function:

$$J = \frac{1}{2} \sum_{i=1}^{N_p} \sum_{k=1}^{n_s^{(i)}} \sum_{j=1}^{n_o^{(i)}} \left[r_{j,k}^{(i)} \right]^2$$

where the residual $r_{j,k}^{(i)} \in \mathcal{R}$ is given by:

$$r_{j,k}^{(i)} = w_{j,k}^{(i)}[g_j^{(i)}[x(\theta_k^{(i)}), u^{(i)}(\theta_k^{(i)}), p^{(i)}, \theta_k^{(i)}] - \tilde{y}_j^{(i)}]$$

where $w_{j,k}^{(i)} \in \mathcal{R}$ is a positive residual weight, $g_j^{(i)}$ is the j-th element of the vector observations function $g^{(i)}$, and $\tilde{y}_j^{(i)}$ is the j-th element of the actual observation vector at time instant $\theta_k^{(i)}$

Note that in the parameter estimation case the times $t_0^{(i)}$ and $t_f^{(i)}$ are assumed to be fixed. The sampling instants need not coincide with the collocation points, but they must obey the relationship:

$$t_0^{(i)} \leq \theta_k^{(i)} \leq t_f^{(i)},\ k = 1, \ldots, n_s^{(i)}$$

1.12.3 Statistical measures on parameter estimates

$\mathcal{PSOPT}$ computes a residual matrix $[r_{j,k}^{(i)}]$ for each phase i and for the final value of the estimated parameters in all phases $\hat{p} \in \mathcal{R}^{n_p}$, where each element of the residual matrix is related to an individual measurement sample and observation within a phase. $\mathcal{PSOPT}$ also computes the covariance matrix $C \in \mathcal{R}^{n_p \times n_p}$ of the parameter estimates using the method described in [23], which uses a QR decomposition of the Jacobian matrix of the equality and active inequality constraints, together with the Jacobian matrix of the residual vector function (a stack of the elements of the residual matrices for all phases) with respect to all decision variables. In addition $\mathcal{PSOPT}$ computes 95% confidence intervals on the estimated parameters. The upper and lower limits of the confidence interval around the estimated value for parameter $\hat{p}_i$ are computed from [25]:

$$\delta_i = \pm t_{N_s - n_p}^{1-(\alpha/2)} \sqrt{C_{ii}} \tag{1.53}$$

where N_s is the total number of individual samples, n_p is the number of parameters, t is the inverse two tailed cumulative t-distribution with confidence level α, and $N - n_p$ degrees of freedom.

The residual matrix, the covariance matrix and the confidence intervals can be used to refine the parameter estimation problem. For instance, if the resulting confidence interval of one particular parameter is found to be small, the value of this parameter can be fixed by the user in a subsequent run, which may improve the estimates other parameters being estimated and reduces the possibility of having an overdetermined problem (i.e. a problem with too many parameters to be estimated). See [37] pages 210-211 for more details.

Notice, however, that the statistical analysis performed is based on a linearization of the model. As a result, the validity of the statistical analysis is dependent on the quality of the linearization and the curvature of the underlying functions being linearized, so care must be taken with the interpretation of results of the statistical analysis of the parameter estimates [37].

1.12.4 Remarks on parameter estimation

- In contrast to continuous optimal control problems, the kind of parameter estimation problem considered involves the evaluation of the objective at a finite number of sampling points. Internally, the values of the state and controls are interpolated over the collocation points to find estimated values at the sampling points. The type of interpolation employed depends on the collocation method specified by the user.

- Note that the sampling instants do not have to be sorted in ascending or descending order. Because of this, it is possible to accommodate problems with non-simultaneous observations of different variables by stacking the measured data and sampling instants for the different variables.

- It is possible to use an alternative objective function where the residuals are weighted with the covariance of the measurements simply by multiplying the observations function and the measurements vectors by the square root of the covariance matrix; see [4] for more details.

- When defining parameter estimation problems, the user needs to ensure that the underlying nonlinear programming problem has sufficient degrees of freedom. This is particularly important as it is common for parameter estimation problems not to involve any control variables. The number of degrees of freedom is the difference between the number of decision variables and the total number of equality and active inequality constraints. For example, in the case of a single phase problem having n_x differential states, n_u control variables (or algebraic states), n_h equality path constraints, and n_e equality event constraints, the number of relevant constraints is given by:

$$n_c = n_x(N+1) + n_h(N+1) + n_e + 1$$

where N is the degree of the polynomial approximation (in the case of a pseudospectral discretisation). The number of decision variables is given by:

$$n_y = n_u(N+1) + n_x(N+1) + n_p + 2$$

The difference is:

$$n_y - n_c = (n_u - n_h)(N+1) + n_p + 1 - n_e$$

For the problem to be solvable it is important that $n_y - n_c \geq 0$, ideally $n_y - n_c \geq 1$. The total numbers of constraints and decision variables are always reported in the terminal window when a $\mathcal{PSOPT}$ problem is run. It should be noted that the nonlinear programming solver (IPOPT) may modify the numbers by eliminating redundant constraints or decision variables. These modifications are also visible when a problem is run.

1.13 Limitations and known issues

1. The discretisation techniques used by $\mathcal{PSOPT}$ give approximate solutions for the state and control trajectories. The software is intended to be used for problems

where the control variables are continuous (within a phase) and the state variables have continuous derivatives (within a phase). If within a phase the solution to the optimal control problem is of a different nature, the results may be incorrect or the optimization algorithm may fail to converge. Furthermore, $\mathcal{PSOPT}$ may not be suitable for solving problems involving differential-algebraic equations with index greater than one. Some of these issues can be avoided by reformulating the problem to have several phases.

2. The solution obtained by $\mathcal{PSOPT}$ corresponds to a local minimum of the discretized optimization problem. If the problem is suspected to have several local minima, then it may be worth trying various initial guesses.

3. The automatic scaling procedures work well for all the examples provided. However, note that the scaling of variables depends on the user provided bounds. If these bounds are not adequate for the problem, then the resulting scaling may be poor and this may lead to incorrect results or convergence problems. In some cases, users may need to provide the scaling factors manually to obtain satisfactory results.

4. The automatic mesh refinement procedures require an initial guess for the number of nodes in the global case (the number and/or initial distribution of nodes in the local case). If this initial guess is not adequate (e.g. the grid is too coarse or to dense), the mesh refinement procedure may fail to converge. In some cases, the user may need to manually tune some of the parameters of the mesh refinement procedure to achieve satisfactory results.

5. The efficiency with which the optimal control problem is solved depends in a good deal on the correct formulation of the problem. Unsuitable formulations may lead to trouble in finding a solution. Moreover, if the constraints are such that the problem is infeasible or if for any other reason the solution does not exist, then the nonlinear programming algorithm will fail.

6. The user supplied functions which define the cost function, DAE's, event and linkage constraints, are all assumed to be continuous and to have continuous first and second derivatives. Non-differentiable functions may cause covergence problems to the optimization algorithm. Moreover, it is known that discontinuities in the second derivatives may also cause convergence problems.

7. Only single phase problems are supported if the dynamics involve delays in the states or controls.

8. Note that the constraints associated with the problem are only enforced at the discretisation nodes, not in the interval between the nodes. The integrated-residual transcription of Chapter 3 mitigates this by measuring, and optionally bounding, the dynamic residual between the nodes.

9. When the problem requires a large number of nodes (say over 200) the nonlinear programming algorithm may have problems to converge if global collocation is being used. This may be due to numerical difficulties within the nonlinear programming solver as the Jacobian (and Hessian) matrices may not be sufficiently sparse. This occurs because the pseudospectral differentiation matrices are dense. When faced with this problem the user may wish to try the local collocation options availble within $\mathcal{PSOPT}$, or to split the problem into multiple segments to increase the sparsity of the derivatives. Note that the sparsity of the Jacobian and Hessian matrices is problem dependent.

10. The co-state approximations resulting from the Legendre pseudospectral method are not as accurate as those obtained by means of the Gauss or Radau pseudospectral methods now available in $\mathcal{PSOPT}$ (Section 1.10); the Gauss method in particular yields the most accurate costates [17]. Moreover, the co-state approximations obtained by $\mathcal{PSOPT}$ using the Chebyshev pseudospectral methods are rather innacurate close to the edges of the time interval within each phase. Also the co-state approximation used in the the case of local discretisations (trapezoidal, Hermite-Simpson) converges at a lower rate (is less accurate) than the states or the controls.

2. Mesh refinement

It is possible to refine the mesh of the discretisation nodes to improve the accuracy of the solutions. With $\mathcal{PSOPT}$, this can be done manually or automatically. To be able to refine a discretisation mesh, it is necessary to have some measure of the discretisation error. This is discussed in the next section.

2.1 Evaluating the discretisation error

$\mathcal{PSOPT}$ evaluates the discretisation error using a method adopted from [4]. Define the error in the differential equation as a function of time:

$$\epsilon(t) = \dot{\tilde{x}}(t) - f[\tilde{x}(t), \tilde{u}(t), p, t]$$

where $\tilde{x}$ is an interpolated value of the state vector given the grid point values of the state vector, $\dot{\tilde{x}}$ is an estimate of the derivative of the state vector given the state vector interpolant, and $\tilde{u}$ is an interpolated value of the control vector given the grid points values of the control vector. The type of interpolation used depends on the collocation method employed. For Legendre and Chebyshev methods, the interpolation done by the Lagrange interpolant. For Trapezoidal and Hermite-Simpson methods, cubic spline interpolation is used. The absolute local error corresponding to state i on a particular interval $t \in [t_k, t_{k+1}]$, is defined as follows:

$$\eta_{i,k} = \int_{t_k}^{t_{k+1}} |\epsilon_i(t)| \mathrm{d}t$$

where the integral is computed using the composite Simpson method. The default number of integration steps for each interval is 10, but this can be changed by means of the input parameter `algorithm.nsteps_error_integration`. The relative local error is defined as:

$$\epsilon_k = \max_i \frac{\eta_{i,k}}{w_i + 1}$$

where

$$w_i = \max_{k=1}^{N} \left[|\tilde{x}_{i,k}|, |\dot{\tilde{x}}_{i,k}| \right]$$

After each $\mathcal{PSOPT}$ run, the sequence ϵ_k for each phase is available through the solution structure as follows:

```
epsilon = solution.get_relative_local_error_in_phase(iphase)
```

where `epsilon` is a MatrixXd object. The error sequence can be analysed by the user to assess the quality of the discretisation. This information may be useful to aid the mesh refinement process.

Additionally, the maximum value of the sequence ϵ_k for each phase is printed in the solution summary at the end of an execution.

2.2 Manual mesh refinement

Manual mesh refinement, which is the default option, is performed by interpolating a previous solution based on n_1 nodes, into a new mesh based on n_2 nodes, where $n_2 > n_1$, and using the interpolated solution as an initial guess for a new optimization. If global collocation is being used, $\mathcal{PSOPT}$ employs Lagrange polynomials to perform the interpolation associated with mesh refinement. If local collocation is being used, $\mathcal{PSOPT}$ employs cubic splines to perform the interpolation. The variables which are interpolated include the controls, states and Lagrange multipliers associated with the differential defect constraints, which are related to the co-states. The other decision variables (start and final times, and static parameters) do not need to be interpolated.

To perform mesh refinement, the user must supply the desired sequence of grid points (or nodes) for each phase through the parameter:

```
problem.phases(iphase).nodes
```

For example, to try the sequence 40, 50 and 80 nodes in phase `iphase`, then the following commandd specify that:

```
RowVectorXi my_vector << 40, 50, 80;
problem.phases(iphase).nodes = my_vector;
```

If the user wishes to try only a single grid size (with no mesh refinement), this is specified by providing a single value as follows:

```
problem.phases(iphase).nodes << INTEGER;
```

In problems with more than one phase, the length of the node sequence to be tried needs to be the same in each phase, but the actual grid sizes need not be the same betweeen phases.

2.3 Automatic *hp*-adaptive mesh refinement with pseudospectral grids

When a global pseudospectral method (Legendre, Chebyshev, Radau or Gauss) is selected and `algorithm.mesh_refinement` is set to `"automatic"`, $\mathcal{PSOPT}$ refines the mesh using an hp-adaptive (or ph) strategy in the spirit of [12, 30, 24]. Instead of raising the order of a single global block, the time domain of each phase is partitioned into K subintervals (segments), and on segment j the state is approximated by a local polynomial of degree n_j. The refinement adjusts simultaneously the number of segments K, the breakpoints that separate them, and the per-segment degrees n_j, so as to reach a user-specified accuracy with an economical distribution of collocation points. The total number of collocation points in a phase is $N_{\text{eff}} = \sum_{j=1}^{K} n_j$.

Per-segment error and smoothness. After each solve $\mathcal{PSOPT}$ computes the maximum relative discretisation error ϵ_j on every segment j (Section 2.1). Segments whose error is already below the tolerance $\epsilon_{\max}$ (`ode_tolerance`) are left unchanged. For each remaining segment the smoothness of the local solution is assessed from the decay rate of its Legendre coefficients: writing the local state as $x(\tau) \approx \sum_{\ell=0}^{n_j} a_\ell P_\ell(\tau)$, a rapid (geometric) decay of $|a_\ell|$ indicates a smooth solution, whereas a slow decay flags a likely nonsmooth feature, such as a kink or a near-discontinuity, within the segment [24].

p- versus h-refinement. The smoothness assessment selects the action taken on each flagged segment:

- *p-refinement* (smooth segment): increase the local polynomial degree $n_j \leftarrow \min(n_j + \Delta n_j,\, N_{\max})$. When the solution is smooth, the spectral convergence of the pseudospectral approximation makes raising the degree the most efficient way to reduce the error.

- *h-refinement* (nonsmooth segment, or n_j already at $N_{\max}$): subdivide the segment into smaller segments, each seeded at degree $N_{\min}$. Placing breakpoints around the nonsmooth feature confines the low-order region and restores fast convergence on the smooth parts.

The degrees are constrained to the band $N_{\min} \leq n_j \leq N_{\max}$ (`mr_min_order` and `mr_max_order`), where $N_{\max}$ is the threshold at which an interval switches from p- to h-refinement. The per-iteration growth of N_{eff} is limited to a factor `mr_max_growth_factor` of its current value, so that the mesh grows gradually and the worst-case problem size is bounded a priori.

Common driver for the four methods. All four global methods share this driver. The Legendre (LGL) and Chebyshev (CGL) methods collocate at both segment endpoints and the Radau (LGR) method at the left endpoint, so adjacent segments share a breakpoint; the Gauss (LG) method collocates at the strictly interior points, and the breakpoints enter as additional non-collocated interface variables (Section 1.10). These structural differences are handled internally, and the user interface and the refinement controls are identical across the four methods.

The user specifies an initial number of nodes for each phase,

```
problem.phases(iphase).nodes << INTEGER;
```

which seeds the first iteration as a single segment, together with (optionally) the controls listed below. The algorithm proceeds as follows.

1. Set the iteration index $m = 1$ and start from the seed mesh.
2. Solve the nonlinear programming problem on the current mesh and compute the per-segment errors $\epsilon_j^{(i,m)}$ for every phase i.
3. If $\max_j \epsilon_j^{(i,m)} \leq \epsilon_{\max}$ for all phases, terminate.
4. If $m \geq m_{\max}$ (`mr_max_iterations`), terminate.
5. For each phase build the next mesh: leave the converged segments unchanged and apply p- or h-refinement to each remaining segment according to its smoothness assessment, subject to the degree band $[N_{\min}, N_{\max}]$ and the per-iteration growth limit.
6. Interpolate the previous solution onto the new mesh to warm-start the next solve, set $m \leftarrow m + 1$, and return to step 2.

The parameters that control the procedure, with their default values, are as follows.

Maximum relative discretisation error, $\epsilon_{\max}$:

```
algorithm.ode_tolerance        = 1.e-3;
```

Maximum number of mesh refinement iterations, $m_{\max}$:

```
algorithm.mr_max_iterations    = 7;
```

Minimum per-segment polynomial degree, $N_{\min}$:

```
algorithm.mr_min_order         = 3;
```

Maximum per-segment polynomial degree, $N_{\max}$:

```
algorithm.mr_max_order         = 12;
```

Maximum per-iteration growth factor of N_{eff}:

```
algorithm.mr_max_growth_factor = 0.4;
```

The reader is referred to [12, 30, 24] for the details and convergence properties of hp-adaptive pseudospectral mesh refinement.

2.4 Automatic mesh refinement with local collocation

If a local collocation method (trapezoidal, Hermite-Simpson) is being used, and the user parameter `algorithm.mesh_refinement` is set to `"automatic"`, then, mesh refinement is carried out as described below. $\mathcal{PSOPT}$ will compute the discretisation error $\epsilon^{(i,m)}$ for every phase i at every mesh refinement iteration m, as described in Section 2.1. The method is the local mesh refinement algorithm of Betts, and is described in narrative form below; readers who need the algorithm stated in full, together with the derivation of its error-prediction and order-reduction formulae, should consult the book by Betts [4], which is the authoritative source. The order p of the discretisation is 2 for the trapezoidal method and 4 for the Hermite-Simpson method, and the two are tied together: raising p from 2 to 4 is what switches $\mathcal{PSOPT}$ from the trapezoidal to the Hermite-Simpson discretisation.

The user specifies an initial number of nodes for each phase, as follows:

```
problem.phases(iphase).nodes << INTEGER.
```

The user may also specify values for the following parameters which control the mesh refinement procedure. The default values are those shown:

Maximum discretisation error, $\epsilon_{\max}$:

```
algorithm.ode_tolerance  = 1.e-3;
```

Minimum increment factor, κ:

```
algorithm.mr_kappa   = 0.1;
```

Maximum growth factor, ρ:

```
algorithm.mr_max_growth_factor     = 0.4;
```

Maximum number of mesh refinement interations, $m_{\max}$:

```
algorithm.mr_max_iterations  = 7;
```

Maximum nodes to add within a single interval, M_1:

```
algorithm.mr_M1   = 5;
```

How the algorithm works

The refinement is driven by the per-interval discretisation error of Section 2.1 and proceeds as an outer loop of NLP solves. At each iteration $\mathcal{PSOPT}$ solves the nonlinear programming problem on the current mesh, computes the error on every interval of every phase, and stops if the largest error in every phase is already below $\epsilon_{\max}$, or if $m_{\max}$ iterations have been performed. Otherwise it builds a new mesh, and the way it does so rests on three ideas.

Choosing between a better method and a better mesh. A large error can mean either that the discretisation is not accurate enough or that the nodes are in the wrong places, and the two call for different remedies. $\mathcal{PSOPT}$ distinguishes between them by comparing the largest interval error in a phase with the average interval error in that phase. If the error is spread fairly evenly over the mesh, no redistribution of nodes will help much, and the limitation is the order of the method: $\mathcal{PSOPT}$ therefore raises p from 2 to 4, which switches the phase from the trapezoidal to the Hermite-Simpson discretisation. If instead the error is concentrated in a few intervals, the mesh is at fault, the order is left alone, and nodes are added where the error is.

Predicting what a node will buy. Adding a node to an interval reduces the error there at a rate governed by the order of the method, but the formal order p is only attained where the solution is smooth. On an interval that contains a corner, a switch or a discontinuity the observed order is lower, and refining such an interval pays much less than the formal order would suggest. $\mathcal{PSOPT}$ therefore estimates the *order reduction* on each interval empirically, by comparing the error on that interval before and after the previous subdivision, and uses the reduced order to predict the effect of adding a further node. This is what stops the algorithm from pouring nodes into an interval where refinement is not working.

Placing the nodes one at a time. New nodes are not distributed in one shot. $\mathcal{PSOPT}$ repeatedly finds the interval with the largest *predicted* error in the phase, adds a single node to it, and then updates that interval's predicted error using the estimated order before looking for the next candidate. Because the prediction is updated after every addition, successive nodes spread themselves over the intervals that need them rather than piling into the worst one. The placement loop for a phase ends when any of the following holds: enough nodes have been added for this iteration, the quota being $M' = \min(M_1, \kappa N_i) + 1$ where N_i is the current number of nodes in the phase; the predicted error everywhere in the phase has fallen below the tolerance; the growth limit of $\rho(N_i - 1)$ added nodes has been reached; or M_1 nodes have already gone into a single interval. The mesh is then rebuilt and the outer loop repeats.

What the parameters do. In terms of the description above, $\epsilon_{\max}$ (`ode_tolerance`) is the accuracy being asked for and the outer-loop stopping test; $m_{\max}$ (`mr_max_iterations`) caps the number of NLP solves; ρ (`mr_max_growth_factor`) caps how fast a phase may grow in one iteration, and so bounds the cost of the next solve; M_1 (`mr_M1`) caps how many nodes may be concentrated in a single interval, which prevents one troublesome interval from consuming the whole budget; and κ (`mr_kappa`) sets both the per-iteration node quota through M' and the margin by which the predicted error must beat the tolerance before the placement loop is allowed to stop early. Loosening ρ and M_1 makes each iteration more aggressive and usually reduces the number of iterations at the cost of a larger NLP; tightening them does the opposite.

The mesh refinement history — the discretisation method, number of nodes, problem size, function-evaluation counts and maximum relative error at each iteration — is reported in the mesh statistics table written by $\mathcal{PSOPT}$, so the behaviour of the algorithm on a given problem can be inspected directly.

3. Integrated-residual transcription

The collocation methods of the previous chapter enforce the dynamics only at a finite set of nodes (see the limitations noted in Section 1.13). For problems with singular arcs, stiff dynamics or near-discontinuous solutions, this can lead to constraint violation between the nodes, oscillatory controls and unreliable convergence. $\mathcal{PSOPT}$ therefore also provides an *integrated-residual* transcription, following the work of Nie, Kerrigan and co-workers [28, 27, 26, 29]. It is selected by setting `algorithm.transcription_method` to `"integrated-residual"`; the default is `"collocation"`.

3.1 Motivation: the integrated residual

Let the dynamic residual of a candidate trajectory be

$$r(t) = \dot{\tilde{x}}(t) - f[\tilde{x}(t), \tilde{u}(t), p, t], \tag{3.1}$$

where $\tilde{x}$ and $\tilde{u}$ are the interpolated state and control. Collocation drives r to zero at the nodes only. The integrated-residual method instead measures the dynamics through the integral of the squared residual over each phase,

$$R = \int_{t_0}^{t_f} \|r(t)\|^2 \, dt, \tag{3.2}$$

which is a genuine measure of how well the trajectory satisfies the differential equations *everywhere*, and not merely at the nodes [28]. Working with R turns the dynamic constraints into a single smooth scalar that can be minimised, penalised or bounded, giving the formulations described below.

3.2 Local representation and residual quadrature

By default, within each mesh interval $[t_k, t_{k+1}]$ the state is represented by a cubic Hermite polynomial built from the node values x_k, x_{k+1} and the node dynamics f_k, f_{k+1}, and the control by a quadratic through u_k, a mid-interval value and u_{k+1}. The residual (3.1) is then a known function of the decision variables on the interval, and the interval contribution to R is evaluated by Gauss–Legendre quadrature at m points,

$$\int_{t_k}^{t_{k+1}} \|r(t)\|^2 \, dt \approx \frac{h_k}{2} \sum_{q=1}^{m} \omega_q \, \|r(\xi_q)\|^2, \tag{3.3}$$

where $h_k = t_{k+1} - t_k$ and ξ_q, ω_q are the quadrature nodes and weights. The number of residual-quadrature points per interval is set by `algorithm.ir_residual_nodes`. Sampling the residual away from the mesh nodes is precisely what allows the method to "see" the inter-node dynamic error.

3.3 Least-squares and regularised formulations

The simplest use of the integrated residual is the *least-squares* transcription, in which the objective is the total residual itself,

$$\min_{y} \; R(y), \tag{3.4}$$

selected by `algorithm.ir_objective = "residual"`. This finds the most dynamically feasible trajectory for the given mesh, and is a robust way to obtain a feasible initial guess, but it ignores the user cost J.

To trade feasibility against optimality in a single solve, $\mathcal{PSOPT}$ can instead minimise a *regularised* objective, in which a weighted integrated residual is added to the user cost while the collocation defects are retained,

$$\min_{y} \; J(y) + \rho\, R(y), \tag{3.5}$$

where the penalty weight ρ is set by `algorithm.ir_regularization`. This integrated-residual regularisation damps the spurious control oscillations associated with singular arcs [28, 26]; it is currently available with the Hermite–Simpson collocation method.

3.4 The residual-box (robust) formulation

A more flexible way to control the dynamic error is to bound it directly. In the *residual-box* formulation the user cost is minimised subject to a box constraint on every residual component at every quadrature point,

$$\min_{y} \; J(y) \quad \text{subject to} \quad |r_j(\xi_q)| \le \delta \quad \forall\, j, q, \tag{3.6}$$

selected by setting `algorithm.ir_objective = "cost"` together with a non-negative tolerance `algorithm.ir_residual_bound` $= \delta$. The box makes the dynamics an explicit, bounded constraint rather than a penalty, so the solution accuracy is governed by the single parameter δ: as δ is reduced, the trajectory is forced closer to dynamic feasibility, at the cost of a tighter nonlinear program.

3.5 The Direct Alternating Integrated Residual (DAIR) algorithm

Choosing δ by hand, and converging the cost minimisation directly from a poor initial guess, can both be difficult. The Direct Alternating Integrated Residual (DAIR) algorithm of Nie and Kerrigan [28] removes both difficulties by alternating two sub-problems at each mesh:

1. a *feasibility* step that minimises the integrated residual R (the least-squares problem of Section 3.3), producing a dynamically feasible primal;
2. an *optimality* step that minimises the user cost J subject to the residual box of Section 3.4, warm-started from the feasibility solution.

The box tolerance is not chosen by the user but tied to the mesh scale. For a local representation of order p the residual behaves like the pth power of the node spacing, so $\mathcal{PSOPT}$ uses the horizon-normalised schedule

$$\delta_i = K\left(\frac{h_i}{T_i}\right)^p = \frac{K}{N_i^p}, \tag{3.7}$$

where h_i is the node spacing in phase i, T_i its time span, $N_i = T_i/h_i$ the number of intervals, p the order of the local representation ($p = 2$ for the default cubic-Hermite representation), and K is set by `algorithm.ir_dair_delta_factor` (default 1). Normalising the spacing by the phase duration is what makes the schedule scale invariant. The absolute spacing h_i grows with the horizon, so the unnormalised choice $\delta_i = K\,h_i^p$ would let the tolerance grow as T_i^p and become non-binding on a long horizon; on the Fuller problem of Section 3.8 ($T = 300$) the unnormalised box exceeded 10^2 and the optimality step collapsed to a degenerate solution. With the normalisation the tolerance depends only on the interval count N_i, so a single K of order unity serves problems whose time scales differ by orders of magnitude. As the mesh-refinement loop refines the grid, N_i grows and hence δ_i falls, and the cost converges monotonically to the optimum without the continuation difficulties of a fixed penalty weight. Each phase receives its own tolerance δ_i. DAIR is enabled by setting `algorithm.ir_dair = true`.

3.6 Costate recovery by adjoint post-processing

When the dynamics are imposed by the residual box rather than by defect equalities, the box multipliers are sensitivities to δ and do not provide costate estimates. $\mathcal{PSOPT}$ therefore recovers the costates by adjoint post-processing: along the computed primal trajectory it integrates the adjoint equation

$$\dot{\lambda} = -\left(\frac{\partial L}{\partial x} + \left(\frac{\partial f}{\partial x}\right)^{\top}\lambda\right) \tag{3.8}$$

backwards in time from the transversality condition

$$\lambda(t_f) = \frac{\partial \varphi}{\partial x_f} + \sum_e \nu_e \frac{\partial e_e}{\partial x_f}, \tag{3.9}$$

where the ν_e are the multipliers of any terminal event constraints. This single expression covers the free-terminal, Mayer and fixed-terminal cases without special handling. The required Jacobians are formed from the user functions by finite differences. The recovered costates are returned in the solution structure in the same way as for the collocation methods.

3.7 The Nie–Kerrigan flexible-order representation

The default cubic-Hermite representation is second order. For smooth problems a higher-order local representation gives much greater accuracy for the same number of nodes. $\mathcal{PSOPT}$ provides the flexible-order representation of Nie and Kerrigan [28, 27], enabled by setting `algorithm.ir_local_order` to an integer $d \geq 2$ (the default 0 retains the cubic-Hermite representation).

The mesh nodes are grouped into elements of $d+1$ local Legendre–Gauss–Lobatto (LGL) sub-nodes; on each element the state and control are degree-d Lagrange polynomials through those sub-nodes, with C^0 continuity at the shared element boundaries. With $M = N/d$ elements, the number of intervals N per phase must be a multiple of d. Refining the mesh (h-refinement) and raising the order d (p-refinement) can both be used to drive the residual down; for smooth solutions the convergence in d is spectral. The residual is integrated per element, and the Lagrange term of the cost is evaluated with the element LGL (Lobatto) quadrature, which is exact to degree $2d-1$ and so matched to the order of the representation.

The flexible-order representation composes with both the residual box and DAIR. When used with DAIR, the box schedule (3.7) is made order-aware, $\delta_i = K\,(h_i/T_i)^d$, so that the tolerance tracks the achievable residual of the degree-d representation. As in the cubic-Hermite case, the costates are recovered by the adjoint post-processing of Section 3.6.

3.8 Behaviour in practice

Two examples distributed with $\mathcal{PSOPT}$ illustrate the behaviour of the transcription and the practical lessons that shaped it.

A smooth singular problem (`examples/x1`). The analytic problem of Neuenhofen and Kerrigan [26],

$$\min \int_0^{\pi/2} \left(y^2 + \cos(t)\,u\right) dt \quad \text{s.t.} \quad \dot{y} = \tfrac{1}{2}y^2 + u,\;\; y(0) = 0,\;\; -1.5 \leq u \leq 1, \tag{3.10}$$

has a known optimal cost $J^\star$ and a smooth solution on which ordinary Hermite–Simpson collocation nonetheless rings. It demonstrates the value of the flexible-order representation directly: with the order-aware box, raising the local order d drives the cost error $|J - J^\star|$ down at the expected p-rate, from order 10^{-2} at $d=2$ to order 10^{-7} at $d=5$. It also exposes a subtlety that motivates the higher-order representation. Tightening the box on the *low*-order cubic-Hermite representation does not improve the cost; it makes it *worse*, because a second-order representation cannot be simultaneously optimal and dynamically accurate to a tight tolerance. Only raising the local order lets the trajectory satisfy a tight box while remaining optimal, so the order of the local representation must be matched to the accuracy demanded of the box.

A chattering problem (`examples/fuller`). Fuller's problem [29], in state form $x_1 = p$, $x_2 = \dot{p}$,

$$\min \int_0^T x_1^2 \, dt \quad \text{s.t.} \quad \dot{x}_1 = x_2, \ \ \dot{x}_2 = u, \ \ |u| \le 0.01, \tag{3.11}$$

with $x_1(0) = x_1(T) = x_2(T) = 0$, $x_2(0) = 1$ and $T = 300$, has an optimal control that chatters: it switches infinitely often, the switches accumulating near $t \approx 280$. Collocation captures a few switches but leaves an uncontrolled inter-node residual of order 10^{-2}. The residual box instead turns the computed cost into a *certified lower bound* on the true optimum that rises towards it as δ is tightened: on a fixed mesh, reducing δ from 10^{-3} to 10^{-6} raises the computed cost monotonically towards the optimal value, tracing an accuracy–cost Pareto front that ordinary collocation cannot provide.

The Fuller example records two honest practical points. First, on a strongly non-smooth problem the integrated-residual optimality step should be warm-started from a feasibility (or collocation) solution; a cold start from a poor guess can converge to a degenerate low-cost, high-residual point. This is exactly the feasibility-then-optimality alternation that DAIR automates. Second, on a fixed mesh raising the local order helps a discontinuous solution only up to a point: beyond order two the switch discontinuities induce Gibbs-type oscillations that a higher-degree polynomial does not remove. Capturing such discontinuities cleanly requires a mesh that can place nodes at the switches — a flexible or *hp*-adaptive mesh [29] — rather than a higher order on a fixed one.

4. Choosing a suitable discretisation approach for a given optimal control problem

$\mathcal{PSOPT}$ provides several families of discretisation: the global pseudospectral methods (the Legendre and Chebyshev methods, which collocate at both endpoints, and the Radau and Gauss methods of Section 1.10); the local methods (trapezoidal and Hermite–Simpson, Section 1.11); optional automatic mesh refinement (the *hp*-adaptive driver for the pseudospectral methods, Section 2.3, or the Betts refinement for the local methods, Section 2.4); and the integrated-residual transcription of Chapter 3. There are no universal rules, but some kinds of problem are solved more easily and accurately by particular approaches. The most relevant factors are the smoothness of the solution, the presence of discontinuities or active constraint arcs, the accuracy required, the problem size, and whether costate estimates or a certified measure of dynamic feasibility are needed. This chapter offers practical guidance; the methods themselves are described in the earlier chapters.

4.1 Pseudospectral or local discretisation?

The pseudospectral methods approximate the trajectory on each mesh segment by a single high-degree polynomial and converge *spectrally* for smooth solutions: the error decreases faster than any fixed power of the number of nodes, so a smooth problem can be solved to very high accuracy with comparatively few nodes. The cost is a dense Jacobian block per segment, whose size grows with the polynomial degree. The local methods place a low-order rule (second-order trapezoidal or fourth-order Hermite–Simpson) on each subinterval; they converge only algebraically, but their banded, sparse Jacobian scales well to large meshes and they are robust when the solution is nonsmooth. As a rule of thumb, a smooth solution for which high accuracy is wanted favours a pseudospectral method, whereas a nonsmooth solution (bang–bang control, kinks, or many active path or state constraints) or one that simply needs a large number of nodes favours a local method — or an *hp*-adaptive pseudospectral mesh, which combines the two behaviours.

4.2 Choosing among the pseudospectral methods

For a smooth single-phase problem the Legendre and Chebyshev methods are good general-purpose choices; both collocate the endpoints and return costate estimates at every node (Section 1.9.1), and the Chebyshev cost integral is evaluated with the spectrally accurate Clenshaw–Curtis quadrature of Section 1.8.8. The Radau method collocates a single endpoint and underpins most of the convergence and costate theory used by the *hp*-adaptive driver, which makes it a robust default when automatic refinement is enabled. The Gauss method collocates the strictly interior points and treats the terminal state as an additional variable; it is attractive when accurate interior costates are important. In practice the four methods give comparable results on smooth problems, and the choice is often dictated by whether automatic refinement is used (the Radau and Gauss methods have the most developed *hp* theory) rather than by accuracy alone.

4.3 Choosing among the local methods

The trapezoidal method is second order and very robust; it is a good choice for an initial or coarse solve, for strongly nonsmooth problems, or as the starting order of an automatic local refinement. The Hermite–Simpson method is fourth order and gives markedly better accuracy for a similar node count when the solution is reasonably smooth. The `switch_order` option lets an automatic local refinement begin with the trapezoidal method and switch to Hermite–Simpson once the mesh is adequate (Section 2.4).

4.4 Whether to use mesh refinement

If the solution is globally smooth, a single pseudospectral solve on a sensible mesh may already meet the required accuracy, and manual refinement (trying a short increasing sequence of node counts) is often enough. Automatic refinement is worthwhile when the solution has localized features — boundary layers, fast transients, or near-discontinuities — or when a target accuracy is wanted without hand-tuning the mesh. The *hp*-adaptive driver (Section 2.3) raises the polynomial degree where the solution is smooth and subdivides the mesh where it is not, and is the recommended automatic strategy for the pseudospectral methods; the Betts refinement (Section 2.4) plays the same role for the local methods.

4.5 When to use the integrated-residual transcription

Standard collocation enforces the dynamics only at the collocation nodes, leaving the residual between nodes uncontrolled. The integrated-residual transcription of Chapter 3 instead measures, and optionally bounds, the dynamic residual *between* the nodes. This provides a certifiable measure of dynamic feasibility and guards against the inter-node constraint violation that ordinary collocation can hide. It is the preferred choice when a guaranteed bound on the dynamic infeasibility is required, when the dynamics are stiff or oscillatory so that the inter-node error matters, or when robustness is valued over raw speed. The DAIR algorithm (Section 3.5) automates the residual-box tolerance schedule, the Nie–Kerrigan flexible-order representation (Section 3.7) raises the local approximation order, and costates can be recovered by adjoint post-processing. The transcription is more expensive than plain collocation, so it is best reserved for the situations above rather than used by default.

4.6 Summary guidance

Table 4.1 summarises which approaches tend to suit which problem features. The entries are guidelines, not guarantees.

Problem feature	**Global PS**	***hp*-PS**	**Local**	**Integ. res.**
Smooth solution, high accuracy wanted	✓	✓	◦	◦
Nonsmooth control (bang–bang, kinks)	–	✓	✓	✓
Localized features (boundary/internal layers)	–	✓	◦	✓
Many active path/state-constraint arcs	◦	✓	✓	✓
Large-scale (very many nodes)	–	◦	✓	◦
Costate / adjoint estimates needed	✓	✓	◦	✓
Certified inter-node feasibility needed	–	–	–	✓
Solution structure unknown, mesh hard to choose	–	✓	◦	✓

Table 4.1: Guidance on discretisation choice by problem feature. “Global PS” denotes a pseudospectral method (Legendre, Chebyshev, Radau or Gauss) on a single fixed mesh; “*hp*-PS” the same methods with *hp*-adaptive refinement; “Local” the trapezoidal or Hermite–Simpson methods; “Integ. res.” the integrated-residual transcription. The symbol ✓ marks an approach that is usually well suited, ◦ one that is usable but with caveats (for example, a local method can reach high accuracy on a smooth problem but only with many nodes), and – one that is normally a poor fit.

In summary, begin with a pseudospectral method when the solution is expected to be smooth and high accuracy is the priority; switch to, or combine with, *hp*-adaptive refinement when the solution has localized or nonsmooth features; prefer a local method for large or strongly nonsmooth problems; and use the integrated-residual transcription when a certified measure of dynamic feasibility, or control of the residual between nodes, is required.

5. Solution diagnostics

Setting the option `algorithm.diagnostic_level` to a positive value causes $\mathcal{PSOPT}$ to print, after the solve has finished, a report that helps to interpret and validate the computed solution. The report is purely diagnostic: at level 1 it reads only quantities that $\mathcal{PSOPT}$ has already computed for the converged solution (the per-interval relative ODE errors, the states, the costates and the stored Hamiltonian), and at level 2 it performs in addition a small number of user-function evaluations. With `diagnostic_level` $= 0$, the default, the diagnostics are switched off entirely and the solve is unaffected.

At `diagnostic_level` ≥ 1 the report contains, for each phase:

- *discretisation-error localisation.* The intervals carrying the largest relative ODE error are listed with their midpoint times, identifying where the discretisation is least accurate.

- *Spectral smoothness.* For the global pseudospectral methods a per-state smoothness indicator σ is reported, computed as the exponential decay rate of the Legendre-coefficient envelope of the state over the phase, in the spirit of the non-smoothness detection of Liu, Hager and Rao [24]. The decay rate is fitted only over the region of genuine decay, so that the round-off plateau of a well-resolved state is not mistaken for slow decay; a small σ flags a state that is not smooth. Read together with the error localisation, this distinguishes the two reasons an interval may be inaccurate: a large error with high σ indicates a smooth but under-resolved region, for which the order should be raised, whereas a large error with low σ indicates genuine non-smoothness, for which the interval should be subdivided or the integrated-residual transcription of Chapter 3 used. The indicator is reported for the global pseudospectral methods only.

- *Costate structure.* For each state the peak and root-mean-square magnitude and the sign behaviour of the costate are summarised, and a costate that is negligible against the dominant one is flagged as near-null, indicating a state whose adjoint is structurally inactive. The peak magnitudes of any path-constraint multipliers are reported alongside.

At `diagnostic_level` ≥ 2 the report additionally contains two first-order optimality residuals associated with the necessary conditions of optimality [5]:

- *Hamiltonian constancy.* The normalised spread $(\max - \min)/(|\text{mean}| + \epsilon)$ of the stored Hamiltonian $H = L + \lambda^\top f$ along the trajectory. For an autonomous problem

H is constant along the optimal trajectory, so a small spread is an optimality indicator; for an explicitly time-varying problem H need not be constant, and a large spread is then not by itself a defect.

- *Control stationarity.* The stationarity condition $\partial H/\partial u = 0$, evaluated by central finite differences of H formed from the user `dae` and `integrand_cost` functions. The condition holds only at a free interior control with no active path constraint, so the residual is summarised as a maximum taken over exactly those nodes; control nodes lying at an active bound or at an active-path node are excluded and counted separately, because there $\partial H/\partial u$ is balanced by the corresponding multiplier and is expected to be nonzero.

The error localisation and the smoothness indicator are most useful for guiding manual mesh refinement (Section 2.3), while the costate and optimality diagnostics are most useful for confirming that a converged solution satisfies the necessary conditions of optimality.

6. Defining optimal control and estimation problems for $\mathcal{PSOPT}$

Defining an optimal control or parameter estimation problem involves specifying all the necessary values and functions that are needed to solve the problem. With $\mathcal{PSOPT}$, this is done by implementing C++ functions (e.g. the cost function), and assigning values to data structures which are described below. Once a $\mathcal{PSOPT}$ has obtained a solution, the relevant variables can be obtained by interrogating a data structure.

6.1 Interface data structures

The role of each structure used in the $\mathcal{PSOPT}$ interface is summarised below.

- *Problem data structure:* This structure is used to specify problem information, including the number of phases and pointers to the relevant functions, as well as phase related information such as number of states, controls, parameters, number of grid points, bounds on variables (e.g. state bounds), and functions (e.g. path function bounds).
- *Algorithm data structure:* This is used to control the solution algorithm and to pass parameters to the NLP solver.
- *Solution data structure:* This is used to store the resulting variables of a $\mathcal{PSOPT}$ run.

6.2 Required functions

Table 6.1 lists and describes the parameters used by the interface functions.

6.2.1 `endpoint_cost` function

The purpose of this function is to specify the terminal costs $\phi_i[\cdot]$, $i = 1, \ldots, N$. The function prototype is as follows:

```
adouble endpoint_cost(adouble* initial_states,
                      adouble* final_states,
                      adouble* parameters,
```

Parameter	Type	Role	Description
`controls`	adouble*	input	Array of intantaneous controls
`derivatives`	adouble*	output	Array of intantaneous state derivatives
`e`	adouble*	output	Array of event constraints
`final_states`	adouble*	input	Array of final states within a phase
`initial_states`	adouble*	input	Array of initial states within a phase
`iphase`	int	input	Phase index (starting from 1)
`linkages`	adouble*	output	Array of linkage constraints
`parameters`	adouble*	input	Array of static parameters within a phase
`states`	adouble*	input	Array of intantaneous states within a phase
`time`	adouble	input	Instant of time within a phase
`t0`	adouble	input	Initial phase time
`tf`	adouble	input	final phase time
`xad`	adouble*	input	vector of scaled decision variables
`workspace`	Workspace*	input	Pointer to workspace structure

Table 6.1: Description of parameters used by the $\mathcal{PSOPT}$ interface functions

```
                      adouble& t0, adouble& tf,
                      adouble* xad, int iphase,
                      Workspace* workspace)
```

The function should return the value of the end point cost, depending on the value of phase index `iphase`, which takes on values between 1 and `problem.nphases`.

Example of writing the endpoint cost function for a single phase problem with the following endpoint cost:

$$\varphi(x(t_f)) = x_1(t_f)^2 + x_2(t_f)^2 \tag{6.1}$$

```
adouble endpoint_cost(adouble* initial_states,
                      adouble* final_states,
                      adouble* parameters,
                      adouble& t0, adouble& tf,
                      adouble* xad, int iphase,
                      Workspace* workspace)
{
    adouble x1f = final_states[ 0 ];
    adouble x2f = final_states[ 1 ];

    return ( x1f*x1f + x2f*x2f);
}
```

6.2.2 `integrand_cost` function

The purpose of this function is to specify the integrand costs $L_i[\cdot]$ for each phase as a function of the states, controls, static parameters and time. The function prototype is as follows:

```
adouble integrand_cost(adouble* states,
                       adouble* controls,
                       adouble* parameters,
                       adouble& time,
                       adouble* xad,
                       int iphase,
                       Workspace* workspace)
```

The function should return the value of the integrand cost, depending on the phase index `iphase`, which takes on values between 1 and `problem.nphases`.

Example of writing the integrand cost for a single phase problem with L given by:

$$L(x(t), u(t), t) = x_1(t)^2 + x_2(t)^2 + 0.01u(t)^2 \tag{6.2}$$

```
adouble integrand_cost(adouble* states,
                       adouble* controls,
                       adouble* parameters,
                       adouble& time,
                       adouble* xad,
                       int iphase,
                       Workspace* workspace)
{
    adouble x1 = states[   0 ];
    adouble x2 = states[   1 ];
    adouble  u = controls[ 0 ];

    return ( x1*x1 + x2*x2 + 0.01*u*u );
}
```

If the problem does not involve any cost integrand, the user may simply not register any `cost_integrand` function, or register it as follows: `problem.cost_integrand=NULL`.

6.2.3 `dae` function

This function is used to speficy the time derivatives of the states $\dot{x}^{(i)} = f_i[\cdot]$ for each phase as a function of the states themselves, controls, static parameters, and time, as well as the algebraic functions related to the path constraints. Its prototype is as follows:

```
void dae(adouble* derivatives,
         adouble* path,
         adouble* states,
         adouble* controls,
         adouble* parameters,
         adouble& time,
         adouble* xad,
         int iphase,
         Workspace* workspace)
```

This is an example of writing the `dae` function for a single phase problem with the following state equations and path constraints:

$$\begin{aligned} \dot{x}_1 &= x_2 \\ \dot{x}_2 &= -3\exp(x_1) - 4x_2 + u \\ 0 &\leq x_1^2 + x_2^2 \leq 1 \end{aligned} \tag{6.3}$$

```
void dae(adouble* derivatives,
         adouble* path,
         adouble* states,
         adouble* controls,
         adouble* parameters,
         adouble& time,
         adouble* xad,
         int iphase,
         Workspace* workspace)
{
    adouble x1 = states[   0 ];
    adouble x2 = states[   1 ];
    adouble  u = controls[ 0 ];

    derivatives[ 0 ] = x2;
    derivatives[ 1 ] = -3*exp(x1)-4*x2+u;
    path[        0 ] = x1*x1 + x2*x2
}
```

Note that the bounds on the path constraints are specified separately, possibly in the `main()` function, as follows:

```
problem.phases(1).bounds.upper.path(0) = 0.0;
problem.phases(1).bounds.lower.path(0) = 1.0;
```

6.2.4 `events` function

This function is used to speficy the values of the event constraint functions $e_i[\cdot]$ for each phase. Its prototype is as follows:

```
void events(adouble* e,
            adouble* initial_states,
            adouble* final_states,
            adouble* parameters,
            adouble& t0,
            adouble& tf,
            int iphase,
            Workspace* workspace)
```

Please note, if the initial or final values of the control variables within a phase need to be accessed within the `events()` function, this can be done through the functions `get_initial_controls()` and `get_final_controls()`. See sections 6.10.11 and 6.10.12.

The following is an example of writing the `events` function for a single phase problem with the event constraints:

$$\begin{aligned} 1 &= e_1(t_0) = x_1(t_0) \\ 2 &= e_2(t_0) = x_2(t_0) \\ -0.1 \leq e_3(t_0) &= x_1(t_f)x_2(t_f) \leq 0.1 \end{aligned} \tag{6.4}$$

```
void events(adouble* e,
            adouble* initial_states,
            adouble* final_states,
            adouble* parameters,
            adouble& t0,
            adouble& tf,
            int iphase,
            Workspace* workspace)
{
    adouble x1i = initial_states[   0 ];
    adouble x2i = initial_states[   1 ];
    adouble x1f = final_states[     0 ];
    adouble x2f = final_states[     1 ];


    e[ 0 ] = x1i;
    e[ 1 ] = x2i;
    e[ 2 ] = x1f*x2f;
```

```
}
```

Note that the bounds on the event constraints are specified separately, possibly in the `main()` function:

```
problem.phases(1).bounds.lower.events(0) = 1.0;
problem.phases(1).bounds.upper.events(0)= 1.0;
problem.phases(1).bounds.lower.events(1)= 2.0;
problem.phases(1).bounds.upper.events(1)= 2.0;
problem.phases(1).bounds.lower.events(2)=-0.1;
problem.phases(1).bounds.upper.events(2)=0.1;
```

6.2.5 `linkages` function

This function is used to speficy the values of the phase linkage constraint functions $\Psi[\cdot]$. Its prototype is as follows:

```
void linkages( adouble* linkages,
               adouble* xad,
               Workspace* workspace)
```

The following is an example of writing the `linkages` function for a two phase problem with two states in each phase and with the linkage constraints:

$$\begin{aligned} 0 = \Psi_1 &= x_1^{(0)}(t_f^{(1)}) - x_1^{(2)}(t_i^{(2)}) \\ 0 = \Psi_2 &= x_2^{(1)}(t_f^{(1)}) - x_2^{(2)}(t_i^{(2)}) \\ 0 = \Psi_3 &= t_f^{(1)} - t_i^{(2)} \end{aligned} \tag{6.5}$$

These type of state and time continuity constraint can be entered automatically by using the `auto_link` function as shown below. Note that phase linkage bounds are set to zero by default. If they are different from zero, they need to be specified explicitly as shown in the second example below.

```
void linkages( adouble* linkages, adouble* xad, Workspace* workspace)
{
    int index = 0;
    // Link phases 1 and 2
    auto_link( linkages, &index, xad, 1, 2 );
}
```

It is of course also possible to implement more general linkage constraints, as illustrated through the following example.

Consider a two phase problem with two states in each phase and with the nonlinear linkage constraints:

$$\begin{aligned} 0 = \Psi_1 &= x_1^{(1)}(t_f^{(1)}) - \sin[x_1^{(2)}(t_i^{(2)})] \\ 0 = \Psi_2 &= x_2^{(1)}(t_f^{(1)}) - \cos[x_2^{(2)}(t_i^{(2)})] \\ 10 \le \Psi_3 &= t_f^{(1)} - t_i^{(2)} \le 100 \end{aligned} \tag{6.6}$$

```
void linkages( adouble* linkages, adouble* xad, Workspace* workspace)
{
     adouble xf_p0[  2  ];
     adouble xi_p1[  2  ];
     adouble tf_p0;
     adouble ti_p1;

     get_final_states(   xf_p0, xad, 1 );
     get_initial_states( xf_p1, xad, 2 );
     tf_p0 = get_final_time( xad, 1);
     ti_p1 = get_initial_time( xad, 2);

     linkages[0]=xf_p0[0]-sin(xi_p1[0]);
     linkages[1]=xf_p0[1]-cos(xi_p1[1]);
     linkages[2]=tf_p0 - ti_p1;
}
```

Note that the bounds are specified separately, possibly in the `main()` function:

```
problem.bounds.lower.linkage(0)= 0.0;
problem.bounds.upper.linkage(0)= 0.0;
problem.bounds.lower.linkage(1)= 0.0;
problem.bounds.upper.linkage(1)= 0.0;
problem.bounds.lower.linkage(2)= 10.0;
problem.bounds.upper.linkage(2)= 100.0;
```

6.2.6 Using the power of Eigen from within the user functions

Eigen is a C++ template library for linear algebra, and thus it is possible to use this to create Eigen matrices with `adouble` elements, and to make operations with them from within the user functions. Recall that `adouble` is a type defined for automatic differentiation purposes. This has the potential of simplifying the implementation of user functions.

As an example, consider the `dae()` function. The list of arguments of this function includes arrays of type adouble named `states`, `controls`, `path` and `derivatives`. It is possible, for example, in the case of a problem with 3 states, 2 controls and 2 path constraints, to map these arrays into Eigen objects as follows:

```
Eigen::Map<Eigen::Matrix<adouble, 3, 1>> x(states, 3);
Eigen::Map<Eigen::Matrix<adouble, 2, 1>> u(controls, 2);
Eigen::Map<Eigen::Matrix<adouble, 2, 1>> p(path, 2);
Eigen::Map<Eigen::Matrix<adouble, 2, 1>> dx(derivatives, 3);
```

The user can then use these objects with the syntax and compatible functions that are available through Eigen. For example, assume that the dynamics are given by the linear equation $\dot{x} = Ax + Bu$, with A and B being matrices of the appropriate dimmension. One could write the following code inside the `dae()` function:

```
// Define the model matrices A and B
Eigen::Matrix<adouble, 3, 3> A;
Eigen::Matrix<adouble, 3, 2> B;
// Assume that values are assigned to the elements of A and B.
// Assume that x, u and dx are defined as above.
// The following code performs the necessary matrix operations
// and assigns the result to the derivatives array.
dx = A*x + B*u;
```

6.2.7 Main function

Declaration of data structures

The `main()` function is used to declare and initialise the `problem`, `solution`, and `algorithm` data structures, to call the $\mathcal{PSOPT}$ algorithm, and to post-process the results. To declare the data structures the user may wish to use the following commands:

```
Alg  algorithm;
Sol  solution;
Prob problem;
```

Problem level information

The user should then define the problem name as follows:

```
problem.name = "My problem";
```

The number of phases and the number of linkage constraints should be declared afterwards. For example, for a single phase problem:

```
problem.nphases=1;
problem.nlinkages=0;
```

This declaration should be followed by the following function call, which initialises problem level structures.

```
psopt_level1_setup(problem);
```

Please note, $\mathcal{PSOPT}$ is able to solve problems with multiple phases, and some parameters need to be passed for each particular phase. In the $\mathcal{PSOPT}$ interface, the numbering of phases starts from 1, so that the first phase of the problem is phase 1, and problems with only a single phase have their phase number identified as 1.

Phase level information

After this, the user needs to specify phase level parameters using the following syntax:

```
problem.phases(iphase).nstates                  = INTEGER;
problem.phases(iphase).ncontrols                = INTEGER;
problem.phases(iphase).nevents                  = INTEGER;
problem.phases(iphase).npath                    = INTEGER;
```

where `iphase` represents the number of the phase for which the parameters are being entered.

To specify the number of nodes (time instants) in the solution grid, there are different possibilities. If a single solution is required with a fixed number of nodes, this can be specified as follows:

```
problem.phases(iphase).nodes                    << INTEGER
```

If a sequence of nodes needs to be tried, so that manual mesh refinement is perform with an increasing number of nodes from solution to solution, and using the previous solution as a guess, then this can be specified in the example below:

```
problem.phases(iphase).nodes=(RowVectorXi(2)<<50, 60).finished();
```

Note, the number 2 in `RowVectorXi(2)` indicates the number of elements in the sequence, and the sequence of nodes itself is specified by the values $50, 60$. It is also possible to define the number of nodes using a previously defined variable of type `RowVectorXd`, as in the followign example:

```
RowVectorXi my_vector << 50, 60;
problem.phases(iphase).nodes = my_vector;
```

Once the phase level dimensions have been specified it is necessary to call the following function:

```
psopt_level2_setup(problem, algorithm);
```

Phase bounds information

The syntax to enter state bounds is as follows:

```
problem.phases(iphase).bounds.lower.states(j) = REAL;
problem.phases(iphase).bounds.upper.states(j) = REAL;
```

where `iphase` is a phase index between 1 and `problem.nphases`, and `j` is an index between 0 and `problem.phases(iphase).nstates-1`.

The syntax to enter control bounds is as follows:

```
problem.phases(iphase).bounds.lower.controls(j) = REAL;
problem.phases(iphase).bounds.upper.controls(j) = REAL;
```

where `j` is an index between 0 and `problem.phases(iphase).ncontrols`-1.

The syntax to enter event bounds is as follows:

```
problem.phases(iphase).bounds.lower.events(j) = REAL;
problem.phases(iphase).bounds.upper.events(j) = REAL;
```

where `j` is an index between 0 and `problem.phases(iphase).nevents`.

The bounds on the start time for each phase are entered using the following syntax:

```
problem.phases(iphase).bounds.lower.StartTime = REAL;
problem.phases(iphase).bounds.upper.StartTime = REAL;
```

The bounds on the end time for each phase are entered using the following syntax:

```
problem.phases(iphase).bounds.lower.EndTime = REAL;
problem.phases(iphase).bounds.upper.EndTime = REAL;
```

Linkage bounds information

The syntax to enter linkage bounds values is as follows:

```
problem.bounds.lower.linkage(j) = REAL;
problem.bounds.upper.linkage(j) = REAL;
```

`j` is an index between 0 and `problem.nlinkages`. The default value of the linkage bounds is zero.

Specifying the initial guess for each phase

The user may wish to specify an initial guess for the solution, rather than allowing $\mathcal{PSOPT}$ to determine the initial guess automatically. The user may specify any of the following for each phase: the control vector history, the state vector history and the time vector corresponding to the control and state histories, as well as a guess for the static parameter vector.

To specify the initial guesses for each phase, the user needs to create DMatrix objects to hold the initial guesses, and then assign the addresses of these objects to relevant pointers within the Guess structure.

The syntax to specify the initial guess in a particular phase is as follows:

```
problem.phases(iphase).guess.controls       = uGuess;
problem.phases(iphase).guess.states         = xGuess;
problem.phases(iphase).guess.parameters     = pGuess;
problem.phases(iphase).guess.time           = tGuess;
```

where uGuess, xGuess, pGuess and tGuess are `MatrixXd` objects that contain the relevant guesses. Users may find it useful to employ the functions `zeros`, `ones`, and `linspace` to specify the initial guesses.

For example, the following code creates an object to store an initial control guess with 20 grid points, and assigns zeros to it.

```
MatrixXd uGuess = zeros(1, 20);
```

The following code defines a linear history in the interval $[10, 15]$ for the first state, and a constant history for the second state, for a system with two states assuming 20 grid points:

```
MatrixXd xGuess(2,20);
xGuess.row(0) = linspace( 10, 15, 20); // First row
xGuess.row(1) = 10*ones( 1, 20); // Second row
```

The following code defines a time vector with equally spaced values in the interval $[0, 10]$ assuming 20 grid points:

```
MatrixXd tGuess = linspace( 0, 10, 20);
```

Scaling information

The user may wish to supply the scaling factors rather than allowing $\mathcal{PSOPT}$ to compute them automatically.

Scaling factors for controls, states, event constraints, derivatives, path constraints, and time for each phase can be entered as follows:

```
problem.phases(iphase).scale.controls(j)         = REAL;
problem.phases(iphase).scale.states(j)           = REAL;
problem.phases(iphase).scale.events(j)           = REAL;
problem.phases(iphase).scale.defects(j)          = REAL;
problem.phases(iphase).scale.time                = REAL;
```

The scaling factor for the objective function is entered as follows:

```
problem.scale.objective                          = REAL;
```

Scaling factors for the linkage constraints are entered as follows:

```
problem.scale.linkage(j)                        = REAL;
```

Passing user data to the interface functions

It is possible to pass user data to the different interface functions by using a void pointer that is a member of the `problem` data structure. This is good programming practice, as it helps avoid the use of global or static variables.

For example, the user could define a data structure to encapsulate some key data values, such as:

```
 typedef struct{

 double c1;
 double c2;
 double c3;

 } Mydata;
```

In the `main` function, an instance of this data structure can be allocated:

```
Mydata* md = (Mydata*) malloc(sizeof(Mydata));
```

and its elements given values:

```
md->c1 = 1.0;
md->c2 = 100.0;
md->c3 = 1000.0;
```

In the `main()` function, after the problem data structure has been declared, then the pointer to the instance of the `Mydata` data structure can be stored in the `user_data` member of the `problem` data structure, as follows:

```
problem.user_data = (void*) md;
```

Finally, the user can access this data from within the interface functions. For example, to access the data from within the `integrand_cost` function, the following can be done:

```
adouble integrand_cost(adouble* states,
adouble* controls,
adouble* parameters,
adouble& time,
adouble* xad,
int iphase,
Workspace* workspace)
{
adouble x1 = states[   0 ];
adouble x2 = states[   1 ];
adouble  u = controls[ 0 ];

Mydata* md = (Mydata*) workspace->problem->user_data;

double c1 = md->c1;
double c2 = md->c2;
double c3 = md->c3;

return ( c1*x1*x1 + c2*x2*x2 + c3*u*u );

}
```

Specifying algorithm options

Algorithm options and parameters can be specified as follows:

```
algorithm.nlp_method                              = STRING;
algorithm.scaling                                 = STRING;
algorithm.defect_scaling                          = STRING;
algorithm.derivatives                             = STRING;
algorithm.collocation_method                      = STRING;
```

```
algorithm.transcription_method                    = STRING;
algorithm.ir_objective                            = STRING;
algorithm.ir_residual_nodes                       = INTEGER;
algorithm.ir_residual_bound                       = REAL;
algorithm.ir_regularization                       = REAL;
algorithm.ir_dair                                 = BOOLEAN;
algorithm.ir_dair_delta_factor                    = REAL;
algorithm.ir_local_order                          = INTEGER;
algorithm.nlp_iter_max                            = INTEGER;
algorithm.nlp_tolerance                           = REAL;
algorithm.print_level                             = INTEGER;
algorithm.jac_sparsity_ratio                      = REAL;
algorithm.hess_sparsity_ratio                     = REAL;
algorithm.hessian                                 = STRING;
algorithm.hessian_verify                          = BOOLEAN;
algorithm.diagnostic_level                        = INTEGER;
algorithm.on_error                                = STRING;
algorithm.mesh_refinement                         = STRING;
algorithm.ode_tolerance                           = REAL;
algorithm.mr_max_iterations                       = INTEGER;
algorithm.mr_min_order                            = INTEGER;
algorithm.mr_max_order                            = INTEGER;
algorithm.mr_max_growth_factor                    = REAL;
algorithm.mr_kappa                                = REAL;
algorithm.mr_M1                                   = INTEGER;
algorithm.switch_order                            = INTEGER;
algorithm.ipopt_linear_solver                        = STRING
```

Note that:

- `algorithm.nlp_method` takes the (only) option "IPOPT" (default).
- `algorithm.scaling` takes the options "automatic" (default) or "user".
- `algorithm.defect_scaling` takes the options "state-based" (default) or "jacobian-based".
- `algorithm.derivatives` takes the options "automatic" (default) or "numerical".
- `algorithm.collocation_method` takes the options "Legendre" (default), "Chebyshev", "trapezoidal", "Hermite-Simpson", "Radau", or "Gauss". The "Radau" and "Gauss" options select the Legendre–Gauss–Radau and Legendre–Gauss pseudospectral discretisations of Section 1.10.
- `algorithm.transcription_method` takes the options "collocation" (default) or "integrated-residual". The latter selects the integrated-residual transcription of Chapter 3; the options below apply only in that case.

- `algorithm.ir_objective` takes the options "residual" (minimise the integrated residual, equation (3.4)) or "cost" (minimise the user cost subject to the residual box, equation (3.6)).

- `algorithm.ir_residual_nodes` is a positive integer giving the number of Gauss–Legendre residual-quadrature points used per interval (equation (3.3)). When the flexible-order representation is used it should be at least `ir_local_order`.

- `algorithm.ir_residual_bound` is the non-negative residual-box tolerance δ used when `ir_objective` is "cost" (equation (3.6)).

- `algorithm.ir_regularization` is the non-negative penalty weight ρ on the integrated residual added to the user cost (equation (3.5)); available with the Hermite–Simpson method.

- `algorithm.ir_dair` is a Boolean (default `false`); if `true`, $\mathcal{PSOPT}$ runs the Direct Alternating Integrated Residual (DAIR) algorithm of Section 3.5.

- `algorithm.ir_dair_delta_factor` is the positive factor K in the DAIR box-tolerance schedule $\delta = K(h/T)^p$ (equation (3.7), default 1).

- `algorithm.ir_local_order` is a non-negative integer selecting the local representation: 0 (default) gives the cubic-Hermite representation, and any $d \geq 2$ gives the degree-d Nie–Kerrigan flexible-order representation of Section 3.7 (the number of intervals per phase must then be a multiple of d).

- `algorithm.diff_matrix` takes the options "standard" (default), "reduced-roundoff", or "central-differences".

- `algorithm.print_level` takes the values 1 (default), which causes $\mathcal{PSOPT}$ and the NLP solver to print information on the screen, or 0 to supress all output.

- `algorithm.nlp_tolerance` is a real positive number that is used as a tolerance to check convergence of the NLP solver (default 10^{-6}).

- `algorithm.jac_sparsity_ratio` is a real number in the interval (0,1] which indicates the maximum Jacobian density, which is ratio of nonzero elements to the total number of elements of the NLP constraint Jacobian matrix (default 0.5).

- `algorithm.hess_sparsity_ratio` is a real number in the interval (0,1] which indicates the maximum Hessian density, this is the ratio of nonzero elements to the total number of elements of the NLP Hessian matrix (default 0.2).

- `algorithm.hessian` takes the options "limited-memory" (default), "exact", or "numerical". The "limited-memory" option uses IPOPT's internal limited-memory BFGS approximation and needs no second derivatives. The "exact" option supplies the exact Hessian of the Lagrangian by automatic differentiation, and so requires `algorithm.derivatives` to be "automatic". The "numerical" option supplies a sparse finite-difference approximation of the same Hessian and is available with either automatic or numerical first derivatives. The "exact" and "numerical" options are only used together with the IPOPT NLP solver. See Section 6.6.2.

- `algorithm.hessian_verify` is a Boolean (default `false`). When `true` and `algorithm.hessian` is "numerical", $\mathcal{PSOPT}$ performs a one-off cross-check of the finite-difference Hessian (Section 6.6.2) and prints the result; it is a diagnostic only and has no effect at its default value.

- `algorithm.diagnostic_level` is a non-negative integer (default 0) selecting the solution diagnostics of Chapter 5: 0 disables them (the solve is unaffected), 1 reports the per-interval discretisation error, the per-state spectral smoothness and the costate structure, and 2 additionally reports the Hamiltonian-constancy and control-stationarity optimality residuals.

- `algorithm.on_error` takes the values "fail-fast" (default) or "fail-soft". It selects how $\mathcal{PSOPT}$ behaves if a solution is accessed after a solve that did not succeed; see Section 6.3.

- `algorithm.nsteps_error_integration` is an integer number that gives the number of integration steps to be taken within each interval when calculating the relative ODE error. The default value is 10.

- `algorithm.mesh_refinement` takes the values "manual" (default) or "automatic". With a global pseudospectral method (Legendre, Chebyshev, Radau or Gauss), the "automatic" option invokes the *hp*-adaptive refinement of Section 2.3; with a local method (trapezoidal or Hermite–Simpson) it invokes the Betts refinement of Section 2.4.

- `algorithm.ode_tolerance` is a small real value that is used as one of the stopping criteria for mesh refinement. If the maximum relative ODE error falls below this value, the mesh refinement iterations are terminated. The default value is 10^{-3}.

- `algorithm.mr_max_iterations` is a positive integer with the maximum number of mesh refinement iterations (default 7).

- `algorithm.mr_min_order` is a positive integer (≥ 2) giving the minimum per-segment polynomial degree N_{min} used by the *hp*-adaptive refinement, i.e. the degree at which a freshly subdivided segment is seeded (default 3). Only used with a global pseudospectral method.

- `algorithm.mr_max_order` is a positive integer ($\geq$ `mr_min_order`) giving the maximum per-segment polynomial degree N_{max} used by the *hp*-adaptive refinement; a segment whose degree would exceed N_{max} is subdivided (*h*-refinement) rather than raised in order (default 12). Only used with a global pseudospectral method.

- `algorithm.mr_max_growth_factor` is a positive real number in the range $(0, 1]$ that bounds the per-iteration growth of the mesh (default 0.4). With a global pseudospectral method it limits the growth of the total number of collocation points N_{eff}; with a local method it limits the number of nodes added per iteration.

- `algorithm.mr_kappa` is a positive real number used by the local mesh refinement algorithm (default 0.1).

- `algorithm.mr_M1` is a positive integer used by the local mesh refinement algorithm (default 5).

- `algorithm.switch_order` is a positive integer indicating the local mesh refinement iteration after which the order is switched from 2 (trapezoidal) to 4 (Hermite-Simpson). If the entered value is zero, then the order is not switched and the collocation method specified through the option `algorithm.collocation_method` is used in all mesh refinement iterations. This option only applies if a local collocation method is specified (default 2).

- `algorithm.ipopt_linear_solver` is a string indicating what linear solver should be used by IPOPT. The default value is "mumps", but other solvers are possible (see the IPOPT documentation). Any specified solver should be linked to the executable program as a dynamic or static library.
  - ma27: use the Harwell routine MA27
  - ma57: use the Harwell routine MA57
  - ma77: use the Harwell routine MA77
  - ma86: use the Harwell routine MA86
  - ma97: use the Harwell routine MA97
  - pardiso: use the Pardiso package
  - wsmp: use WSMP package
  - mumps: use MUMPS package (default)

Calling $\mathcal{PSOPT}$

Once everything is ready, then the `psopt` algorithm can be called as follows:

```
psopt(solution, problem, algorithm);
```

Error checking

The `psopt()` function is *total*: it always returns and never propagates an exception to the caller. It sets `solution.error_flag` to a non-zero value if a run-time error is caught — for example, an inconsistency among the user-supplied parameters, many of which $\mathcal{PSOPT}$ checks automatically — and to zero on success. A diagnostic message is printed to the screen and is also available from `solution.error_msg`. The value returned by `psopt()` equals `solution.error_flag`, and the return value carries the `[[nodiscard]]` attribute, so a compiler will warn if it is ignored. The recommended pattern is to check it immediately after the call, as in the following example (here the program exits with code 1 when the solve did not succeed).

```
if (psopt(solution, problem, algorithm))
{
     exit(1);
}
```

The behaviour when a failed solution is subsequently accessed, and how it can be configured for power users, are described in Section 6.3.

Postprocessing the results

The `psopt()` function returns the results of the optimisation within the `solution` data structure. The results may then be post-processed.

For example, to save the time, control and state vectors of the first phase, the user may use the following commands:

```
MatrixXd x = solution.get_states_in_phase(1);
MatrixXd u = solution.get_controls_in_phase(1);
MatrixXd t = solution.get_time_in_phase(1);

Save(x,"x.dat");
Save(u,"u.dat");
Save(t,"t.dat");
```

Plotting with the GNUplot interface

If the software GNUplot is available in the system where $\mathcal{PSOPT}$ is being run, then the user may employ the `plot()`, `multiplot()`, `surf()`, `plot3()` and `polar()` functions, which constitute a simple interface to GNUplot implemented within the $\mathcal{PSOPT}$ library.

The prototype of the `plot()` function is as follows:

```
void plot(MatrixXd& x,
          MatrixXd& y,
          const string& title,
          char* xlabel,
          char* ylabel,
          char* legend=NULL,
```

```
          char* terminal=NULL,
          char* output=NULL);
```

where x is a column or row vector with n elements and y is a matrix with one either row or column dimension equal to n. `xlabel` is a string with the label for the x-axis, `ylabel` is a string with the label for y-axis, `legend` is a string with the legends for each curve that is plotted, separated by commas. `terminal` is a string with the GNUplot terminal to be used (see Table 6.2), and `output` is a string with the filename to be used for the output, if any.

The function is overloaded, such that the user may plot together curves generated from different x, y pairs, up to three pairs. The additional prototypes are as follows:

```
void plot(MatrixXd& x1, MatrixXd& y1,
          MatrixXd& x2, MatrixXd& y2,
          const string& title,
          char* xlabel,
          char* ylabel,
          char* legend=NULL,
          char* terminal=NULL,
          char* output=NULL);
```

```
void plot(MatrixXd& x1, MatrixXd& y1,
          MatrixXd& x2, MatrixXd& y2,
          MatrixXd& x3, MatrixXd& y3,
          const string& title,
          char* xlabel,
          char* ylabel,
          char* legend=NULL,
          char* terminal=NULL,
          char* output=NULL);
```

For example, if the user wishes to display a plot of the control trajectories of a system with two control variables which have been stored in MatrixXd object “u”, and assuming that the corresponding time vector has been stored in MatrixXd object “t”, then an example of the syntax to call the `plot()`function is:

```
plot(t,u,"Control variable","time (s)", "u", "u1 u2");
```

It is also possible to save plots to graphical files supported by GNUplot. For example, to save the above plot to an encapsulated postscript file (instead of displaying it), the command is as follows:

```
plot(t,u,"Control variable", "time (s)", "u", "u1 u2",
          "postscript eps", "filename.eps");
```

The function `spplot()` allows to plot one or more curves together with one or more sets of isolated points (without joining the dots). This can be useful, for example, to compare how an estimated continuous variable compares with experimental data points. The prototype is as follows:

```
void spplot(MatrixXd& x1, MatrixXd& y1,
            MatrixXd& x2, MatrixXd& y2,
            const string& title,
            char* xlabel,
            char* ylabel,
            char* legend=NULL,
            char* terminal=NULL,
            char* output=NULL);
```

where `x1` is a column or row vector with n_1 elements and `y1` is a matrix with one either row or column dimension equal to n_1. The pair `(x1, y1)` is used to generate curve(s). `x2` is a column or row vector with n_2 elements and `y2` is a matrix with one either row or column dimension equal to n_2. The pair `(x2, y2)` is used to plot data points.

For example, if the user wishes to display a curve on the basis of the pair (`t` , `y1`) and on the same plot compare with experimental points stored in the pair (`te` , `ye`), then an example of the syntax to call the `spplot()`function is:

```
plot(t,y,te, ye, "Data fit for y","time (s)", "u", "y ye");
```

The `multiplot()` function allows the user to plot on a single window an array of sub-plots, having one curve per subplot. The function prototype is as follows:

```
void multiplot(MatrixXd& x,
               MatrixXd& y,
               const string& title,
               char* xlabel,
               char* ylabel,
               char* legend,
               int nrows=0,
               int ncols=0,
               char* terminal=NULL,
               char* output=NULL ) ;
```

where x is a column or row vector with n elements and y is a matrix with one either row or column dimension equal to n, `xlabel` should be a string with the common label for the x-axis of all subplots, `ylabel` should be a string with the labels for all y-axes of all subplots, separated by spaces, `nrows` is the number of rows of the array of subplots, `ncols` is the number of columns of the array of subplots. If `nrows` and `ncols` are not provided, then the array of subplots has a single column. Note that the product `nrows*ncols` should be equal to n, which is the number of curves to be plotted.

For example, if the user wishes to display an array of subplots of the state trajectories of a system with four state variables which have been stored in MatrixXd object "y", and assuming that the corresponding time vector has been stored in MatrixXd object "t", then an example of the syntax to call the `multiplot()`function is:

```
multiplot(t,y,"State variables","time (s)",
               "y1 y2 y3 y4", "y1 y2 y3 y4");
```

In the above case, a 4×1 array of sub-plots is produced. If a 2×2 array of sub-plots is required, then the following command can be used:

```
multiplot(t,y,"State variables","time (s)",
          "y1 y2 y3 y4", "y1 y2 y3 y4", 2, 2);
```

The function `surf()` plots the colored parametric surface defined by three matrix arguments. The prototype of the `surf()` function is as follows:

```
void surf(MatrixXd& x,
          MatrixXd& y,
          MatrixXd& z,
          const string& title,
          char* xlabel,
          char* ylabel,
          char* zlabel,
          char* terminal=NULL,
          char* output=NULL,
          char* view=NULL);
```

Here `view` is a character string with two constants `<rot_x>,<rot_y>` (e.g. "50,60"), where `rot_x` is an angle in the interval $[0, 180]$ degrees, and `rot_y` is an angle in the interval $[0, 360]$ degrees. This is used to set the viewing angle of the surface plot.

For example, if the user wishes to display a surface plot of a $N \times M$ matrix Z with respect to the $1 \times N$ vector X and the $1 \times M$ vector Y, stored, respectively, in MatrixXd objects "z", "x" and "y", then an example of the syntax to call the `surf()`function is:

```
surf(x, y,  z, "Title", "x-label", "y-label", "z-label");
```

The function `plot3()` plots a 3D parametric curve defined by three vector arguments. The prototype of the `plot3()` function is as follows:

```
void plot3(MatrixXd& x,
           MatrixXd& y,
           MatrixXd& z,
           const string& title,
           char* xlabel,
           char* ylabel,
           char* zlabel,
           char* terminal=NULL,
           char* output=NULL
           char* view = NULL);
```

Here `view` is a character string with two constants `<rot_x>,<rot_y>` (e.g. “50,60”), where `rot_x` is an angle in the interval $[0, 180]$ degrees, and `rot_y` is an angle in the interval $[0, 360]$ degrees. This is used to set the viewing angle of the 3D plot.

For example, if the user wishes to display a 3D parametric curve of a $1 \times N$ vector Z with respect to the $1 \times N$ vector X and the $1 \times M$ vector Y, stored, respectively, in MatrixXd objects “z”, “x” and “y”, then an example of the syntax to call the `plot3()`function is:

```
plot3(x, y,  z, "Title", "x-label", "y-label", "z-label");
```

The function `polar()` plots a polar curve defined by two vector arguments. The prototype of the `polar()` function is as follows:

```
void polar(MatrixXd& theta,
           MatrixXd& r,
           const string& title,
           char* legend=NULL,
           char* terminal=NULL,
           char* output=NULL);
```

For example, if the user wishes to display a polar plot using a of a $1 \times N$ vector θ (the angle values in radians), and a $1 \times N$ vector r (the corresponding values of the radius), stored, respectively, in MatrixXd objects “theta” and “r”, then an example of the syntax to call the `polar()`function is:

```
polar(theta, r,  "Title");
```

Terminal	Description
postscript eps	Encapsulated postscript
pdf	Adobe portable document format (pdf)
Jpeg	jpg graphical format
Png	png graphical format
latex	LaTeX graphical code

Table 6.2: Some of the available GNUplot output graphical formats

The `polar()` function is overloaded so that the user may plot together up to three different polar curves. The additional prototypes are given below. For two polar curves:

```
void polar(MatrixXd& theta,
           MatrixXd& r,
           MatrixXd& theta2,
           MatrixXd& r2,
           const string& title,
           char* legend=NULL,
           char* terminal=NULL,
           char* output=NULL);
```

For three polar curves.

```
void polar(MatrixXd& theta,
           MatrixXd& r,
           MatrixXd& theta2,
           MatrixXd& r2,
           MatrixXd& theta3,
           MatrixXd& r3,
           const string& title,
           char* legend=NULL,
           char* terminal=NULL,
           char* output=NULL);
```

Some common GNUplot terminals (graphical formats) are given in Table 6.2. See the GNUplot documentation for further details on the keywords needed to specify different graphical formats.

`http://www.gnuplot.info/documentation.html`

6.3 Error handling

PSOPT is designed so that no library call can crash the host program or leave it in an undefined state, whatever the outcome of a solve. This section describes the default behaviour, the reasoning behind it, and how it can be changed by the user.

6.3.1 The `psopt()` contract

The `psopt()` function is *total*: it always returns, and it never allows an exception to propagate to the caller. Internally *PSOPT* uses exceptions to report problems — such as an inconsistent option detected during the validation of the user-supplied data — but these are caught at the boundary of `psopt()` and converted into a status. On return, `solution.error_flag` is zero if the solve succeeded and non-zero otherwise, `solution.error_msg` holds the diagnostic message (which is also printed to the screen), and the integer returned by `psopt()` is equal to `solution.error_flag`. The return value carries the `[[nodiscard]]` attribute, so a compiler will warn if it is discarded; the recommended pattern is to check it immediately, as shown in the "Error checking" part of Section 6.2.7.

6.3.2 Accessing a solution after a failed solve

If a solve does not succeed, the per-phase result arrays are empty. Reaching into them — through the `get_*_in_phase()` accessors, or by passing the empty results to an output utility such as `plot()` — would read past the end of empty data and crash. To prevent this, every solution accessor first checks `solution.error_flag`; if it is set, the accessor reports the situation and *PSOPT* acts according to the `algorithm.on_error` policy below, rather than returning meaningless data.

6.3.3 Default behaviour: `fail-fast`

By default (`algorithm.on_error` is "fail-fast") the first access to a failed solution prints a diagnostic naming the accessor and the underlying error, and then stops the program cleanly with a non-zero exit code. No exception is thrown, so nothing escapes the user's `main()`, and no undefined behaviour occurs. This is the safe default: a clear, immediate stop is preferable to silently continuing a post-processing pipeline with empty or meaningless results, which previously could crash deep inside an output routine.

6.3.4 Rationale

The design follows three principles: exceptions are used only internally and are always caught at the `psopt()` boundary; every failure is reported through `solution.error_flag` and `solution.error_msg`; and no public call — the solve, a solution accessor, or an output utility — may produce undefined behaviour, regardless of the state of the solution. The fail-fast default exists because the most common situation is a user who does not check `solution.error_flag` and proceeds straight to extracting and plotting results; that case should fail visibly and immediately rather than crash.

6.3.5 Configuring the policy: `algorithm.on_error`

A power user who would rather a failed solve not halt the program can set

```
algorithm.on_error = "fail-soft";
```

Under "fail-soft", a solution accessor on a failed solve prints a warning and returns an empty matrix instead of stopping the program; the output utilities (below) then skip that empty data, and execution continues. This is useful, for instance, in a batch that solves many problems and should carry on past an individual failure, or when the caller checks `solution.error_flag` itself and wants full control of the program flow. The two permitted values are "fail-fast" (the default) and "fail-soft".

6.3.6 Output utilities and file output

The output utilities are protected in two complementary ways. First, `plot()` (and its variants `multiplot`, `spplot`, `polar`, `surf` and `plot3`), together with `Save()` and `Save_to_json_file()`, skip with a warning when handed empty data: an empty matrix is never plotted or written. Consequently, under "fail-soft" a failed solve cannot crash a subsequent plotting or saving call. Second, if one of these utilities cannot open its output file — because, for example, the path does not exist or is not writable — it prints a warning and skips that operation; it does not throw and does not stop the program. The computation itself is unaffected; only the individual output is skipped. The same applies to `load_data()`, which returns an empty matrix if its input file cannot be opened or is malformed.

6.4 Specifying a parameter estimation problem

To use the parameter estimation facilities implemented in $\mathcal{PSOPT}$ for problems where the observation function is defined, and where there is a set of observed data at given sampling points (see section 1.12). The user needs to specify, for each phase, the number of observed variables and the number of sampling points:

```
problem.phases(iphase).nobserved        = INTEGER;
problem.phases(iphase).nsamples         = INTEGER;
```

where `nobserved` is the number of simultaneous measurements taking place at each sampling node, and `nsamples` is the total number of sampling nodes. The above parameters should be entered before calling the function `psopt_level2_setup(problem, algorithm)`. After this, additional information may be entered:

```
problem.phases(iphase).observation_nodes      = MatrixXd OBJECT;
problem.phases(iphase).observations           = MatrixXd OBJECT;
problem.phases(iphase).residual_weights       = MatrixXd OBJECT
```

where `observation_nodes` is a $1\times$ `nsamples` matrix, `observations` is a `nobserved` $\times$ `nsamples` matrix, `residual_weights` is a $1\times$ `nobserved` $\times$ `nsamples` matrix. The `residual_weights` matrix is by default full of ones.

If parameter estimation data for a particular phase is saved in a text file with the column format specified below, then an auxiliary function, which is described below, can be used to load the data:

```
 < Time > < Obs. # 1> < Weight # 1> ... <Obs. # n> < Weight # n>
```

where each column is separated by either tabs or spaces, the first column contains the time stamps of the samples, the second column contains the observations of the first variable, the third column contains the weights for each observation of the first variable, and so on. It is then possible to load observation nodes, observations, and residual weights and assign them to the appropriate fields of the `problem` structure by using the function `load_parameter_estimation_data`, whose prototype is given below.

```
void  load_parameter_estimation_data() (
         Prob& problem,
         int iphase,
         char* filename
);
```

Note that the user should not register the `problem.end_point_cost` or the `problem.integrand_cost` functions, but the user needs to register `problem.observation_function`. The prototype of this function is as follows:

```
void  observation_function(
                         adouble* observations,
                         adouble* states,
                         adouble* controls,
                         adouble* parameters,
                         adouble& time_k,
                         int k,
                         adouble* xad,
                         int iphase  );
```

where on output the function should return the array of observed variables corresponding to sampling index k at sampling instant `time_k`. The rest of the interface is the same as for general optimal control problems.

6.5 Automatic scaling

If the user specifies the option `algorithm.scaling` as "automatic", then $\mathcal{PSOPT}$ will calculate scaling factors as follows.

1. Scaling factors for controls, states, static parameters, and time, are computed based on the user supplied bounds for these variables. For finite bounds, the variables are scaled such that their original value multipled by the scaling factor results in a number within the range $[-1, 1]$. If any of the bounds is greater or equal than the constant `inf`, then the variable is scaled to lie within the intervals, $[-\text{inf}, 1]$, $[1, \text{inf}]$ or $[-\text{inf}, \text{inf}]$.The constant `inf` is defined in the include file `psopt.h` as 1×10^{19}.

2. Scaling factors for all constraints (except for the differential defect constraints) are computed as follows. The scaling factor for the i-th constraint is the reciprocal of the norm of the i-th row of the Jacobian of the constraints [4]. If the computed norm is zero, then the scaling factor is set to 1.

3. The scaling factors of each differential defect constraint is by default equal to the scaling factor of the corresponding state by default [4]. However, if `algorithm.defect_scaling` is set to "jacobian-based", then the scaling factors of the differential defect constraints are computed as is done for the other constraints.

4. The scaling factor for the objective function is the reciprocal of the norm of the gradient of the objective function evaluated at the initial guess. If the norm of the objective function at the initial guess is zero, then the scaling factor of the objective function is set to one.

6.6 Differentiation

Users are encouraged to use, whenever possible, the automatic differentiation facilities provided by the CppAD library. The use of automatic derivatives is the default behaviour, but it may be specified explicitly by setting the `derivatives` option to "automatic". $\mathcal{PSOPT}$ uses the CppAD drivers for sparsity determination, Jacobian and gradient evaluation. Automatic derivatives are more accurate than numerical derivatives as they are free of truncation errors. Moreover, $\mathcal{PSOPT}$ works faster when using automatic derivatives.

There may be cases, however, where it is preferrable or necessary to use numerical derivatives. If the user specifies the option `algorithm.derivatives` as "numerical", then the derivatives required by the nonlinear programming algorithm as follows.

6.6.1 First derivatives: numerical gradient and Jacobian

If IPOPT is being used for optimization, then the Jacobian of the constraints is computed by using sparse finite differences, such that groups of variables are perturbed simultaneously [11]. The gradient of the objective function is computed by perturbing one variable at a time. Normally the central difference formula is used, but if the perturbed variable is at (or very close to) one of its bounds, then the forward or backward difference formulas are employed. It is assumed that the Jacobian of the constraint function $G(y)$ is divided into constant and variable terms as follows:

$$\frac{\partial G(y)}{\partial y} = A + \frac{\partial g(y)}{\partial y} \tag{6.7}$$

where matrices A and $\partial g(y)/\partial y$ do not have non-zero elements with the same indices. The constant part A of the constraint Jacobian is estimated first, and only the variable part of the jacobian $\partial g(y)/\partial y$ is estimated by sparse finite differences.

If SNOPT is being used, then its internal algorithms for numerical differentiation implemented are employed.

6.6.2 Second derivatives: the numerical Hessian

When IPOPT is used as the NLP solver it can make use of the Hessian of the Lagrangian

$$H(y, \lambda, \sigma_f) = \sigma_f \, \nabla^2 F(y) + \sum_k \lambda_k \, \nabla^2 G_k(y), \tag{6.8}$$

where F is the objective, the G_k are the constraints, σ_f is the objective factor and the λ_k are the constraint multipliers. How this Hessian is supplied is selected by the `algorithm.hessian` option (Section 6.2.7). The "exact" option supplies (6.8) by automatic differentiation and is therefore available only with automatic first derivatives. The "numerical" option, described here, supplies a sparse finite-difference approximation of the same Hessian, and is available whether the first derivatives are automatic or numerical. It is intended for problems whose user functions are not amenable to second-order automatic differentiation, while still giving the faster and more robust convergence of a genuine second-order method in place of the limited-memory approximation.

The numerical Hessian is built in two stages. First its sparsity pattern is determined. A structural superset is taken from the non-constant part $\partial g(y)/\partial y$ of the constraint Jacobian, and this superset is then refined by probing the Lagrangian at several non-degenerate points and discarding entries that are structurally absent or negligible, using a threshold that combines a relative bar with a finite-difference round-off floor. Probing at several points rather than one guards against an entry that happens to vanish at a particular point. Second, the values on the detected pattern are computed by central finite differences: the diagonal entries from $2n$ single-variable second differences of the Lagrangian, and the off-diagonal entries either by direct mixed differences or by the index-set recovery described next.

Computing every off-diagonal entry by a separate mixed difference costs four constraint evaluations per nonzero. Following the sparse-finite-difference approach of Betts [4], $\mathcal{PSOPT}$ reuses the Curtis–Powell–Reid colouring [11] that it already forms for the constraint Jacobian: two variables in the same colour group never share a constraint row, so perturbing a whole group at once and taking the four-corner difference of the constraint vector isolates a single Hessian entry per constraint. The constraint part of the off-diagonal entries is then recovered in a number of constraint-vector evaluations proportional to the number of *interacting* colour-group pairs, which is typically far smaller than the number of nonzeros. The objective contribution, being a single global function whose group differences would mix entries, is added separately over its own detected pattern.

The colouring and the choice of assembly method are refreshed at every mesh. The cheaper of the index-set recovery and direct mixed differences is selected automatically by comparing their predicted cost. The index-set recovery wins when the same colour-group pairs recur across many nonzeros, as they do for the local and *hp* collocation methods, where the coupling is local and the saving grows as the mesh is refined. For free-final-time global pseudospectral discretisations the time-scaled differentiation matrix makes the defect rows nearly dense, so almost all group pairs interact; the automatic selection then falls back to direct mixed differences. Dedicated symmetric (star or acyclic) colourings of the Hessian itself [32, 10] could reduce the count further, but $\mathcal{PSOPT}$ currently reuses the constraint-Jacobian colouring.

Setting `algorithm.hessian_verify` to `true` (the default is `false`) performs a one-off diagnostic at the first well-conditioned iterate: it cross-checks the two finite-difference assembly methods against each other and, when the user functions are automatic-differentiable, against the automatic-differentiation Hessian, reporting the largest discrepancies. It is intended for validation and adds no cost at its default value.

6.7 Generation of initial guesses

If no guesses are supplied by the user, then $\mathcal{PSOPT}$ computes the initial guess for the unspecified decision variables as follows. Each variable is assumed to be constant and equal to the mean value of its bounds, provided none of the bounds is defined as `inf` or `-inf`. If only one of the bounds is `inf` or `-inf`, then the variable is initialized with the value of the other bound. If the upper and lower bounds are `inf` and `-inf`, respectively, then the variable is initialized at zero.

The variables that are initialized automatically for each phase include: the control variables, the state variables, the static parameters, the initial time, and the final time.

The user may also compute initial guesses for the state variables by propagating the differential equations associated with the problem. Two auxiliary functions are provided for this purpose. See section 6.10 for more details.

6.7.1 LaTeX code generation

LaTeX code is generated automatically producing a table with a summary of information about the mesh refinement process. It may be useful to include this summary in publications that incorporate results generated with $\mathcal{PSOPT}$. A file named `mesh_statistics_$$$.tex` is automatically created, unless `algorithm.print_level` is set to zero, where `$$$` represents the characters of `problem.outfilename` which occur to the left of the file extension point ".".

Abbreviation	**Description**
LGL-ST	Legendre-Gauss-Lobatto (LGL) collocation with standard differentiation matrix given by equation (1.12)
LGL-RR	Legendre-Gauss-Lobatto (LGL) collocation with reduced round-off differentiation matrix given by equation (1.22)
CGL-ST	Chebyshev-Gauss-Lobatto (CGL) collocation with standard differentiation matrix given by equation (1.19)
CGL-RR	Chebyshev-Gauss-Lobatto (CGL) collocation with reduced round-off differentiation matrix given by equation (1.22)
LGR	Legendre–Gauss–Radau (LGR) collocation, see Section 1.10
LG	Legendre–Gauss (LG) collocation, see Section 1.10
TRP	Trapezoidal discretisation, see equation (1.49)
H-S	Hermit-Simpson discretisation, see equation (1.50)
IR	Integrated-residual transcription, see Chapter 3

Table 6.3: Description of the abbreviations used for the discretisation methods which are shown in the automatically generated LaTeX table

To include the generated table in a LaTeX document, simply use the command:
`\input{mesh_statistics_$$$.tex}`

The generated table includes a caption associated with the problem name as set through `problem.name`, as well as a label which is generated by concatenating the string `"mesh_stats_"` with the characters of `problem.outfilename` which occur to the left of the file extension point ".". The caption and label can easily be changed to suit the user requirements by editing or renaming the generated file. A key to the abbreviations used in the file is also printed. The abbreviations for the discretisation methods used are described in Table 6.3.

6.8 Using the integrated-residual transcription

The integrated-residual transcription described in Chapter 3 is selected and controlled through the algorithm options listed in Section 6.2.7. The transcription is enabled with

```
algorithm.transcription_method = "integrated-residual";
```

Its behaviour is then determined by `algorithm.ir_objective` and the options that follow. The four typical use cases are illustrated below.

Least-squares feasibility. To compute the most dynamically feasible trajectory for the mesh:

```
algorithm.transcription_method = "integrated-residual";
algorithm.ir_objective         = "residual";
algorithm.ir_residual_nodes    = 4;     // residual-quadrature points per interval
```

Residual box. To minimise the cost subject to a fixed residual tolerance δ:

```
algorithm.transcription_method = "integrated-residual";
algorithm.ir_objective         = "cost";
algorithm.ir_residual_bound    = 1.0e-3;   // delta
algorithm.ir_residual_nodes    = 4;
```

DAIR. To let $\mathcal{PSOPT}$ set the tolerance automatically and alternate the feasibility and optimality steps over the mesh-refinement sequence:

```
algorithm.transcription_method = "integrated-residual";
algorithm.ir_dair              = true;
algorithm.ir_dair_delta_factor = 1.0;      // K in delta = K (h/T)^p
algorithm.ir_residual_nodes    = 4;
```

Because DAIR composes with mesh refinement, supplying a sequence of node counts through `problem.phases(iphase).nodes` drives the tolerance down from one mesh to the next and the cost to the optimum.

Flexible-order representation. Any of the above may be combined with the Nie–Kerrigan flexible-order representation by selecting a local order $d \geq 2$:

```
algorithm.ir_local_order    = 4;    // degree-d local Lagrange representation
algorithm.ir_residual_nodes = 6;    // should be >= ir_local_order
```

The number of intervals per phase must be a multiple of `ir_local_order`. For smooth problems this typically delivers substantially higher accuracy for the same number of nodes.

For all of these formulations the costates are recovered as described in Section 3.6, and can be read back in the usual way, for example

```
MatrixXd lambda = solution.get_dual_costates_in_phase(1);
```

6.9 Implementing multi-segment problems

Sometimes, it is useful for computational or other reasons to define a multi-segment problem. A multi-segment problem is an optimal control problem with multiple sequential phases that has the same dynamics and path constraints in each phase. The multi-segment facilities implemented in $\mathcal{PSOPT}$ allow the user to specify multi-segment problems in an easier way than defining a multi-phase problem. Special functions are called automatically to patch consecutive segments and ensure state and time continuity across the segment boundaries.

To specify a multi-segment problem the it is necessary to create a data structure of the type `MSdata` (in addition to the `problem`, `algorithm` and `solution` structures) and assign values to its elements as follows:

```
MSdata msdata;

msdata.nsegments            = INTEGER;
msdata.nstates              = INTEGER;
msdata.ncontrols            = INTEGER;
msdata.nparameters          = INTEGER;
msdata.npath                = INTEGER;
msdata.n_initial_events     = INTEGER;
msdata.n_final_events       = INTEGER;
msdata.nodes                = RowVectorXi OBJECT
msdata.continuous_controls  = BOOLEAN
```

If it is desired to enforce control continuity across the segment boundaries, then set `msdata.continuous_controls` to `true`. By default the controls are allowed to be discontinuous across the segment boundaries.

The number of nodes per segment can be speficied as follows (note that it is possible to create grids with segments that have different number of nodes):

- as a single value (e.g. 30), such that the same number of nodes is employed in each segment.
- If `algorithm.mesh_refinement` is set to `"manual"`, a character string can be entered with the node sequence to be tried per segment as part of a manual mesh refinement strategy (e.g. "[30, 50, 60]"). Here the number of values corresponds to the number of mesh refinement iterations to be performed. It is assumed that the same node sequence is tried for each segment. If `algorithm.mesh_refinement` is set to `"automatic"`, then only the first value of the specified sequence is used to start the automatic mesh refinement iterations.
- If `algorithm.mesh_refinement` is set to `"manual"`, a matrix can be entered, such that each row of the matrix corresponds to the node sequence to be tried in the corresponding segment (a character string can be used to enter the matrix, e.g. "[30, 50, 60; 10, 15, 20; 5, 10, 15]", noting the semicolons that separate the rows). Here the number of rows corresponds to the number of segments, and the number of columns corresponds to the number of manual mesh refinement iterations to be performed. If `algorithm.mesh_refinement` is set to `"automatic"`, then only the first value of the specified sequence for each segment is used to start the automatic mesh refinement iterations.

After this, the following function should be called:

```
multi_segment_setup(problem, algorithm, msdata);
```

The upper and lower bounds on the relevant event times of the problem (start time for each segment, and end time for the last segment) can be entered as follows:

```
problem.bounds.lower.times = MatrixXd object;
problem.bounds.upper.times = MatrixXd object;
```

where entered value specifies the time bounds in the following order:

$$[t_0^{(1)}, t_0^{(2)}, \ldots, t_0^{(N_p)}, t_f^{(N_p)}]$$

For example, consider the following code:

```
MatrixXd tlower << 10.0, 20.0, 30.0;
MatrixXd tupper << 15.0, 25.0, 35.0;
problem.bounds.lower.times =  tlower;
problem.bounds.upper.times =  tupper;
```

At this point, the bound information for segment 1 (phase 1) can be entered (bounds for states, controls, event constraints, path constraints, and parameters), as described in section 6.2.7. This should be followed by the bound information for the event constraints of the last phase or segment. After entering the bound information, the auxiliary function `auto_phase_bounds` should be called as follows:

```
auto_phase_bounds(problem);
```

The initial guess for the solution can be specified by a call to the function `auto_phase_guess`. See section 6.10.

See the $\mathcal{PSOPT}$ Application Examples Document for an example on the use of the multi-segment facilities available within $\mathcal{PSOPT}$.

6.10 Other auxiliary functions available to the user

$\mathcal{PSOPT}$ implements a number of auxiliary functions to help the user define optimal control problems. Most (but not all) of these functions are suitable for use with automatic differentiation. All the functions can also be used with numerical differentiation. See the examples section for further details on the use of these functions.

6.10.1 `cross` function

This function takes two arrays of adoubles x and y, each of dimension 3, and returns in array z (also of dimension 3) the result of the vector cross product of x and y. The prototype of the function is as follows:

```
void cross(adouble* x, adouble* y, adouble* z);
```

6.10.2 dot function

This function takes two arrays of adoubles x and y, each of dimension n, and returns the dot product of x and y. The prototype of the function is as follows:

```
adouble dot(adouble* x, adouble* y, int n);
```

6.10.3 get_delayed_state function

This function allows the user to implement DAE's with delayed states. Use only in single-phase problems. Its prototype is as follows:

```
void get_delayed_state(adouble* delayed_state,
                       int state_index,
                       int iphase,
                       adouble& time,
                       double delay,
                       adouble* xad);
```

The function parameters are as follows:

- `delayed_state`: on output, the variable pointed by this pointer contrains the value of the delayed state.
- `state_index`: is the index of the state vector whose delayed value is to be found (starting from 1).
- `iphase`: is the phase index (starting from 1).
- `time`: is the value of the current instant of time within the phase.
- `delay`: is the value of the delay.
- `xad`: is the vector of scaled decision variables.

6.10.4 get_delayed_control function

This function allows the user to implement DAE's with delayed controls. Use only in single-phase problems. Its prototype is as follows:

```
void get_delayed_control(adouble* delayed_control,
                         int control_index,
                         int iphase,
                         adouble& time,
                         double delay,
                         adouble* xad);
```

- `delayed_control`: on output, the variable pointed by this pointer contrains the value of the delayed state.
- `control_index`: is the index of the control vector whose delayed value is to be found (starting from 1).
- `iphase`: is the phase index (starting from 1).
- `time`: is the value of the current instant of time within the phase.
- `delay`: is the value of the delay.
- `xad`: is the vector of scaled decision variables.

6.10.5 `get_interpolated_state` function

This function allows the user to obtain interpolated values of the state at arbitrary values of time within a phase. Its prototype is as follows:

```
void get_interpolated_state(adouble* interp_state,
                            int state_index,
                            int iphase,
                            adouble& time,
                            adouble* xad);
```

- `interp_state`: on output, the variable pointed by this pointer contrains the value of the interpolated state.
- `state_index`: is the index of the state vector whose interpolated value is to be found (starting from 1).
- `iphase`: is the phase index (starting from 1).
- `time`: is the value of the current instant of time within the phase.
- `xad`: is the vector of scaled decision variables.

6.10.6 `get_interpolated_control` function

This function allows the user to obtain interpolated values of the control at arbitrary values of time within a phase. Its prototype is as follows:

```
void get_interpolated_control(adouble* interp_control,
                              int control_index,
                              int iphase,
                              adouble& time,
                              adouble* xad);
```

- interp_control: on output, the variable pointed by this pointer contains the value of the interpolated control.
- control_index: is the index of the control vector whose interpolated value is to be found (starting from 1).
- iphase: is the phase index (starting from 1).
- time: is the value of the current instant of time within the phase.
- xad: is the vector of scaled decision variables.

6.10.7 get_control_derivative function

This function allows the user to obtain the value of the derivative of a specified control variable at arbitrary values of time within a phase. Its prototype is as follows:

```
void get_control_derivative(adouble* control_derivative,
                            int control_index,
                            int iphase,
                            adouble& time,
                            adouble* xad)
```

- control_derivative: on output, the variable pointed by this pointer contains the value of the control derivative.
- control_index: is the index of the control vector whose interpolated value is to be found (starting from 1).
- iphase: is the phase index (starting from 1).
- time: is the value of the current instant of time within the phase.
- xad: is the vector of scaled decision variables.

6.10.8 get_state_derivative function

This function allows the user to obtain the value of the derivative of a specified state variable at arbitrary values of time within a phase. Its prototype is as follows:

```
void get_state_derivative(adouble* control_derivative,
                            int state_index,
                            int iphase,
                            adouble& time,
                            adouble* xad)
```

- state_derivative: on output, the variable pointed by this pointer contains the value of the state derivative.

- `state_index`: is the index of the state vector whose interpolated value is to be found (starting from 1).
- `iphase`: is the phase index (starting from 1).
- `time`: is the value of the current instant of time within the phase.
- `xad`: is the vector of scaled decision variables.

6.10.9 `get_initial_states` function

This function allows the user to obtain the values of the states at the initial time of a phase. Its prototype is as follows:

```
void get_initial_states(adouble* states, adouble* xad, int iphase);
```

- `states`: on output, this array contains the values of the initial states within the specified phase.
- `iphase`: is the phase index (starting from 1).
- `xad`: is the vector of scaled decision variables.

6.10.10 `get_final_states` function

This function allows the user to obtain the values of the states at the final time of a given phase. Its prototype is as follows:

```
void get_initial_states(adouble* states, adouble* xad, int iphase);
```

- `states`: on output, this array contains the values of the final states within the specified phase.
- `iphase`: is the phase index (starting from 1).
- `xad`: is the vector of scaled decision variables.

6.10.11 `get_initial_controls` function

This function allows the user to obtain the values of the controls at the initial time of a phase. Its prototype is as follows:

```
void get_initial_controls(adouble* controls, adouble* xad, int iphase);
```

- `controls`: on output, this array contains the values of the initial controls within the specified phase.
- `iphase`: is the phase index (starting from 1).
- `xad`: is the vector of scaled decision variables.

6.10.12 get_final_controls function

This function allows the user to obtain the values of the controls at the final time of a given phase. Its prototype is as follows:

```
void get_initial_controls(adouble* controls, adouble* xad, int iphase);
```

- `controls`: on output, this array contains the values of the final controls within the specified phase.
- `iphase`: is the phase index (starting from 1).
- `xad`: is the vector of scaled decision variables.

6.10.13 get_initial_time function

This function allows the user to obtain the value of the initial time of a given phase. Its prototype is as follows:

```
adouble get_initial_time(adouble* xad, int iphase);
```

- `iphase`: is the phase index (starting from 1).
- `xad`: is the vector of scaled decision variables.
- The function returns the value of the initial time within the specified phase as an `adouble` type.

6.10.14 get_final_time function

This function allows the user to obtain the value of the final time of a given phase. Its prototype is as follows:

```
adouble get_final_time(adouble* xad, int iphase);
```

- `iphase`: is the phase index (starting from 1).
- `xad`: is the vector of scaled decision variables.
- The function returns the value of the final time within a phase as an `adouble` type.

6.10.15 auto_link function

This function allows the user to automatically link two phases by generating suitable state and time continuity constraints. It is assumed that the number of states in the two phases being linked is the same. The function is intended to be called from within the user supplied `linkages` function. Each call to `auto_link` generates an additional number of linkage constraints given by the number of states being linked plus one.

The function prototype is as follows:

```
void auto_link(adouble* linkages,
               int* index,
               adouble* xad,
               int iphase_a,
               int iphase_b);
```

- `linkages`: on output, this is the updated array of linkage constraint values.
- `index`: on input, the variable pointed to by this pointer contains the next value of the linkages array to be updated. On output, this value is updated to be used in the next call to the `auto_link` function. The first time the function is called, the value should be 0.
- `xad`: is the vector of scaled decision variables.
- `iphase_a`: is the phase index (starting from 1) of one phase to be linked.
- `iphase_b`: is the phase index (starting from 1) of the other phase to be linked.

6.10.16 `auto_link_2` function

This function works in a simular way as the auto_link function, but it also forces the control variables to be continuous at the boundaries. It requires a match in the number of states and in the number of controls between the phases being linked. Each call to `auto_link_2` generates an additional number of linkage constraints given by the number of states plus the number of controls, plus one.

The function prototype is as follows:

```
void auto_link_2(adouble* linkages,
               int* index,
               adouble* xad,
               int iphase_a,
               int iphase_b);
```

- `linkages`: on output, this is the updated array of linkage constraint values.
- `index`: on input, the variable pointed to by this pointer contains the next value of the linkages array to be updated. On output, this value is updated to be used in the next call to the `auto_link_2` function. The first time the function is called, the value should be 0.
- `xad`: is the vector of scaled decision variables.
- `iphase_a`: is the phase index (starting from 1) of one phase to be linked.
- `iphase_b`: is the phase index (starting from 1) of the other phase to be linked.

6.10.17 auto_phase_guess function

This function allows the user to automatically specify the initial guess in a multi-segment problem. The function prototype is as follows:

```
void auto_phase_guess(Prob& problem,
                      MatrixXd& controls,
                      MatrixXd& states,
                      MatrixXd& param,
                      MatrixXd& time);
```

so that the controls, states, time and static parameters are specified as if the problem was single-phase.

6.10.18 linear_interpolation function

This function interpolates a point defined function using classical linear interpolation. The function is not suitable for automatic differentiation, so it should only be used with numerical differentiation. This is useful when the problem involves tabular data. The function prototype is as follows:

```
void linear_interpolation(adouble* y,
                          adouble& x,
                          MatrixXd& pointx,
                          MatrixXd& pointy,
                          int npoints);
```

- `y`: on output, the variable pointed to by this pointer contains the interpolated function value.
- `x`: is the value of the independent variable for which the interpolated function value is sought.
- `pointx`: is the MatrixXd object of independent data points.
- `pointy`: is a MatrixXd object of dependent data points.
- `npoints`: is the number of points in the data objects `pointx` and `pointy`.

6.10.19 smoothed_linear_interpolation function

This function interpolates a point defined function using a smoothed linear interpolation. The method used avoids joining sharp corners between adjacent linear segments. Instead, smooothed pulse functions are used to join the segments. The function is suitable for automatic differentiation. This is useful when the problem involves tabular data. The function prototype is as follows:

```
void smoothed_linear_interpolation(adouble* y,
                                   adouble& x,
                                   MatrixXd& pointx,
                                   MatrixXd& pointy,
                                   int npoints);
```

- y: on output, the variable pointed to by this pointer contains the interpolated function value.
- x: is the value of the independent variable for which the interpolated function value is sought.
- pointx: is the MatrixXd object of independent data points.
- pointy: is a MatrixXd object of dependent data points.
- npoints: is the number of points in the data objects pointx and pointy.

6.10.20 spline_interpolation function

This function interpolates a point defined function using cubic spline interpolation. The function is not suitable for automatic differentiation, so it should only be used with numerical differentiation. This is useful when the problem involves tabular data. The function prototype is as follows:

```
void spline_interpolation(     adouble* y,
                               adouble& x,
                               MatrixXd& pointx,
                               MatrixXd& pointy,
                               int npoints);
```

- y: on output, the variable pointed to by this pointer contains the interpolated function value.
- x: is the value of the independent variable for which the interpolated function value is sought.
- pointx: is the MatrixXd object of independent data points.
- pointy: is a MatrixXd object of dependent data points.
- npoints: is the number of points in the data objects pointx and pointy.

6.10.21 `bilinear_interpolation` function

The function interpolates functions of two variables on a regular grid using the classical bilinear interpolation method. This is useful when the problem involves tabular data. The function prototype is as follows.

```
void bilinear_interpolation(adouble* z,
                            adouble& x,
                            adouble& y,
                            MatrixXd& X,
                            MatrixXd& Y,
                            MatrixXd& Z)
```

- `z`: on output the `adouble` variable pointed to by this pointer contains the interpolated function value.
- The `adouble` pair of variables (`x`, `y`) represents the point at which the interpolated value of the function is returned.
- `X`: is a vector (MatrixXd object) of dimension `nxpoints` $\times$ 1.
- `Y`: is a vector (MatrixXd object) of dimension `nypoints` $\times$ 1.
- `Z`: is a matrix (MatrixXd object) of dimensions `nxpoints` $\times$ `nypoints`. Each element `Z(i,j)` corresponds to the pair `( X(i), Y(j) )`

The function does not deal with sparse data. This function does not allow the use of automatic differentiation, so it should only be used with numerical differentiation.

6.10.22 `smooth_bilinear_interpolation` function

The function interpolates functions of two variables on a regular grid using the a smoothed bilinear interpolation method which allows the use of automatic differentiation. This is useful when the problem involves tabular data. The function prototype is as follows.

```
void smooth_bilinear_interpolation(adouble* z,
                            adouble& x,
                            adouble& y,
                            MatrixXd& X,
                            MatrixXd& Y,
                            MatrixXd& Z)
```

- `z`: on output the `adouble` variable pointed to by this pointer contains the interpolated function value.
- The `adouble` pair of variables (`x`, `y`) represents the point at which the interpolated value of the function is returned.

- X: is a vector (MatrixXd object) of dimension nxpoints $\times$ 1.
- Y: is a vector (MatrixXd object) of dimension nypoints $\times$ 1.
- Z: is a matrix (MatrixXd object) of dimensions nxpoints $\times$ nypoints. Each element Z(i,j) corresponds to the pair (X(i), Y(j))

The function does not deal with sparse data.

6.10.23 spline_2d_interpolation function

The function interpolates functions of two variables on a regular grid using the a cubic spline interpolation method. This is useful when the problem involves tabular data. The function prototype is as follows.

```
void spline_2d_interpolation(adouble* z,
                             adouble& x,
                             adouble& y,
                             MatrixXd& X,
                             MatrixXd& Y,
                             MatrixXd& Z)
```

- z: on output the adouble variable pointed to by this pointer contains the interpolated function value.
- The adouble pair of variables (x, y) represents the point at which the interpolated value of the function is returned.
- X: is a vector (MatrixXd object) of dimension nxpoints $\times$ 1.
- Y: is a vector (MatrixXd object) of dimension nypoints $\times$ 1.
- Z: is a matrix (MatrixXd object) of dimensions nxpoints $\times$ nypoints. Each element Z(i,j) corresponds to the pair (X(i), Y(j))

The function does not deal with sparse data. This function does not allow the use of automatic differentiation, so it should only be used with numerical differentiation.

6.10.24 smooth_heaviside function

This function implements a smooth version of the Heaviside function $H(x)$, defined as $H(x) = 1, x > 0$, $H(x) = 0$ otherwise. The approximation is implemented as follows:

$$H(x) \approx 0.5(1 + \tanh(x/a)) \quad (6.9)$$

where $a > 0$ is a small real number. The function prototype is as follows:

```
adouble smooth_heaviside(adouble x, double a);
```

6.10.25 `smooth_sign` function

This function implements a smooth version of the function $\text{sign}(x)$, defined as $\text{sign}(x) = 1, x > 0$, $\text{sign}(x) = -1, x < 0$, and $\text{sign}(0) = 0$. The approximation is implemented as follows:

$$\text{sign}(x) \approx \tanh(x/a) \tag{6.10}$$

where $a > 0$ is a small real number. The function prototype is as follows:

```
adouble smooth_sign(adouble x, double a);
```

See the examples section for further details on usage of this function.

6.10.26 `smooth_fabs` function

This function implements a smooth version of the absolute value function $|x|$. The approximation is implemented as follows:

$$|x| \approx \sqrt{x^2 + a^2} \tag{6.11}$$

where $a > 0$ is a small real number. The function prototype is as follows:

```
adouble smooth_fabs(adouble x, double a);
```

6.10.27 `integrate` function

The `integrate` function computes the numerical quadrature Q of a scalar function g over the a single phase as a function of states, controls, static parameters and time.

$$Q = \int_{t_0^{(i)}}^{t_f^{(i)}} g[x^{(i)}(t), u^{(i)}(t), p^{(i)}, t]dt \tag{6.12}$$

The integration is done using the Gauss-Lobatto method. This is useful, for example, to incorporate constraints involving integrals over a phase, which can be included as additional event constraints:

$$Q_l \leq \int_{t_0^{(i)}}^{t_f^{(i)}} g[x^{(i)}(t), u^{(i)}(t), p^{(i)}, t]dt \leq Q_u \tag{6.13}$$

Function `integrate` has the following prototype:

```
adouble integrate(
  adouble (*integrand)(adouble*,
                       adouble*,
                       adouble*,
                       adouble&,
                       adouble*,int),
  adouble* xad,
  int iphase )
```

- `integrand`: this is a pointer to the function to be integrated.
- `xad`: is the vector of scaled decision variables.
- `iphase`: is the phase index (starting from 1).
- The function returns the value of the integral as an `adouble` type.

The user needs to implement separately the `integrand` function, which must have the prototype:

```
adouble integrand( adouble* states,
                  adouble* controls,
                  adouble* parameters,
                  adouble& time,
                  adouble* xad,
                  int iphase)
```

- `states`: this is an array of instantaneous states.
- `controls`: is an array of instantaneous controls.
- `parameters`: is an array of static parameter values.
- `time`: is the value of the current instant of time within the phase.
- `xad`: is the vector of scaled decision variables.
- `iphase`: is the phase index (starting from 1).
- the function must return the value of the integrand function given the supplied parameters as an `adouble` type.

6.10.28 `product_ad` functions

There are two versions of this function. The first version has the prototype:

```
void product_ad(const MatrixXd& A,
                const adouble* x,
                int nx,
                adouble* y);
```

This function multiplies a constant matrix stored in MatrixXd object `A` by `adouble` vector stored in array `x`, which has length `nx`, and returns the result in `adouble` array `y`.

The second version has the prototype:

```
void product_ad(adouble* Apr,
                adouble* Bpr,
                int na,
                int ma,
                int nb,
                int mb,
                adouble* ABpr);
```

This function multiplies the (`na` $\times$ `nb`) matrix stored in column-major format column in `adouble` array `Apr`, by the (`nb` $\times$ `mb`) matrix stored in column-major format in `adouble` array `Bpr`. The result is stored in column-major format in `adouble` array `ABpr`.

6.10.29 `sum_ad` function

This function adds a matrix or vector stored columnwise in `adouble` array `a`, to a matrix or vector of the same dimensions stored columnwise in adouble array `b`. Both arrays are assumed to have a total of `n` elements. The result is returned in `adouble` array `c`. The function prototype is as follows.

```
void sum_ad(const adouble* a,
            const adouble*b,
            int n,
            adouble* c);
```

6.10.30 `subtract_ad` function

This function subtracts a matrix or vector stored columnwise in `adouble` array `a`, to a matrix or vector of the same dimensions stored columnwise in adouble array `b`. Both arrays are assumed to have a total of `n` elements. The result is returned in `adouble` array `c`. The function prototype is as follows.

```
void subtract_ad(const adouble* a,
            const adouble*b,
            int n,
            adouble* c);
```

6.10.31 `inverse_ad` function

This function computes the inverse of an `n` $\times$`n` square matrix stored columnwise in `adouble` array `a`. The result is returned in `adouble` array `ainv`, also using columnwise storage. The function prototype is as follows.

```
void inverse_ad(adouble* a,
                int n,
                adouble* ainv)
```

6.10.32 resample_trajectory function

This function resamples a trajectory given new values of the time vector using natural cubic spline interpolation.

```
void resample_trajectory(MatrixXd& Y,
                         MatrixXd& t,
                         MatrixXd& Ydata,
                         MatrixXd& tdata )
```

- `Y` is, on output, a MatrixXd object with dimensions $n_y \times N$ with the interpolated values of the dependent variable.
- `t` is a MatrixXd object of dimensions $1 \times N$ with the values of the independent variable at which the interpolated values are required. The elements of this vector should be monotonically increasing, i.e. `t(j+1) > t(j)`. The following restrictions should be satisfied: `t(1)` $\geq$ `tdata(1)`, and `t(N)` $\leq$ `tdata(M)`.
- `Ydata` is a MatrixXd object of dimensions $n_y \times M$ with the data values of the dependent variable.
- `tdata` is a MatrixXd object of dimensions $1 \times M$ with the data values of the independent variable. The elements of this vector should be monotonically increasing, i.e. `t(j+1) > t(j)`.

6.10.33 linspace function

This function generate a sequence of real values and returns it in a MatrixXd object of dimensions `1` $\times$ `N`.

```
MatrixXd linspace(double X1, double X2, long N);
```

- `X1` is the initial value of the sequence
- `X1` is the final value of the sequence
- `N` is the number of elements of the sequence.

6.10.34 zeros function

This function returns a MatrixXd object of dimensions `nrows` $\times$ `ncols`, having a zero value in all its elements.

```
MatrixXd zeros(long nrows, long ncols)
```

- `nrows` is the number of rows of the matrix
- `ncols` is the number of columns of the matrix

6.10.35 `ones` function

This function returns a MatrixXd object of dimensions `nrows` $\times$ `ncols`, having a value of 1 in all its elements.

```
MatrixXd ones(long nrows, long ncols)
```

- `nrows` is the number of rows of the matrix
- `ncols` is the number of columns of the matrix

6.10.36 `eye` function

This function returns a MatrixXd object of dimensions `n` $\times$ `n`, having a value of 1 in all its diagonal elements and zeros elsewhere, this is, an identity matrix.

```
MatrixXd eye(long n)
```

- `n` is the number of rows and columns of the matrix

6.10.37 `GaussianRandom` function

This function returns a MatrixXd object of dimensions `nrows` $\times$ `ncols` and each of its elements has a random value obeying a Gaussian distribution with a mean of 0 and a standard deviation of 1.

```
MatrixXd RandomGaussian(long nrows, long ncols)
```

- `nrows` is the number of rows of the matrix
- `ncols` is the number of columns of the matrix

6.10.38 Elementwise mathematical functions on `MatrixXd` objects

A number of functions have been implemented returning each a MatrixXd object of dimensions `nrows` $\times$ `ncols` with an element-wise evaluation of a number of mathematical functions. The generic function prototype is as follows:

```
MatrixXd <Function_Name>(MatrixXd& m)
```

- `m` is the input matrix, which is the argument of the mathematical function being used. Note that the output matrix has the same dimensions of the input matrix.

The following functions are implemented:

- `sin`: trigonometic sine function
- `cos`: trigonnometric cosine function

- `tan`: trigonometric tangent function
- `asin`: arc sine function function
- `acos`: arc cosine function
- `atan`: arc tangent function
- `sinh`: hyperbolic sine function
- `cosh`: hyperbolic cosine function
- `tanh`: hyperbolic tangent function
- `exp`: exponential function
- `log`: natural logarithm function
- `log10`: base-10 logarithm function
- `sqrt`: square root function

6.10.39 `Save_to_json_file` function

This function exports the complete solution of a problem to a single file in JSON format, which is convenient for post-processing the results with external tools such as Python, MATLAB, or JavaScript. It is normally called after the call to `psopt()`.

```
void Save_to_json_file(const string& filename,
Prob& problem,
Sol& solution,
Alg& algorithm)
```

- `filename` is a string with the name of the JSON file to be created, for example `"solution.json"`. Any existing file with the same name is overwritten.
- `problem` is the `Prob` object describing the problem.
- `solution` is the `Sol` object, returned by $\mathcal{PSOPT}$, that contains the solution.
- `algorithm` is the `Alg` object containing the algorithm options.

The file contains a `"summary"` object and a `"phases"` array. The `"summary"` object holds the problem name, the NLP solver used, the $\mathcal{PSOPT}$ release and build date, the run date and time, the CPU time, the NLP return code, a Boolean success flag, the solver message, the optimal value of the objective function, and the number of phases. The `"phases"` array contains one object per phase, each reporting the number of nodes, states, controls and parameters, the endpoint and integrated costs, the initial and final times, the maximum relative local error, and the trajectories themselves: the time vector together with the states, controls, costates, relative_errors, and parameters arrays.

An example of the syntax to call the function is:

```
Save_to_json_file("solution.json", problem, solution, algorithm);
```

The resulting file can be read by any JSON-aware environment. Note that the states, controls, costates, and relative_errors arrays are stored row-wise, so they are read back as matrices of dimension (number of variables) × (number of nodes), while time and parameters are read as vectors. The following snippets load "solution.json" and extract the optimal cost together with the time vector and state trajectories of the first phase.

In Python (using the json and NumPy libraries):

```
import json, numpy as np
with open("solution.json") as f:
data = json.load(f)

print(data["summary"]["optimal_cost"])
phase    = data["phases"][0]            # first phase
t        = np.array(phase["time"])      # shape (n_nodes,)
states   = np.array(phase["states"])    # shape (n_states, n_nodes)
controls = np.array(phase["controls"])  # shape (n_controls, n_nodes)
```

In MATLAB:

```
data = jsondecode(fileread("solution.json"));

disp(data.summary.optimal_cost)
phase    = data.phases(1);              % first phase
t        = phase.time;                  % n_nodes x 1
states   = phase.states;                % n_states x n_nodes
controls = phase.controls;              % n_controls x n_nodes
```

In R (using the jsonlite package):

```
library(jsonlite)
data <- fromJSON("solution.json", simplifyDataFrame = FALSE)

print(data$summary$optimal_cost)
phase    <- data$phases[[1]]            # first phase
t        <- phase$time                  # length n_nodes
states   <- phase$states                # n_states x n_nodes matrix
controls <- phase$controls              # n_controls x n_nodes matrix
```

6.11 Pre-defined constants

The following constants are defined within the header file `psopt.h`:

- `pi`: defined as 3.141592653589793;
- `inf`: defined as 1×10^{19}.

6.12 Standard output

PSOPT will by default produce output information on the screen as it runs. *PSOPT* will produce a short file with a summary of information named with the string provided in `algorithm.outfilelname`. This file contains the problem name, the total CPU time spent, the NLP solver used, the optimal value of the objective function, the values of the endpoint cost function and cost integrals, the initial and final time, the maximum discretisation error, and the output string from the NLP solver.

Additionally, every time a *PSOPT* excutable is run, it will produce a file named `psopt_solution_$$$.txt` (`$$$` represents the characters of `problem.outfilename` which occur to the left of the file extension point "."). This file contains the problem name, time stamps, a summary of the algorithm options used, and results obtained, the final grid points, the final control variables, the final state variables, the final static parameter values. The file also contains a summary of all constraints functions associated with the NLP problem, including their final scaled value, bounds, and scaling factor used; a summary of the final NLP decision variables, including their final unscaled values, bounds and scaling factors used; and a summary of the mesh refinement process. An indication is given at the end of a constraint line, or decision variable line, if a scaled constraint function or scaled decision variable is within `algorithm.nlp_tolerance` of one of its bounds, or if a scaled constraint function or scaled decision variable has violated one of its bounds by more than `algorithm.nlp_tolerance`. For parameter estimation problems this file also contains the covariance matrix of the parameter vector, and the 95% confidence interval for each estimated parameter.

LaTeX code to produce a table with a summary of the mesh refinement process is also automatically generated as described in section 6.7.1.

If `algorithm.print_level` is set to zero, then no output is produced.

6.13 Implementing your own problem

A template C++ file named `user.cxx` is provided in the directory:

`psopt/examples/user`

This file can be modified by the user to implement their own problem. and an executable can then be built easily.

6.13.1 Building the user code

After modifying the `user.cxx` code, open a command prompt and cd the user example directory under the build tree and execute the `make` command:

```
$ cd psopt/build/examples/user
$ make
```

If no compilation errors occur, an executable named `user` should be created in the directory `psopt /PSOPT/examples/user`.

7. Integer (discrete-valued) controls

The problems considered so far assume that every control is continuous. $\mathcal{PSOPT}$ also supports optimal control problems in which one control per phase is restricted to a finite set of admissible values, for example an on/off actuator, a gear selection or a mode switch. Such mixed-integer optimal control problems are handled by an ergonomic *declaration* interface: the dynamics are written once, with the discrete control accessed as an ordinary control, and $\mathcal{PSOPT}$ performs the underlying reformulation internally. This chapter describes the method, the interface and its conventions, the recovery of the integer control from the solution, and the current limitations. It builds directly on the problem-definition machinery of Chapter 6.

7.1 Method

$\mathcal{PSOPT}$ solves integer-control problems in two stages, following the partial outer convexification and sum-up rounding approach of Sager and co-workers [34, 36].

7.1.1 Relaxation by outer convexification

Suppose that a single control $w(t)$ in a phase is restricted to the set of M admissible values $\{v_0, v_1, \ldots, v_{M-1}\}$, while the remaining controls $u(t)$ are continuous. Outer convexification replaces w by M continuous *mode weights* $\omega_0(t), \ldots, \omega_{M-1}(t)$, each in $[0, 1]$, subject to the special ordered set condition

$$\sum_{i=0}^{M-1} \omega_i(t) = 1, \tag{7.1}$$

an SOS1 equality. The dynamics are evaluated at each admissible value and combined convexly,

$$\dot{x} = \sum_{i=0}^{M-1} \omega_i(t)\, f[x(t), u(t), v_i, p, t], \tag{7.2}$$

and likewise for any running cost that depends on the discrete control. The result is a smooth nonlinear program in the weights and the continuous variables, whose optimum is a *lower bound* on the integer optimum.

7.1.2 Sum-up rounding

The relaxed weights are generally fractional. They are converted to a control that is active in exactly one mode on each mesh interval by *sum-up rounding*: sweeping across the mesh, the algorithm accumulates, for each mode, the integral of the relaxed weight minus the integral of the rounding already committed, and activates on the current interval the mode with the largest accumulated value. If ω_i denotes the relaxed weight of mode i and $p_i \in \{0, 1\}$ the rounded indicator, this keeps the control integrals close,

$$\left| \int_{t_0}^{t} \big(\omega_i(\tau) - p_i(\tau)\big)\, d\tau \right| \;\le\; \bar{h}, \qquad i = 0, \ldots, M-1, \tag{7.3}$$

where $\bar{h}$ is the largest mesh interval width. Because the state and objective errors induced by the rounding are governed by this integral gap, they are $O(\bar{h})$ and vanish as the mesh is refined [36]. The objective obtained by simulating the rounded control is therefore an *upper bound* on the integer optimum that approaches it as $\bar{h} \to 0$.

For a bang-bang problem the relaxed and integer optima coincide. For a singular problem no optimal integer control exists, but the relaxed infimum can be approached arbitrarily closely by a control that switches sufficiently often; refining the mesh then reduces the gap while increasing the number of switches.

7.2 Declaring an integer control

The interface is provided by a single header, which should be included in addition to `psopt.h`:

```
#include "integer_controls.h"
```

The phase dynamics are written *once*, with the discrete control accessed as an ordinary entry of the `controls` array, exactly as if it were continuous:

```
void dae(adouble* derivatives, adouble* path, adouble* states,
         adouble* controls, adouble* parameters, adouble& time,
         adouble* xad, int iphase, Workspace* workspace)
{
    adouble w = controls[0];   // the integer control: a single value
    // ... use w like any other control ...
}
```

The control is then declared as integer-valued, after `psopt_level2_setup`, by

```
RowVectorXd values(2); values << -0.05236, 0.05236;
declare_integer_control(problem, /*iphase=*/1, /*control_index=*/0, values);
```

The function

```
void declare_integer_control(Prob& problem, int iphase,
                             int control_index, const RowVectorXd& values);
```

records, on the given phase, that the control at `control_index` is restricted to the admissible set `values`. The declaration is consumed by `psopt` at solve time.

7.2.1 Conventions

- **The integer control is counted in `ncontrols`.** Set `ncontrols` to the total number of controls, including the integer one, exactly as the `dae` indexes them. The bounds and guess arrays are auto-sized to `ncontrols` as usual (Chapter 6); no manual sizing is required.

- **Bounds for an integer control are ignored.** Its admissible values come from `declare_integer_control`, not from the control bounds; $\mathcal{PSOPT}$ owns the weight bounds $[0, 1]$. The bounds entry for the integer control may be left at any value (writing the value range there reads naturally). A one-line notice is printed at solve time to make this explicit.

- **One integer control per phase.** A phase may declare at most one integer control. Different phases may each declare one.

- **Do not add the SOS1 constraint yourself.** Set `npath` to the number of your own path constraints only; $\mathcal{PSOPT}$ appends the SOS1 equality (7.1) internally.

7.2.2 Initial guess

Provide the integer control's guess row in *value terms* (each entry close to one of the admissible values). $\mathcal{PSOPT}$ maps each entry to the nearest admissible value and expands the row into one-hot mode weights; continuous control guesses and the state guess are passed through unchanged. If no control guess is supplied, the weights take their default.

7.3 What $\mathcal{PSOPT}$ does internally

The reformulation is performed automatically at the start of `psopt` and undone when it returns. For each phase that declares an integer control:

- the integer control slot is removed and M weight controls are appended, so the internal control count becomes $(\texttt{ncontrols} - 1) + M$;

- one SOS1 equality path constraint (7.1) is appended, so `npath` becomes $\texttt{npath} + 1$;

- the control and path bounds, scale and guess are rewritten to match; and

- the user `dae` and running cost are evaluated at each admissible value and combined as in (7.2).

When psopt returns, the problem object is restored to its original layout, but *the solution controls are in the weights layout*: the trailing M rows of `get_controls_in_phase(iphase)` are the mode weights $\omega_0, \ldots, \omega_{M-1}$. The integer control is recovered with the helper of the next section. This whole process is inactive when no integer control is declared, so problems without integer controls are unaffected.

7.4 Reconstructing the integer control

After a solve, the rounded integer control is obtained in a single call:

```
IntegerControlReconstruction rec = reconstruct_integer_control(solution,
    ↪ problem, iphase);
```

which returns

```
struct IntegerControlReconstruction {
    RowVectorXd control;         // 1 x P : rounded integer control per
    ↪ interval
    RowVectorXi mode_index;      // 1 x P : active admissible-value index per
    ↪  interval
    RowVectorXd interval_widths; // 1 x P : mesh interval widths
    double      integral_gap;    // accumulated sum-up-rounding gap (<= max
    ↪ interval width)
    int         n_switches;      // number of mode switches
};
```

The helper extracts the trailing M weight rows from the solution, forms the interval widths from the phase time, and applies sum-up rounding using the admissible values recorded at declaration. The relaxed objective read from the solved problem is a lower bound; the objective obtained by simulating `rec.control` is an upper bound that tends to the relaxed value as the mesh is refined, in accordance with (7.3).

7.5 Worked example: the F-8 aircraft

The F-8 aircraft time-optimal control problem [35] steers the aircraft to the origin in minimum time using a tail deflection restricted to $w \in \{-0.05236, +0.05236\}$ rad. The dynamics are written once with the deflection as a single control:

```
void dae(adouble* d, adouble* path, adouble* states, adouble* controls,
         adouble* parameters, adouble& time, adouble* xad, int i, Workspace*
    ↪ w)
{
    adouble x0 = states[0], x1 = states[1], x2 = states[2];
    adouble u  = controls[0];               // the integer tail deflection
    f8_rhs<adouble>(x0, x1, x2, u, d);      // cubic-in-control dynamics
}
```

```
int main() {
    Alg algorithm; Sol solution; Prob problem;
    problem.nphases = 1; problem.nlinkages = 0;
    psopt_level1_setup(problem);

    problem.phases(1).nstates   = 3;
    problem.phases(1).ncontrols = 1;       // one control: the deflection
    problem.phases(1).nevents   = 6;
    problem.phases(1).npath     = 0;       // the SOS1 constraint is added
    ↪ internally
    problem.phases(1).nodes     << 60;
    psopt_level2_setup(problem, algorithm);

    RowVectorXd values(2); values << -0.05236, 0.05236;
    declare_integer_control(problem, 1, 0, values);

    // ... state and event bounds, times, guess, function pointers, algorithm
    ↪  ...

    psopt(solution, problem, algorithm);

    IntegerControlReconstruction rec = reconstruct_integer_control(solution,
    ↪ problem, 1);
    // rec.control is the rounded bang-bang deflection; rec.n_switches its
    ↪ switches.
}
```

The relaxed solve reproduces the best-known minimum time $T = 3.78086$ with the expected three-switch bang-bang structure. Complete, runnable examples are provided under `examples/f8` (minimum-time, bang-bang), `examples/lotka` (the singular Lotka–Volterra fishing problem [35], with a mesh sweep showing the relaxed objective converging to 1.34408 and the integer gap closing at $O(\bar{h})$), and `examples/mixed_ic` (a continuous and an integer control in the same phase).

7.6 Limitations

The current implementation supports one integer control per phase. Several independent integer controls in a single phase would require a product-of-modes convexification and are not yet supported. The running cost may depend on the integer control, as it is convex-combined in the same way as the dynamics; the endpoint cost does not receive the controls and so must not depend on the integer control directly. User path constraints are assumed independent of the integer control, being evaluated at a representative mode. Finally, for a minimum-time problem the rounded control does not, in general, reach the target exactly in the relaxed final time; the terminal miss is the integer approximation error and shrinks under mesh refinement, while an exactly feasible integer final time would require switching-time optimisation.

8. Integer (discrete-valued) static parameters

The previous chapter restricted a control to a finite set of values. $\mathcal{PSOPT}$ also supports optimal control problems in which one or more *static parameters* of a phase are restricted to a finite set of admissible values — a discrete design choice such as a component count, a gear ratio selected from a catalogue, or a material index. These integer static parameters are handled by a dedicated driver, `psopt_solve_integer`, which solves the problem *exactly* by enumeration. The dynamics, cost and events are written once, with the discrete parameter accessed as an ordinary static parameter, and a single declaration marks it integer-valued. This chapter describes the method, the interface and its conventions, the driver, the recovery of the selected value, and the current limitations. It builds on the problem-definition machinery of Chapter 6 and is a companion to the integer-control interface of Chapter 7.

8.1 Method

Unlike a control, a static parameter takes a single value over the whole phase; it has no time profile. $\mathcal{PSOPT}$ therefore solves an integer-static-parameter problem by *exact enumeration* rather than by the convex relaxation used for integer controls.

Suppose a problem declares static parameters $p_{j_1}, \ldots, p_{j_D}$ as integer, the a-th restricted to an admissible set of M_a values $\{v_0^{(a)}, \ldots, v_{M_a-1}^{(a)}\}$. The driver forms the Cartesian product of these sets and, for each combination, fixes every declared parameter to its chosen admissible value — by setting its lower and upper bounds equal — and solves the resulting fixed-parameter optimal control problem with the ordinary `psopt` solver. The combination that yields the smallest objective among the successful solves is returned. Because every admissible combination is tried, the result is the global integer optimum over the declared sets; no rounding or relaxation gap is incurred.

The number of solves is the product of the set sizes, taken across all phases,

$$N_{\mathrm{comb}} = \prod_{a=1}^{D} M_a. \tag{8.1}$$

This grows multiplicatively, so exact enumeration is intended for small admissible spaces. To prevent an accidentally huge sweep, the driver refuses to start if N_{comb} exceeds `algorithm.max_integer_combinations` (default 4096).

8.1.1 Relation to integer controls

The outer-convexification and sum-up-rounding method of Chapter 7 realises a fractional *blend* of admissible values as a control that switches over time, with an integrality gap that vanishes as the mesh is refined [36]. A static parameter has no time dimension over which to switch, so a convex blend of admissible values is not realisable and no mesh refinement closes the gap. Exact enumeration is therefore the appropriate method for integer static parameters, in the small-space regime for which they are intended.

8.2 Declaring an integer parameter

The interface is provided by a single header, which should be included in addition to `psopt.h`:

```
#include "integer_parameters.h"
```

The discrete parameter is used *once*, as an ordinary entry of the `parameters` array, in the dynamics, cost or events, exactly as if it were continuous:

```
void dae(adouble* derivatives, adouble* path, adouble* states,
         adouble* controls, adouble* parameters, adouble& time,
         adouble* xad, int iphase, Workspace* workspace)
{
    adouble p = parameters[0];   // the integer static parameter
    // ... use p like any other static parameter ...
}
```

It is then declared integer-valued, after `psopt_level2_setup`, by

```
RowVectorXd values(4); values << 0.0, 1.0, 2.0, 3.0;
declare_integer_parameter(problem, /*iphase=*/1, /*parameter_index=*/0,
    ↪ values);
```

The function

```
void declare_integer_parameter(Prob& problem, int iphase,
                               int parameter_index, const RowVectorXd& values
    ↪ );
```

records, on the given phase, that the static parameter at `parameter_index` is restricted to the admissible set `values`. The record is dormant under an ordinary `psopt` call — which ignores it — and is consumed by `psopt_solve_integer` (Section 8.3).

8.2.1 Conventions

- **The parameter is counted in `nparameters`.** Declare it as an ordinary static parameter: set `nparameters` to include it, and access it at the same index in `dae`, `events` and the cost. Its bounds and guess arrays are auto-sized as usual (Chapter 6); no manual sizing is required.

- **Set the bounds to the admissible hull.** Give the parameter a continuous bound range spanning its admissible set, for example from its smallest to its largest value. `psopt_solve_integer` overwrites these bounds to pin the parameter to each admissible value in turn, and restores them when it returns; the range you set is used for the initial guess and automatic scaling.

- **Several parameters may be declared.** A phase may declare more than one integer parameter, and different phases may each declare their own. The driver enumerates the full Cartesian product across all phases, bounded by `algorithm.max_integer_combinations` as in (8.1).

- **Continuous parameters are unaffected.** Any static parameter that is not declared integer remains a free continuous variable and is optimised normally within each fixed-combination solve.

8.3 Solving with the dedicated driver

An integer-static-parameter problem is solved by the dedicated driver in place of `psopt`:

```
int rc = psopt_solve_integer(solution, problem, algorithm);
```

whose signature is

```
[[nodiscard]] int psopt_solve_integer(Sol& solution, Prob& problem, Alg&
    ↪ algorithm);
```

The driver enumerates the Cartesian product of the declared admissible sets; for each combination it fixes the declared parameters, solves the fixed-parameter problem with the ordinary `psopt`, and keeps the combination of smallest objective among the successful solves. On return, `solution` holds that best solve — states, controls, parameters, cost and diagnostics — exactly as an ordinary `psopt` call would leave it, and the return value `rc` is that solve's `psopt` return code. Combinations whose solve fails are skipped; if none succeeds, the last attempt is returned so that its return code remains visible to the caller.

If no integer parameter is declared, `psopt_solve_integer` performs a single ordinary solve and is equivalent to calling `psopt` directly. This makes it safe to route all solves of a given problem through the driver.

Before enumerating, the driver checks the combination count (8.1) against `algorithm.max_integer_combinations` and, if it is exceeded, aborts through the usual $\mathcal{PSOPT}$ error handler (Section 6.3) rather than launching an unintended long sweep. The count may be inspected beforehand with

```
long n = integer_parameter_combination_count(problem);
```

8.4 Reconstructing the selected value

After a solve, the selected admissible value of a declared parameter is read back in a single call:

```
IntegerParameterReconstruction sel =
    reconstruct_integer_parameter(solution, problem, /*iphase=*/1, /*
    ↪ parameter_index=*/0);
```

which returns

```
struct IntegerParameterReconstruction {
    double value;   // selected admissible value (NaN if not declared)
    int    index;   // index into the admissible set (-1 if not declared)
};
```

Because the winning combination was solved with the parameter fixed, the solved solution already carries the integer value exactly; the helper reads it from `get_parameters_in_phase` and snaps it to the nearest admissible value as a safety net. The optimal objective is available as usual in `solution.cost`.

8.5 Worked example: a discrete acceleration

The example under `examples/int_static_linear` is a minimal, control-free double integrator whose constant acceleration is an integer static parameter. With states s (position) and v (velocity), no controls, and one parameter p,

$$\dot{s} = v, \qquad \dot{v} = p, \qquad s(0) = v(0) = 0, \qquad t \in [0, 1], \tag{8.2}$$

so that $v(1) = p$. The parameter is restricted to $p \in \{0, 1, 2, 3\}$ and the objective is $J = (v(1) - 2.3)^2$, chosen so that the discrete optimum $p = 2$ (giving $J = 0.09$) differs from the continuous optimum $p = 2.3$. The dynamics read the parameter directly:

```
void dae(adouble* d, adouble* path, adouble* states, adouble* controls,
         adouble* parameters, adouble& time, adouble* xad, int i, Workspace*
    ↪ w)
{
    d[0] = states[1];        // s_dot = v
    d[1] = parameters[0];    // v_dot = p
}

int main() {
    // ... psopt_level1_setup / level2_setup, states, events, times, guess
    ↪ ...

    RowVectorXd values(4); values << 0.0, 1.0, 2.0, 3.0;
    declare_integer_parameter(problem, 1, 0, values);
```

```
    int rc = psopt_solve_integer(solution, problem, algorithm);

    IntegerParameterReconstruction sel =
        reconstruct_integer_parameter(solution, problem, 1, 0);
    // sel.value == 2.0 and solution.cost == 0.09
}
```

Enumeration solves the four fixed-parameter problems and selects $p = 2$ with $J = 0.09$, matching the closed-form optimum. Because the endpoint cost and the events function both receive the parameter array, the discrete parameter may enter the cost and the boundary conditions directly — in contrast to the integer-control interface, where the endpoint cost does not receive the control. A two-parameter co-design, each parameter drawn from a four-value set, is written in the same way and enumerates the full $4\times 4 = 16$ combinations.

8.6 Limitations

The method is exact but combinatorial: the number of solves is the product (8.1) of the admissible-set sizes, so it is intended for small discrete spaces — a handful of parameters over modest sets. Wide ranges, or many jointly-declared parameters, quickly exceed a practical budget and are guarded by `max_integer_combinations`.

Each combination is solved independently, from the supplied initial guess; this version does not warm-start one combination from another and does no relaxation-based pruning. A failed fixed-parameter solve is skipped, so a poor guess that defeats every combination yields no usable solution; a guess that is valid across the admissible hull is advisable.

Finally, the worked example (8.2) is deliberately degenerate — one non-trivial state and no controls — to isolate the mechanism. A realistic application will have the discrete parameter enter non-trivial dynamics, cost or constraints alongside continuous states and controls.

9. The Python interface

$\mathcal{PSOPT}$ provides a Python interface in which an entire optimal-control or estimation problem — its dynamics, cost, path constraints, boundary events, phase linkages and observation functions — is written symbolically in Python using CasADi [1], and solved by the native $\mathcal{PSOPT}$ /IPOPT core. The C++ engine is not modified. The Python layer traces the user's symbolic maths, generates and just-in-time compiles the corresponding derivative code, and drives the existing solver through a compiled extension module. Every problem is therefore solved by exactly the same engine as a hand-written C++ example, and the numerical results match to solver tolerance.

9.1 Overview and when to use it

The Python interface is convenient for rapid prototyping, for users who prefer Python to C++, and for problems whose data are most naturally assembled with NumPy — for example an estimation problem driven by measured samples, or dynamics built from tabulated coefficients. Because each problem is compiled to native code and solved by the unchanged core, there is no interpreted inner loop and no loss of accuracy relative to the C++ route.

The cost of this convenience is a dependency, at run time, on CasADi and a C++ toolchain: the first solve of a given problem compiles its maths to a small shared library (subsequently cached, see Section 9.2). Where $\mathcal{PSOPT}$ must be embedded in a larger C++ application, where no compiler is available at run time, or where a feature not yet surfaced in the Python API is required, the C++ interface of Chapter 6 remains the primary route. The two interfaces describe the same problem structure, so a problem prototyped in Python maps directly onto its C++ counterpart.

9.2 How it works

The interface is built from two layers.

- A *driver* (`src/_psopt_driver.cpp`), a pybind11 extension module that constructs the $\mathcal{PSOPT}$ `Prob` and `Alg` objects in the required order, registers the user functions, calls the solver, and returns the trajectory as NumPy arrays.

- A *code generator* (`psopt/codegen.py` and `psopt/_emitter.py`). Each problem's CasADi maths is traced to an `adouble`-templated C++ source file, just-in-time compiled to a small mathematics shared library, and cached by a hash of that source. The driver `dlopen`s the library and registers its `extern "C"` functions as the $\mathcal{PSOPT}$ user-function pointers.

The mathematics library performs `adouble` arithmetic only and links CppAD, so the automatic differentiation used by the solver is identical to that of a hand-written example. $\mathcal{PSOPT}$'s own symbols (such as `auto_link`) are resolved at load time: the extension module whole-archives `libPSOPT` and is imported with `RTLD_GLOBAL`, so those symbols are visible to each dynamically loaded mathematics library. This arrangement requires a position-independent `libPSOPT`, the extension to be linked with `--whole-archive` and `--export-dynamic` (`-rdynamic`), and default symbol visibility on the extension. These requirements are handled automatically by the supplied build (Section 9.3); they are noted here because a symbol-resolution error at import or first solve almost always indicates that one of them has been lost.

The just-in-time cache defaults to `$XDG_CACHE_HOME/psopt` and can be redirected with the environment variable `PSOPT_CACHE_DIR`. Because compiled problems are keyed by the hash of their generated source, an unchanged problem is compiled only once across runs.

9.3 Building and installing

The recommended build is in-tree, from the top-level $\mathcal{PSOPT}$ build, which makes `libPSOPT` position-independent and builds the binding against it:

```
cmake -S . -B build -DPSOPT_PYTHON=ON
cmake --build build
```

The binding may also be built standalone against a prebuilt position-independent `libPSOPT.a`:

```
cmake -S python -B build/python \
      -DPSOPT_ROOT=<psopt source root> \
      -DPSOPT_LIB=<.../libPSOPT.a> \
      -Dpybind11_DIR=$(python -m pybind11 --cmakedir)
cmake --build build/python
```

In either case CMake builds the extension `psopt/_psopt*.so` and *generates* `psopt/_toolchain.py` — the compiler, flags, include directories and CppAD library used for the just-in-time compilation — from the very configuration used to build $\mathcal{PSOPT}$. There is no hand-maintained, machine-specific toolchain file to keep in step with the installation.

Installation with `pip` drives the same CMake build through scikit-build-core:

```
pip install ./python
```

For a standalone install the $\mathcal{PSOPT}$ location is passed through, for example `--config-settings=cmake.define.PSOPT_ROOT=...` and the corresponding `PSOPT_LIB`.

9.4 Defining and solving a problem

A problem is a `psopt.Problem`. Phases are added to it with `add_phase`, which returns a phase object on which the user maths, bounds and initial guess are set. An `Algorithm` object carries the solver options, and `Problem.solve` returns a `Solution`.

The user maths are supplied as Python callables over CasADi symbols. The expected callables and their signatures are listed in Table 9.1; any that are not set default to the natural empty value (no path constraints, a zero cost term, and so on). In every signature `x`, `u` and `p` are the CasADi vectors of the states, controls and static parameters, `t` is the time, and `xi`, `xf` are the initial and final state vectors.

Attribute	Signature	Returns
`dynamics`	`f(x,u,p,t)`	state derivative $\dot{x}$ (length `nstates`)
`path`	`g(x,u,p,t)`	path constraint vector (length `npath`)
`integrand`	`L(x,u,p,t)`	Lagrange (running) cost term, scalar
`endpoint`	`phi(xi,xf,p,t0,tf)`	Mayer (terminal) cost term, scalar
`events`	`e(xi,xf,p,t0,tf)`	boundary-event vector (length `nevents`)
`observation`	`h(x,u,p,t)`	observed outputs (length `nobserved`)

Table 9.1: The user-maths callables set on a phase. Each is a plain Python function (often a `lambda`) written in CasADi. Only `dynamics` is normally required; the rest are optional and `observation` is used only for estimation (Section 9.6).

Bounds are set through `ph.bounds`: `lower` and `upper` each carry `states`, `controls`, `events`, `path` and `parameters` (Python lists), while `ph.bounds.t0` and `ph.bounds.tf` are `(lower, upper)` tuples for the initial and final times. The initial guess is set through `ph.guess` (`states`, `controls`, `time` and `parameters` as NumPy arrays, the state and control guesses shaped as `nstates`$\times N$ and `ncontrols`$\times N$), and `ph.nodes` is the list of node counts that seeds the mesh.

The complete single-phase Bryson–Denham problem reads as follows.

```
import numpy as np, casadi as ca
import psopt

prob = psopt.Problem(name="bryson_denham")
ph = prob.add_phase(nstates=3, ncontrols=1, nevents=5)
ph.nodes = [50]

# user maths -- pure CasADi, traced and CppAD-taped under the hood
ph.dynamics = lambda x, u, p, t: ca.vertcat(x[1], u[0], 0.5*u[0]**2)
ph.endpoint = lambda xi, xf, p, t0, tf: xf[2]                    # minimise x3(
    ↪ tf)
```

```
ph.events   = lambda xi, xf, p, t0, tf: ca.vertcat(xi[0], xi[1], xi[2], xf
    ↪ [0], xf[1])

ph.bounds.lower.states   = [0.0, -10.0, -10.0]
ph.bounds.upper.states   = [1.0/9.0, 10.0, 10.0]
ph.bounds.lower.controls = [-10.0]
ph.bounds.upper.controls = [10.0]
ph.bounds.lower.events   = [0.0, 1.0, 0.0, 0.0, -1.0]
ph.bounds.upper.events   = [0.0, 1.0, 0.0, 0.0, -1.0]
ph.bounds.t0 = (0.0, 0.0)
ph.bounds.tf = (0.0, 50.0)

x0 = np.zeros((3, 50))
x0[1, :] = np.linspace(1.0, -1.0, 50)
ph.guess.states   = x0
ph.guess.controls = np.zeros((1, 50))
ph.guess.time     = np.linspace(0.0, 0.5, 50).reshape(1, 50)

alg = psopt.Algorithm(collocation_method="Legendre", nlp_iter_max=1000,
    ↪ nlp_tolerance=1e-6)
sol = prob.solve(alg)

print(sol.objective)        # 3.9995386676223115
```

The returned `Solution` carries `objective`, the `states` and `controls` arrays (shaped `nstates`$\times N$ and `ncontrols`$\times N$), the `time` vector, and, for estimation problems, the `parameters` estimates. A small number of guess helpers (`psopt.ramp`, `psopt.const` and `psopt.times`) are provided for building these arrays compactly.

9.5 Multiphase problems and linkages

A multiphase problem is built by adding several phases and linking consecutive ones. `link_phases(a, b)` enforces continuity of every state and of time across the boundary between phases `a` and `b`; an optional `jumps` dictionary subtracts a fixed offset from a chosen state's continuity residual, which expresses a discrete jump such as the mass jettisoned between launch stages:

```
prob = psopt.Problem(name="launch")
p1 = prob.add_phase(nstates=7, ncontrols=3, nevents=8, npath=1)
p2 = prob.add_phase(nstates=7, ncontrols=3,           npath=1)
# ... set maths, bounds and guess on each phase ...
prob.link_phases(1, 2, jumps={6: m_dry})   # state index 6 (mass) drops by
    ↪ m_dry at the join
# ... further phases and links ...
sol = prob.solve(alg)
sol.states[0]      # phase-1 state history; sol.time, sol.controls are per-
    ↪ phase lists
```

When more than one phase or any linkage is present, `solve` returns a `MultiSolution` whose `states`, `controls` and `time` are lists with one entry per phase. The four-phase Delta-III ascent in `examples/launch.py` is a complete worked example, including the orbital-element dynamics helper `examples/rv2oe_casadi.py`.

9.6 Parameter estimation

For a parameter-estimation problem a phase carries an `observation` function $h(x, u, p, t)$ returning `nobserved` outputs, together with the sample times `observation_nodes` (shaped 1×`nsamples`) and the measured data `observations` (shaped `nobserved`×`nsamples`). *PSOPT* then minimises the least-squares mismatch between the observed outputs and the model. Static parameters are declared through `nparameters` and bounded through `ph.bounds.lower.parameters` and `ph.bounds.upper.parameters`; their estimates are returned in `sol.parameters`.

```
ph = prob.add_phase(nstates=2, ncontrols=0, nparameters=3, nobserved=2,
    ↪ nsamples=21)
ph.dynamics    = lambda x, u, p, t: ca.vertcat(-(p[0]+p[2])*x[0]**2,
                                              p[0]*x[0]**2 - p[1]*x[1])
ph.observation = lambda x, u, p, t: ca.vertcat(x[0], x[1])   # observe both
    ↪ states
ph.observation_nodes = np.array(tmeas).reshape(1, 21)
ph.observations      = np.vstack([y1meas, y2meas])
# ... bounds and guess ...
sol = prob.solve(alg)
sol.parameters       # estimated kinetic coefficients
```

The full example is `examples/cracking.py`.

9.7 Algorithm options and transcriptions

The `Algorithm` constructor exposes the same options as the C++ `algorithm` structure of Section 6.2.7. The core options `collocation_method`, `nlp_method`, `derivatives`, `scaling`, `nlp_iter_max` and `nlp_tolerance` always take effect. The remaining options default to `None`, meaning "leave the C++ default", and are forwarded to the engine only when set. They cover:

- the *hp*-adaptive mesh refinement of Section 2.3 (`mesh_refinement`, `mr_max_iterations`, `ode_tolerance`, `mr_max_growth_factor`, `mr_min_order`, `mr_max_order`);
- the integrated-residual transcription and Nie–Kerrigan flexible-order representation of Chapter 3 (`transcription_method`, `ir_residual_nodes`, `ir_regularization`, `ir_objective`, `ir_residual_bound`, `ir_dair`, `ir_dair_delta_factor`, `ir_local_order`).

Each option has exactly the meaning documented for its C++ counterpart, so the Python interface reaches mesh refinement and the integrated-residual transcription with no additional machinery; the examples `examples/bryson_mesh.py` and `examples/bryson_ir.py` exercise them directly.

9.8 Examples and validation

The examples distributed with the binding each reproduce the objective of their hand-written C++ counterpart, as summarised in Table 9.2. Taken together they exercise single- and multiphase problems, phase linkages, parameter estimation, the integrated-residual transcription and *hp*-adaptive mesh refinement.

Example	**Exercises**	**Objective (C++ baseline)**
`bryson_denham.py`	single phase	3.999539 (bit-identical)
`launch.py`	four phases, linkages (Delta-III)	-7529.66 (within NLP tolerance)
`cracking.py`	parameters + observation	4.3195×10^{-3} (bit-identical)
`bryson_ir.py`	integrated-residual transcription	5.0490×10^{-6}
`bryson_mesh.py`	*hp* mesh refinement ($10 \to 25$ nodes)	3.999997
`rv2oe_casadi.py`	CasADi orbital-element helper	(used by `launch.py`)

Table 9.2: Python-interface examples and the native C++ objectives they reproduce.

Provenance. The Python interface was co-developed with the assistance of an AI system (Anthropic's Claude). All of its results were validated against the corresponding native $\mathcal{PSOPT}$ baselines, as recorded in Table 9.2.

Acknowledgements

The author is very grateful to Philipp Waxweiler from DLR, Germany, for developing the CMake build process for $\mathcal{PSOPT}$ and for providing suggestions for improving the code, including the use of Eigen3.

The author is thankful to Martin Otter from the Institute for Robotics and System Dynamics, DLR, Germany, for kindly allowing the publication of a translated version of the DLR model 2 of the Manutec R3 robot (original Fortran subroutine R3M2SI) with the distribution of $\mathcal{PSOPT}$.

The author is also grateful to Naz Bedrossian from the Draper Laboratory (USA) for facilitating the thesis by S. Bhatt, where the dynamic model of the International Space Station is described.

Finally, the author is indebted to all users who have provided feedback on $\mathcal{PSOPT}$ since its first release.

References

[1] Joel A. E. Andersson, Joris Gillis, Greg Horn, James B. Rawlings, and Moritz Diehl. CasADi – a software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1):1–36, 2019.

[2] D. A. Benson. *A Gauss Pseudospectral Transcription for Optimal Control*. PhD thesis, MIT, Department of Aeronautics and Astronautics, Cambridge, Mass., 2004.

[3] D. A. Benson, G. T. Huntington, T. P. Thorvaldsen, and A. V. Rao. Direct Trajectory Optimization and Costate Estimation via an Orthogonal Collocation Method. *Journal of Guidance, Control, and Dynamics*, 29(6):1435–1440, 2006.

[4] John T. Betts. *Practical Methods for Optimal Control Using Nonlinear Programming*, volume 36 of *Advances in Design and Control*. SIAM, Philadelphia, Pennsylvania, third edition, 2020.

[5] A.E. Bryson and Yu-Chi Ho. *Applied Optimal Control*. Halsted Press, 1975.

[6] R.L. Burden and J.D. Faires. *Numerical Analysis*. Thomson Brooks/Cole, 2005.

[7] C. Canuto, M.Y. Hussaini, A. Quarteroni, and T.A. Zang. *Spectral Methods: Fundamentals in Single Domains*. Springer-Verlag, Berlin, 2006.

[8] C. Canuto, M.Y. Hussaini, A. Quarteroni, and T.A. Zang. *Spectral Methods: Evolution to Complex Geometries and Applications to Fluid Dynamics*. Springer, Heidelberg, Germany, 2007.

[9] C.G. Canuto, M.Y. Hussaini, A. Quarteroni A., and T.A. Zang. *Spectral Methods in Fluid Dynamics*. Springer-Verlag, 1988.

[10] T. F. Coleman and J. J. Moré. Estimation of Sparse Hessian Matrices and Graph Coloring Problems. *Mathematical Programming*, 28(3):243–270, 1984.

[11] A. R. Curtis, M. J. D. Powell, and J. K. Reid. On the Estimation of Sparse Jacobian Matrices. *Journal of the Institute of Mathematics and Applications*, 13:117–120, 1974.

[12] C. L. Darby, W. W. Hager, and A. V. Rao. An hp-Adaptive Pseudospectral Method for Solving Optimal Control Problems. *Optimal Control Applications and Methods*, 32(4):476–502, 2011.

[13] G. Elnagar, M. A. Kazemi, and M. Razzaghi. The Pseudospectral Legendre Method for Discretizing Optimal Control Problems. *IEEE Transactions on Automatic Control*, 40:1793–1796, 1995.

[14] F. Fahroo and I.M. Ross. Costate Estimation by a Legendre Pseudospectral Method. *Journal of Guidance, Control, and Dynamics*, 24:270–277, 2001.

[15] F. Fahroo and I.M. Ross. Costate Estimation by a Legendre Pseudospectral Method. *Journal of Guidance, Control, and Dynamics*, 24:270–277, 2002.

[16] F. Fahroo and I.M. Ross. Direct Trajectory Optimization by a Chebyshev Pseudospectral Method. *Journal of Guidance Control and Dynamics*, 25, 2002.

[17] D. Garg, M. Patterson, W. W. Hager, A. V. Rao, D. A. Benson, and G. T. Huntington. A Unified Framework for the Numerical Solution of Optimal Control Problems Using Pseudospectral Methods. *Automatica*, 46(11):1843–1851, 2010.

[18] D. Garg, M. A. Patterson, C. Francolin, C. L. Darby, G. T. Huntington, W. W. Hager, and A. V. Rao. Direct Trajectory Optimization and Costate Estimation of Finite-Horizon and Infinite-Horizon Optimal Control Problems Using a Radau Pseudospectral Method. *Computational Optimization and Applications*, 49(2):335–358, 2011.

[19] J. Hesthaven, S. Gottlieb, and D. Gottlieb. *Spectral Methods for Time-Dependent Problems*. Cambridge University Press, Cambridge, 2007.

[20] S. Kameswaran and L. T. Biegler. Convergence Rates for Direct Transcription of Optimal Control Problems Using Collocation at Radau Points. *Computational Optimization and Applications*, 41(1):81–126, 2008.

[21] W. Kang and N. Bedrossian. Pseudospectral Optimal Control Theory Makes Debut Flight, Saves nasa $1m in Under Three Hours. *SIAM News*, 40, 2007.

[22] W. Kang, Q. Gong, I. M. Ross, and F. Fahroo. On the Convergence of Nonlinear Optimal Control Using Pseudospectral Methods for Feedback Linearizable Systems. *International Journal of Robust and Nonlinear Control*, 17:1251–1277, 2007.

[23] E. Kostina, M.A. Saunders, and I. Schierle. Computation of covariance matrices for constrained parameter estimation problems using lsqr. Technical Report SOL-2009-1, Standford University, System Optimization Laboratory, 2009.

[24] F. Liu, W. W. Hager, and A. V. Rao. Adaptive Mesh Refinement Method for Optimal Control Using Nonsmoothness Detection and Mesh Size Reduction. *Journal of the Franklin Institute*, 352(10):4081–4106, 2015.

[25] S. Marsili-Libelli, S. Guerrizio, and N. Checchi. Confidence regions of estimated parameters for ecological systems. *Ecological Modelling*, 165:127–146, 2003.

[26] M. P. Neuenhofen and E. C. Kerrigan. An Integral Penalty-Barrier Direct Transcription Method for Optimal Control. In *2020 59th IEEE Conference on Decision and Control (CDC)*, pages 456–463, 2020.

[27] Y. Nie, O. J. Faqir, and E. C. Kerrigan. ICLOCS2: Try this Optimal Control Problem Solver Before you Try the Rest. In *2018 UKACC 12th International Conference on Control (CONTROL)*, pages 336–336. IEEE, 2018.

[28] Y. Nie and E. C. Kerrigan. Solving Dynamic Optimization Problems to a Specified Accuracy: An Alternating Approach Using Integrated Residuals. *IEEE Transactions on Automatic Control*, 68(1):548–555, 2023.

[29] L. Nita, E. C. Kerrigan, E. M. G. Vila, and Y. Nie. Solving Optimal Control Problems with Non-smooth Solutions Using an Integrated Residual Method and Flexible Mesh. In *2022 IEEE 61st Conference on Decision and Control (CDC)*, pages 1211–1216, 2022.

[30] M. A. Patterson, W. W. Hager, and A. V. Rao. A ph Mesh Refinement Method for Optimal Control. *Optimal Control Applications and Methods*, 36(4):398–421, 2015.

[31] J.A. Pietz. Pseudospectral Collocation Methods for the Direct transcription of Optimal Control Problems. Master's thesis, Rice University, Houston, Texas, 2003.

[32] M. J. D. Powell and Ph. L. Toint. On the Estimation of Sparse Hessian Matrices. *SIAM Journal on Numerical Analysis*, 16(6):1060–1074, 1979.

[33] I.M. Ross and F. Fahroo. Pseudospectral Knotting Methods for Solving Nonsmooth Optimal Control Problems. *Journal of Guidance Control and Dynamics*, 27:397–405, 2004.

[34] S. Sager. *Numerical Methods for Mixed-Integer Optimal Control Problems.* PhD thesis, University of Heidelberg, Heidelberg, Germany, 2005. Published by Der andere Verlag, Tönning, Lübeck, Marburg.

[35] S. Sager. A Benchmark Library of Mixed-Integer Optimal Control Problems. In J. Lee and S. Leyffer, editors, *Mixed Integer Nonlinear Programming*, volume 154 of *The IMA Volumes in Mathematics and its Applications*, pages 631–670. Springer, 2012.

[36] S. Sager, H. G. Bock, and M. Diehl. The Integer Approximation Error in Mixed-Integer Optimal Control. *Mathematical Programming*, 133(1–2):1–23, 2012.

[37] K. Schittkowski. *Numerical Data Fitting in Dynamical Systems.* Kluwer Academic Publishers, Dordrecht, The Netherlands, 2002.

[38] Lloyd N. Trefethen. *Spectral Methods in MATLAB.* SIAM, Philadelphia, 2000.

[39] J. Vlassenbroeck and R. Van Doreen. A Chebyshev Technique for Solving Nonlinear Optimal Control Problems. *IEEE Transactions on Automatic Control*, 33:333–340, 1988.

Licensing Agreement

The software package $\mathcal{PSOPT}$ is distributed under the GNU Lesser General Public License version 2.1. Users of the software must abide by the terms of the license.

```
complete object files to the recipients, so that they can relink them
with the library after making changes to the library and recompiling
it.  And you must show them these terms so they know their rights.

  We protect your rights with a two-step method: (1) we copyright the
library, and (2) we offer you this license, which gives you legal
permission to copy, distribute and/or modify the library.

  To protect each distributor, we want to make it very clear that
there is no warranty for the free library.  Also, if the library is
modified by someone else and passed on, the recipients should know
that what they have is not the original version, so that the original
author's reputation will not be affected by problems that might be
introduced by others.

  Finally, software patents pose a constant threat to the existence of
any free program.  We wish to make sure that a company cannot
effectively restrict the users of a free program by obtaining a
restrictive license from a patent holder.  Therefore, we insist that
any patent license obtained for a version of the library must be
consistent with the full freedom of use specified in this license.

  Most GNU software, including some libraries, is covered by the
ordinary GNU General Public License.  This license, the GNU Lesser
General Public License, applies to certain designated libraries, and
is quite different from the ordinary General Public License.  We use
this license for certain libraries in order to permit linking those
libraries into non-free programs.

  When a program is linked with a library, whether statically or using
a shared library, the combination of the two is legally speaking a
combined work, a derivative of the original library.  The ordinary
General Public License therefore permits such linking only if the
entire combination fits its criteria of freedom.  The Lesser General
Public License permits more lax criteria for linking other code with
the library.

  We call this license the "Lesser" General Public License because it
does Less to protect the user's freedom than the ordinary General
Public License.  It also provides other free software developers Less
of an advantage over competing non-free programs.  These disadvantages
are the reason we use the ordinary General Public License for many
libraries.  However, the Lesser license provides advantages in certain
special circumstances.

  For example, on rare occasions, there may be a special need to
encourage the widest possible use of a certain library, so that it becomes
a de-facto standard.  To achieve this, non-free programs must be
allowed to use the library.  A more frequent case is that a free
library does the same job as widely used non-free libraries.  In this
case, there is little to gain by limiting the free library to free
software only, so we use the Lesser General Public License.

  In other cases, permission to use a particular library in non-free
programs enables a greater number of people to use a large body of
free software.  For example, permission to use the GNU C Library in
non-free programs enables many more people to use the whole GNU
operating system, as well as its variant, the GNU/Linux operating
system.

  Although the Lesser General Public License is Less protective of the
```

```
users' freedom, it does ensure that the user of a program that is
linked with the Library has the freedom and the wherewithal to run
that program using a modified version of the Library.

  The precise terms and conditions for copying, distribution and
modification follow.  Pay close attention to the difference between a
"work based on the library" and a "work that uses the library".  The
former contains code derived from the library, whereas the latter must
be combined with the library in order to run.

    GNU LESSER GENERAL PUBLIC LICENSE
   TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION

  0. This License Agreement applies to any software library or other
program which contains a notice placed by the copyright holder or
other authorized party saying it may be distributed under the terms of
this Lesser General Public License (also called "this License").
Each licensee is addressed as "you".

  A "library" means a collection of software functions and/or data
prepared so as to be conveniently linked with application programs
(which use some of those functions and data) to form executables.

  The "Library", below, refers to any such software library or work
which has been distributed under these terms.  A "work based on the
Library" means either the Library or any derivative work under
copyright law: that is to say, a work containing the Library or a
portion of it, either verbatim or with modifications and/or translated
straightforwardly into another language.  (Hereinafter, translation is
included without limitation in the term "modification".)

  "Source code" for a work means the preferred form of the work for
making modifications to it.  For a library, complete source code means
all the source code for all modules it contains, plus any associated
interface definition files, plus the scripts used to control compilation
and installation of the library.

  Activities other than copying, distribution and modification are not
covered by this License; they are outside its scope.  The act of
running a program using the Library is not restricted, and output from
such a program is covered only if its contents constitute a work based
on the Library (independent of the use of the Library in a tool for
writing it).  Whether that is true depends on what the Library does
and what the program that uses the Library does.

  1. You may copy and distribute verbatim copies of the Library's
complete source code as you receive it, in any medium, provided that
you conspicuously and appropriately publish on each copy an
appropriate copyright notice and disclaimer of warranty; keep intact
all the notices that refer to this License and to the absence of any
warranty; and distribute a copy of this License along with the
Library.

  You may charge a fee for the physical act of transferring a copy,
and you may at your option offer warranty protection in exchange for a
fee.

  2. You may modify your copy or copies of the Library or any portion
of it, thus forming a work based on the Library, and copy and
distribute such modifications or work under the terms of Section 1
above, provided that you also meet all of these conditions:
```

```
    a) The modified work must itself be a software library.

    b) You must cause the files modified to carry prominent notices
    stating that you changed the files and the date of any change.

    c) You must cause the whole of the work to be licensed at no
    charge to all third parties under the terms of this License.

    d) If a facility in the modified Library refers to a function or a
    table of data to be supplied by an application program that uses
    the facility, other than as an argument passed when the facility
    is invoked, then you must make a good faith effort to ensure that,
    in the event an application does not supply such function or
    table, the facility still operates, and performs whatever part of
    its purpose remains meaningful.

    (For example, a function in a library to compute square roots has
    a purpose that is entirely well-defined independent of the
    application.  Therefore, Subsection 2d requires that any
    application-supplied function or table used by this function must
    be optional: if the application does not supply it, the square
    root function must still compute square roots.)

These requirements apply to the modified work as a whole.  If
identifiable sections of that work are not derived from the Library,
and can be reasonably considered independent and separate works in
themselves, then this License, and its terms, do not apply to those
sections when you distribute them as separate works.  But when you
distribute the same sections as part of a whole which is a work based
on the Library, the distribution of the whole must be on the terms of
this License, whose permissions for other licensees extend to the
entire whole, and thus to each and every part regardless of who wrote
it.

Thus, it is not the intent of this section to claim rights or contest
your rights to work written entirely by you; rather, the intent is to
exercise the right to control the distribution of derivative or
collective works based on the Library.

In addition, mere aggregation of another work not based on the Library
with the Library (or with a work based on the Library) on a volume of
a storage or distribution medium does not bring the other work under
the scope of this License.

  3. You may opt to apply the terms of the ordinary GNU General Public
License instead of this License to a given copy of the Library.  To do
this, you must alter all the notices that refer to this License, so
that they refer to the ordinary GNU General Public License, version 2,
instead of to this License.  (If a newer version than version 2 of the
ordinary GNU General Public License has appeared, then you can specify
that version instead if you wish.)  Do not make any other change in
these notices.

  Once this change is made in a given copy, it is irreversible for
that copy, so the ordinary GNU General Public License applies to all
subsequent copies and derivative works made from that copy.

  This option is useful when you wish to copy part of the code of
the Library into a program that is not a library.
```

```
  4. You may copy and distribute the Library (or a portion or
derivative of it, under Section 2) in object code or executable form
under the terms of Sections 1 and 2 above provided that you accompany
it with the complete corresponding machine-readable source code, which
must be distributed under the terms of Sections 1 and 2 above on a
medium customarily used for software interchange.

  If distribution of object code is made by offering access to copy
from a designated place, then offering equivalent access to copy the
source code from the same place satisfies the requirement to
distribute the source code, even though third parties are not
compelled to copy the source along with the object code.

  5. A program that contains no derivative of any portion of the
Library, but is designed to work with the Library by being compiled or
linked with it, is called a "work that uses the Library".  Such a
work, in isolation, is not a derivative work of the Library, and
therefore falls outside the scope of this License.

  However, linking a "work that uses the Library" with the Library
creates an executable that is a derivative of the Library (because it
contains portions of the Library), rather than a "work that uses the
library".  The executable is therefore covered by this License.
Section 6 states terms for distribution of such executables.

  When a "work that uses the Library" uses material from a header file
that is part of the Library, the object code for the work may be a
derivative work of the Library even though the source code is not.
Whether this is true is especially significant if the work can be
linked without the Library, or if the work is itself a library.  The
threshold for this to be true is not precisely defined by law.

  If such an object file uses only numerical parameters, data
structure layouts and accessors, and small macros and small inline
functions (ten lines or less in length), then the use of the object
file is unrestricted, regardless of whether it is legally a derivative
work.  (Executables containing this object code plus portions of the
Library will still fall under Section 6.)

  Otherwise, if the work is a derivative of the Library, you may
distribute the object code for the work under the terms of Section 6.
Any executables containing that work also fall under Section 6,
whether or not they are linked directly with the Library itself.

  6. As an exception to the Sections above, you may also combine or
link a "work that uses the Library" with the Library to produce a
work containing portions of the Library, and distribute that work
under terms of your choice, provided that the terms permit
modification of the work for the customer's own use and reverse
engineering for debugging such modifications.

  You must give prominent notice with each copy of the work that the
Library is used in it and that the Library and its use are covered by
this License.  You must supply a copy of this License.  If the work
during execution displays copyright notices, you must include the
copyright notice for the Library among them, as well as a reference
directing the user to the copy of this License.  Also, you must do one
of these things:

    a) Accompany the work with the complete corresponding
```

```
    machine-readable source code for the Library including whatever
    changes were used in the work (which must be distributed under
    Sections 1 and 2 above); and, if the work is an executable linked
    with the Library, with the complete machine-readable "work that
    uses the Library", as object code and/or source code, so that the
    user can modify the Library and then relink to produce a modified
    executable containing the modified Library.  (It is understood
    that the user who changes the contents of definitions files in the
    Library will not necessarily be able to recompile the application
    to use the modified definitions.)

    b) Use a suitable shared library mechanism for linking with the
    Library.  A suitable mechanism is one that (1) uses at run time a
    copy of the library already present on the user's computer system,
    rather than copying library functions into the executable, and (2)
    will operate properly with a modified version of the library, if
    the user installs one, as long as the modified version is
    interface-compatible with the version that the work was made with.

    c) Accompany the work with a written offer, valid for at
    least three years, to give the same user the materials
    specified in Subsection 6a, above, for a charge no more
    than the cost of performing this distribution.

    d) If distribution of the work is made by offering access to copy
    from a designated place, offer equivalent access to copy the above
    specified materials from the same place.

    e) Verify that the user has already received a copy of these
    materials or that you have already sent this user a copy.

  For an executable, the required form of the "work that uses the
Library" must include any data and utility programs needed for
reproducing the executable from it.  However, as a special exception,
the materials to be distributed need not include anything that is
normally distributed (in either source or binary form) with the major
components (compiler, kernel, and so on) of the operating system on
which the executable runs, unless that component itself accompanies
the executable.

  It may happen that this requirement contradicts the license
restrictions of other proprietary libraries that do not normally
accompany the operating system.  Such a contradiction means you cannot
use both them and the Library together in an executable that you
distribute.

  7. You may place library facilities that are a work based on the
Library side-by-side in a single library together with other library
facilities not covered by this License, and distribute such a combined
library, provided that the separate distribution of the work based on
the Library and of the other library facilities is otherwise
permitted, and provided that you do these two things:

    a) Accompany the combined library with a copy of the same work
    based on the Library, uncombined with any other library
    facilities.  This must be distributed under the terms of the
    Sections above.

    b) Give prominent notice with the combined library of the fact
    that part of it is a work based on the Library, and explaining
    where to find the accompanying uncombined form of the same work.
```

```
  8. You may not copy, modify, sublicense, link with, or distribute
the Library except as expressly provided under this License.  Any
attempt otherwise to copy, modify, sublicense, link with, or
distribute the Library is void, and will automatically terminate your
rights under this License.  However, parties who have received copies,
or rights, from you under this License will not have their licenses
terminated so long as such parties remain in full compliance.

  9. You are not required to accept this License, since you have not
signed it.  However, nothing else grants you permission to modify or
distribute the Library or its derivative works.  These actions are
prohibited by law if you do not accept this License.  Therefore, by
modifying or distributing the Library (or any work based on the
Library), you indicate your acceptance of this License to do so, and
all its terms and conditions for copying, distributing or modifying
the Library or works based on it.

  10. Each time you redistribute the Library (or any work based on the
Library), the recipient automatically receives a license from the
original licensor to copy, distribute, link with or modify the Library
subject to these terms and conditions.  You may not impose any further
restrictions on the recipients' exercise of the rights granted herein.
You are not responsible for enforcing compliance by third parties with
this License.

  11. If, as a consequence of a court judgment or allegation of patent
infringement or for any other reason (not limited to patent issues),
conditions are imposed on you (whether by court order, agreement or
otherwise) that contradict the conditions of this License, they do not
excuse you from the conditions of this License.  If you cannot
distribute so as to satisfy simultaneously your obligations under this
License and any other pertinent obligations, then as a consequence you
may not distribute the Library at all.  For example, if a patent
license would not permit royalty-free redistribution of the Library by
all those who receive copies directly or indirectly through you, then
the only way you could satisfy both it and this License would be to
refrain entirely from distribution of the Library.

If any portion of this section is held invalid or unenforceable under any
particular circumstance, the balance of the section is intended to apply,
and the section as a whole is intended to apply in other circumstances.

It is not the purpose of this section to induce you to infringe any
patents or other property right claims or to contest validity of any
such claims; this section has the sole purpose of protecting the
integrity of the free software distribution system which is
implemented by public license practices.  Many people have made
generous contributions to the wide range of software distributed
through that system in reliance on consistent application of that
system; it is up to the author/donor to decide if he or she is willing
to distribute software through any other system and a licensee cannot
impose that choice.

This section is intended to make thoroughly clear what is believed to
be a consequence of the rest of this License.

  12. If the distribution and/or use of the Library is restricted in
certain countries either by patents or by copyrighted interfaces, the
original copyright holder who places the Library under this License may add
an explicit geographical distribution limitation excluding those countries,
```

```
so that distribution is permitted only in or among countries not thus
excluded.  In such case, this License incorporates the limitation as if
written in the body of this License.

  13. The Free Software Foundation may publish revised and/or new
versions of the Lesser General Public License from time to time.
Such new versions will be similar in spirit to the present version,
but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number.  If the Library
specifies a version number of this License which applies to it and
"any later version", you have the option of following the terms and
conditions either of that version or of any later version published by
the Free Software Foundation.  If the Library does not specify a
license version number, you may choose any version ever published by
the Free Software Foundation.

  14. If you wish to incorporate parts of the Library into other free
programs whose distribution conditions are incompatible with these,
write to the author to ask for permission.  For software which is
copyrighted by the Free Software Foundation, write to the Free
Software Foundation; we sometimes make exceptions for this.  Our
decision will be guided by the two goals of preserving the free status
of all derivatives of our free software and of promoting the sharing
and reuse of software generally.

       NO WARRANTY

  15. BECAUSE THE LIBRARY IS LICENSED FREE OF CHARGE, THERE IS NO
WARRANTY FOR THE LIBRARY, TO THE EXTENT PERMITTED BY APPLICABLE LAW.
EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR
OTHER PARTIES PROVIDE THE LIBRARY "AS IS" WITHOUT WARRANTY OF ANY
KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE
IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE.  THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE
LIBRARY IS WITH YOU.  SHOULD THE LIBRARY PROVE DEFECTIVE, YOU ASSUME
THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

  16. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN
WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY
AND/OR REDISTRIBUTE THE LIBRARY AS PERMITTED ABOVE, BE LIABLE TO YOU
FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR
CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE
LIBRARY (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING
RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A
FAILURE OF THE LIBRARY TO OPERATE WITH ANY OTHER SOFTWARE), EVEN IF
SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH
DAMAGES.

       END OF TERMS AND CONDITIONS
```